

Course Outline

1 Foundations

- Introduction to motor control
- The basics
- DC motors & their control
- Rotating magnetic fields

2 Motor Types Covered in the Workshop

- BLDC
- PMSM
- Induction

3 Control of 3 Phase Motors

- BLDC
- Induction
- PMSM

4 Motor Drive Implementation

- 3-phase inverters
 - Modulation techniques
- C2000 Family of MCUs
 - Architecture
 - Dedicated motor control peripherals
 - Hardware/software and dev tools

5 Lab Exercises

1. Foundations

- **We will start with an easy to understand explanation of the foundations needed to grasp motor control concepts**
 - **Introduction to motor control**
 - **The basics**
 - **DC motors & their control**
 - **Rotating magnetic fields**

Introduction: Ideal Motor Control

- Decoupled control of torque and speed
- Large starting torque
- Max torque at all speeds including zero speed
- Fast response

DC motor

- High speed
- Low maintenance
- No sparks
- Cheap

AC motor

Introduction: Ideal Motor Control

- **DC motor is the gold standard**
 - DC motor inherently has the best performance
 - It is very easy to control
 - but it has commutators
- **AC motors are more robust and can be cheaper**
 - but they are not easy to control and have a nonlinear torque speed relationship (we will talk about this in detail after we have explained some basic mechanical concepts)

If we could force the AC motor to behave like the DC motor we could have the best of both worlds – this is our main objective and the reason why we need field orientation algorithms and big processing power

Foundations

As an embedded systems engineer aiming to design a motor controller, understanding some fundamentals is very useful. However there is no need to delve in to much detail. The following topics will be covered briefly with the audience in mind:

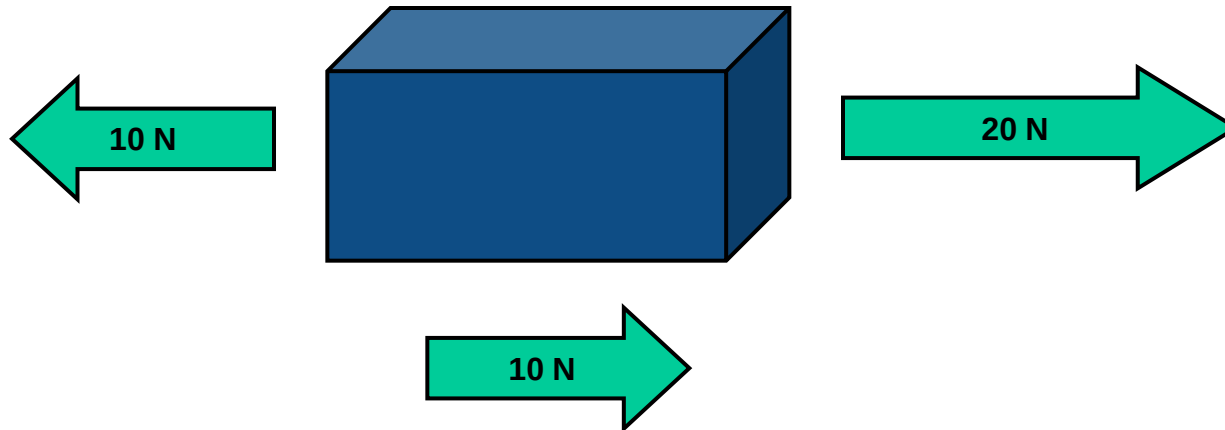
- **Foundation Mechanics**
 - Force/ Work
 - Torque/Power
- **Foundation Electromagnetics**

Foundation Mechanics (for SW Engineers)

Force

Force is the ability to cause an object to accelerate (i.e. Newton's second law) and is measured in Newtons. In mechanical terms, force is the ability to give an object a push or a pull.

Force is a vector and hence the direction of the push or pull must be taken into consideration



Work

When a force that is applied to an object causes the object to move “work” has been done. Work is equal to force x distance moved and is measured in Newton-meters or Joules

Foundation Mechanics

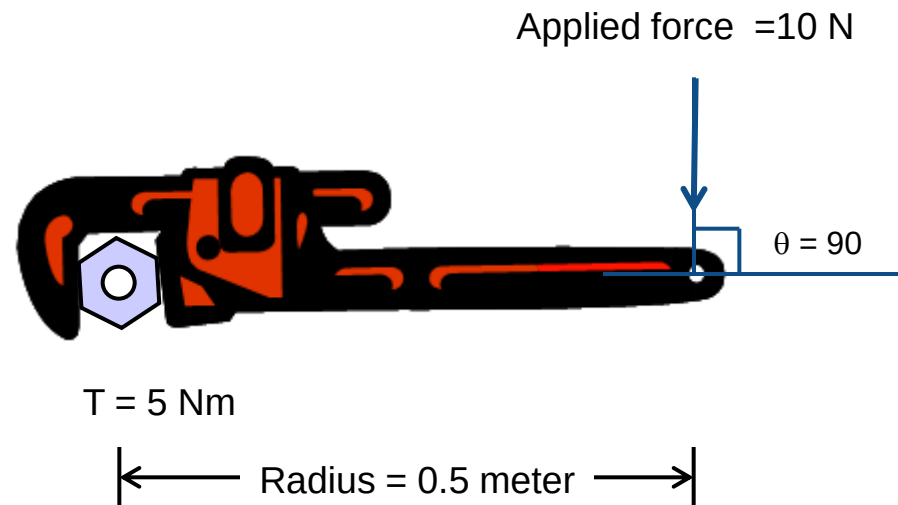
Torque

Torque is analogous to “rotational force” and is measured in Newton-meters. It is the measure of a force's tendency to produce rotation about an axis (e.g. as in a motor).

Assuming that the force is being applied at 90° to the pivot, torque can be calculated by multiplying the amount of force at 90° by the distance to the pivot.

If the force is being applied at an angle other than 90°, then in order to calculate the torque correctly we must resolve along the correct axis. Therefore:

The above equation has some important implications for us. We will explain these in the next slide



$$T = \text{Force} \times \text{distance} \times \sin \theta$$

Foundation Mechanics

$$T = \text{Force} \times \text{distance} \times \sin \theta$$

- The presence of the “sin” term in the torque equation above has some very important implications for us and explains why the behaviour of DC motors differ from AC motors:
 - The “sin” term implies that as long as we apply our constant force at a constant 90° to the pivot, we will have maximum torque
 - This is the case of a DC motor and it is the reason why this type of machine behaves so well
 - If we apply our force at any other constant angle we will not get maximum torque out of our motor, but will still have constant torque
 - If we apply our force at a varying angle then we will get fluctuating torque

Foundation Mechanics

Power

Power is the rate of doing work. Given that work was defined as (force x distance moved), it follows that power is:

$$\frac{\text{Force} \times \text{Distance}}{\text{Time}}$$

The unit of power can be Newton-Meters per Second or Joules per second or Watts

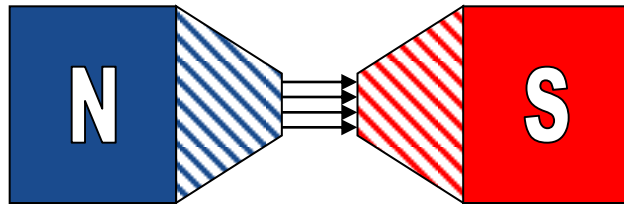
Note that Distance divided by time is the same as velocity!

$$\text{Force} \times \left(\frac{\text{Distance}}{\text{time}} \right) = \text{Force} \times \text{velocity}$$

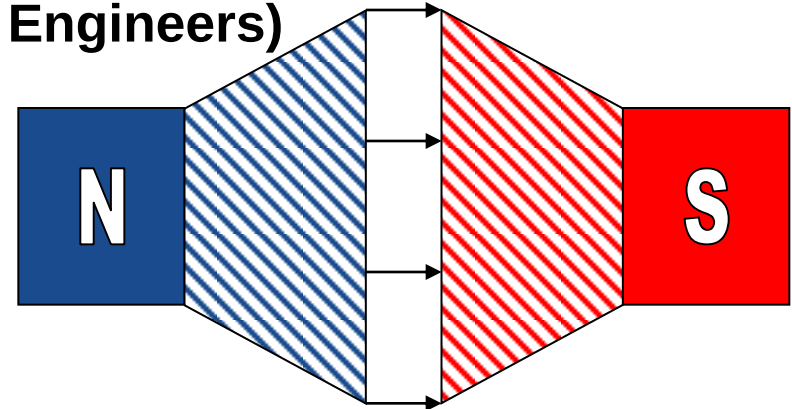
Most importantly for us in the case of rotational forces i.e. motors :

$$\text{Power} = \text{Torque} \times \text{Angular Velocity i.e. } P = T \times \omega$$

Foundation Electromagnetics (for SW Engineers)



A magnet of fixed flux f acting on a small area



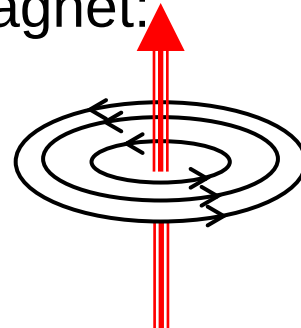
The same magnet acting on a large area

- We measure the amount of magnetic “oomph” that we have in Webers and call this Magnetic Flux ϕ (or just flux)
- Magnetic flux is not the best way of measuring how much work we can do with our magnet as it does not take in to account the area over which the magnetic field lines are acting
 - We need to define another quantity that takes into account the area, we call this “Flux Density” B measured in Teslas
 - Flux density is the amount of flux (magnetic oomph!) acting per unit area:

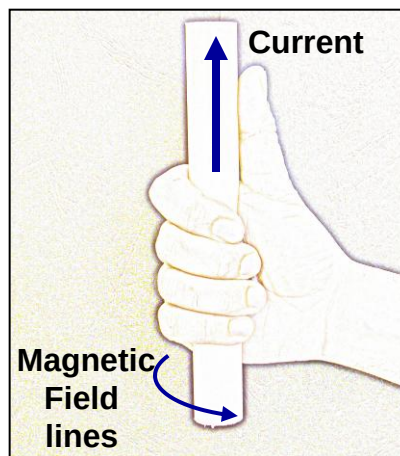
$$B = \frac{\phi}{A} \quad \text{or} \quad \phi = B \times A$$

Foundation Electromagnetics

- A current carrying conductor also possesses a magnetic field just like a permanent magnet:



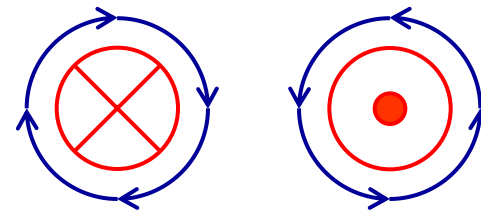
- The direction of the magnetic field lines is given by the right hand curl rule:



- Right Hand Curl Rule**

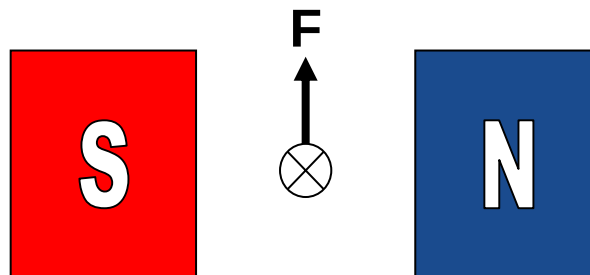
Thumb $\Rightarrow$ direction of current

Other fingers $\Rightarrow$ Field Direction



Foundation Electromagnetics

- When we place a current carrying conductor (which has a magnetic field) in a magnetic field, the two magnetic fields will interact with one another and a “force” is exerted on the conductor:



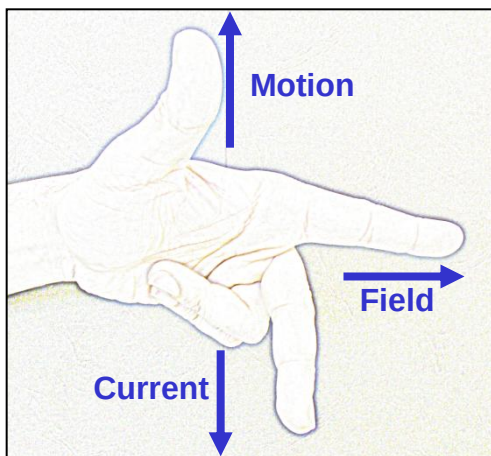
- Assuming that the conductor is at 90° to the magnetic field; the magnitude of the force is given by:

$$F = B I l$$

- We can use “Fleming’s Left-hand Rule” to quickly work out the direction of the force on the current carrying conductor

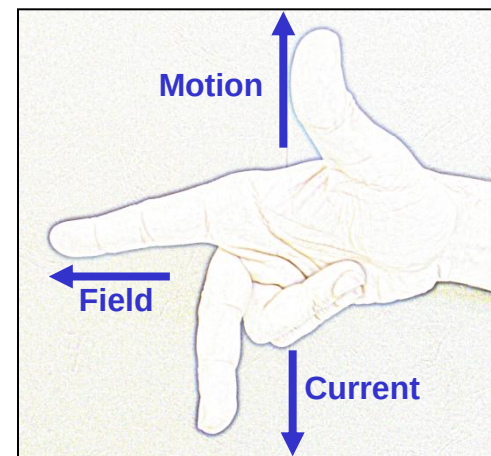
Foundation Electromagnetics

Fleming's Left Hand Rule: Motoring



- **Motoring Rule (Left Hand):**
First finger $\Rightarrow$ **F**ield,
Se**C**ond Finger $\Rightarrow$ **C**urrent,
Thu**M**b $\Rightarrow$ Direction of **M**otion

Fleming's Right Hand Rule: Generating

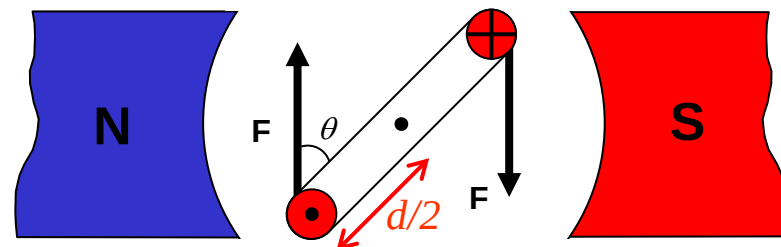


- **Generating Rule (Right Hand):**
First finger $\Rightarrow$ **F**ield,
Thu**M**b $\Rightarrow$ **M**otion
Se**C**ond Finger $\Rightarrow$ Direction of **C**urrent,

In mathematicians' language: Fleming's left hand rule finds the Vector Cross Product of the magnetic field vector and the current vector

Foundation Electromagnetics

- From the previous discussions we can now get a good understanding of how all motors operate
- We know that if we apply a magnetic field to N turns of a current carrying conductor we will produce a force:



$$Force = (B I l) \times N \text{ Newtons}$$

- From the previous slides and the above diagram: substitute

$$T = Force \times distance \times \sin \theta$$

$$l \times d = \text{Area}$$

$$Torque = B I \cancel{l} d N \sin \theta = B A N I \sin \theta$$

DC Motor Operation

- Motor torque equation:

$$\text{Torque} = BAN I \sin \theta$$

- In case of a DC motor θ is always 90°
 - This means that we will always have maximum torque
 - This is due to the action of the commutators – we will explain this shortly
- Finally note that BA = total flux ϕ &
- total flux times the number of turns = flux linkage λ .

In the case of the dc motor $\theta = 90^\circ$ and $BAN = \lambda \therefore$ for a dc motor :

$$\text{Torque} = \lambda_f I_a \quad \text{Nm}$$

Note: The only difference between flux and flux linkage is the number of turns, so rather confusingly these two terms are sometimes used interchangeably

DC Motor Operation

- From previous slides DC motor torque equation:

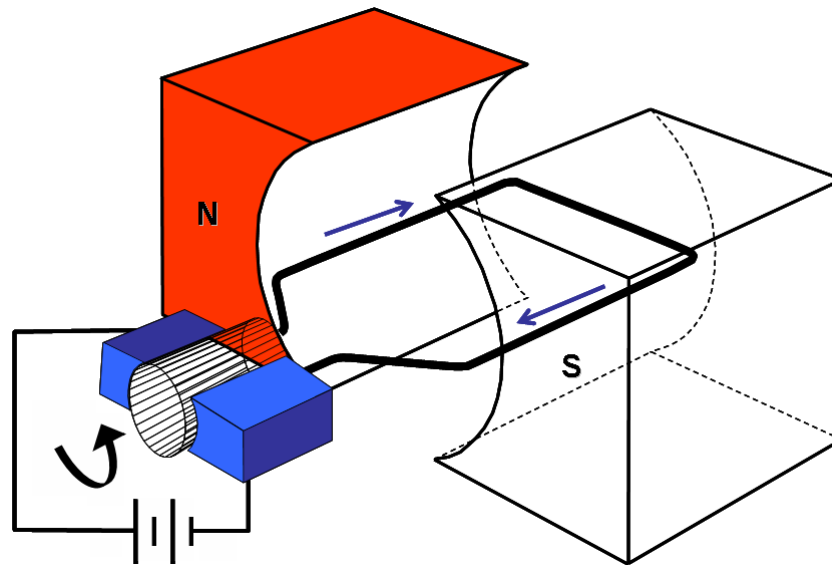
$$\text{Torque} = \lambda_f I_a \quad \text{Nm}$$

- Where λ_f is the total magnetic flux (linkage) produced by the field winding and I_a is the armature current.
 - Therefore if we keep the field constant (which we usually do) then torque of the DC motor is directly proportional to the armature current
 - We always have maximum torque independent of the speed
 - This is why the DC motor performs so well

DC Motor Basics

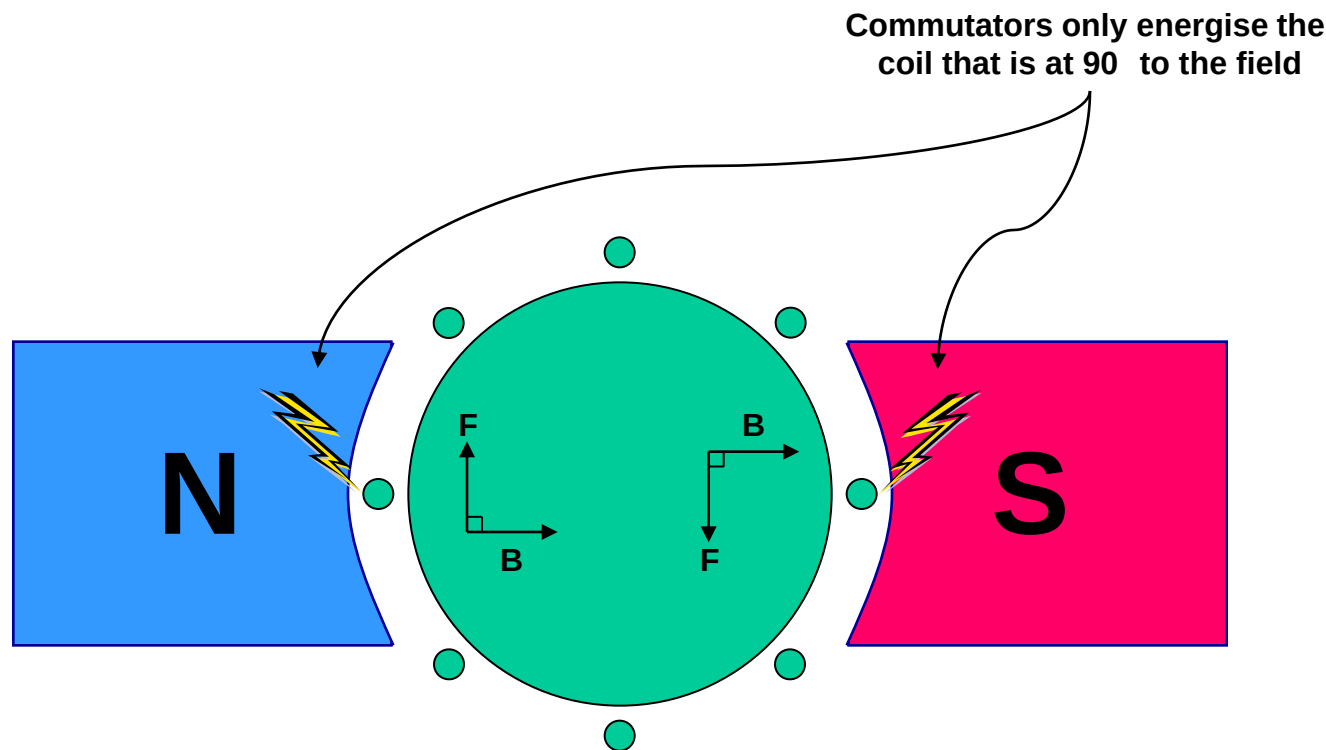
Commutation

Although the function of commutators is well known, what is less known is that DC motors ideal torque-speed characteristics is purely the result of the presence of commutators



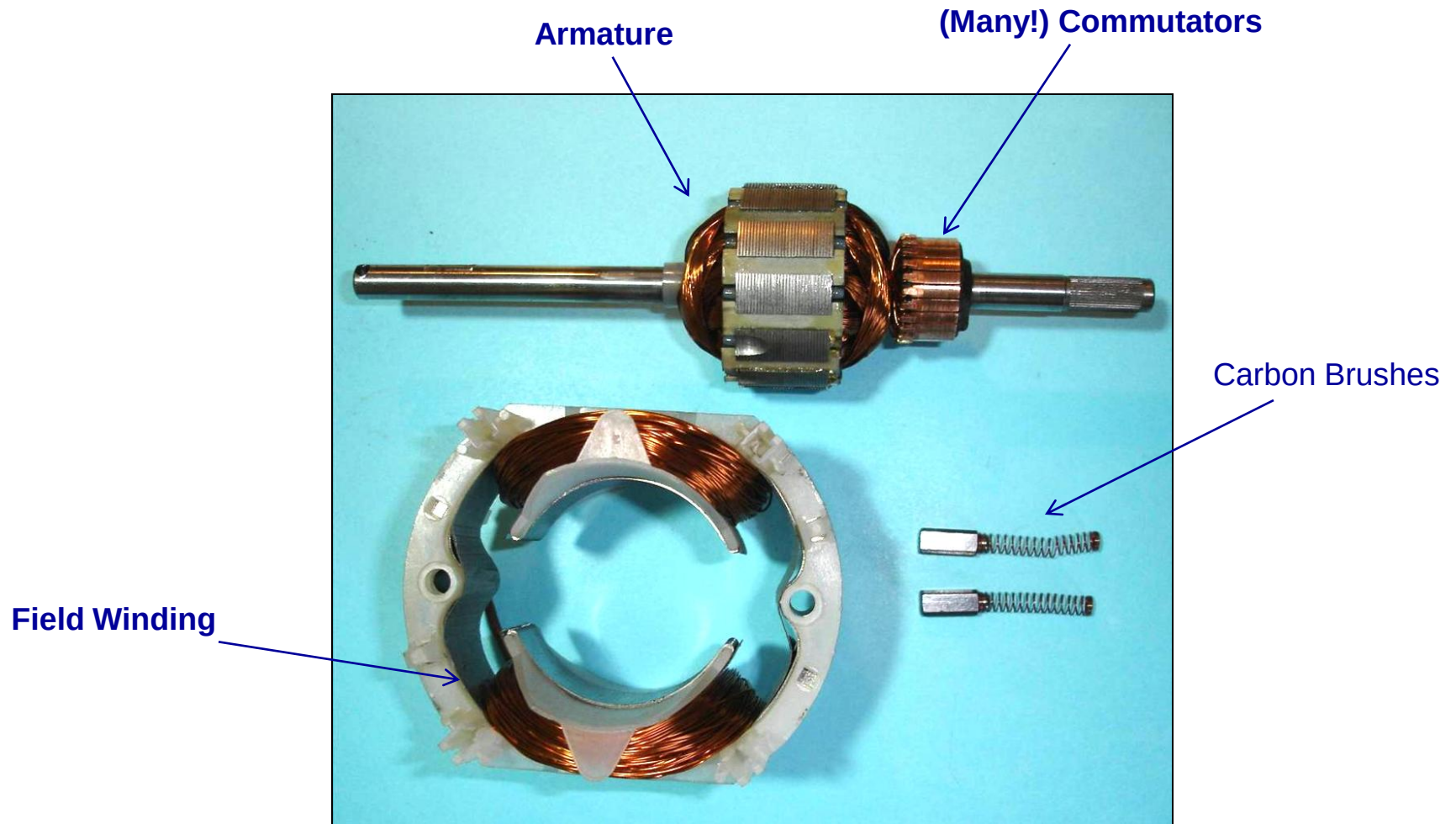
In practice the armature has *many coils with each coil ending in commutators*, this results in maximum decoupled torque generation. We will explain how in the next slide

DC Motor Basics



In simplistic terms, the commutators will only connect the winding that is in an ideal position for maximum torque production. In practice many coils are energised, however they are connected in such a way that their magnetic axis is perpendicular to magnetic axis of the field winding. This is not the case with AC machines.

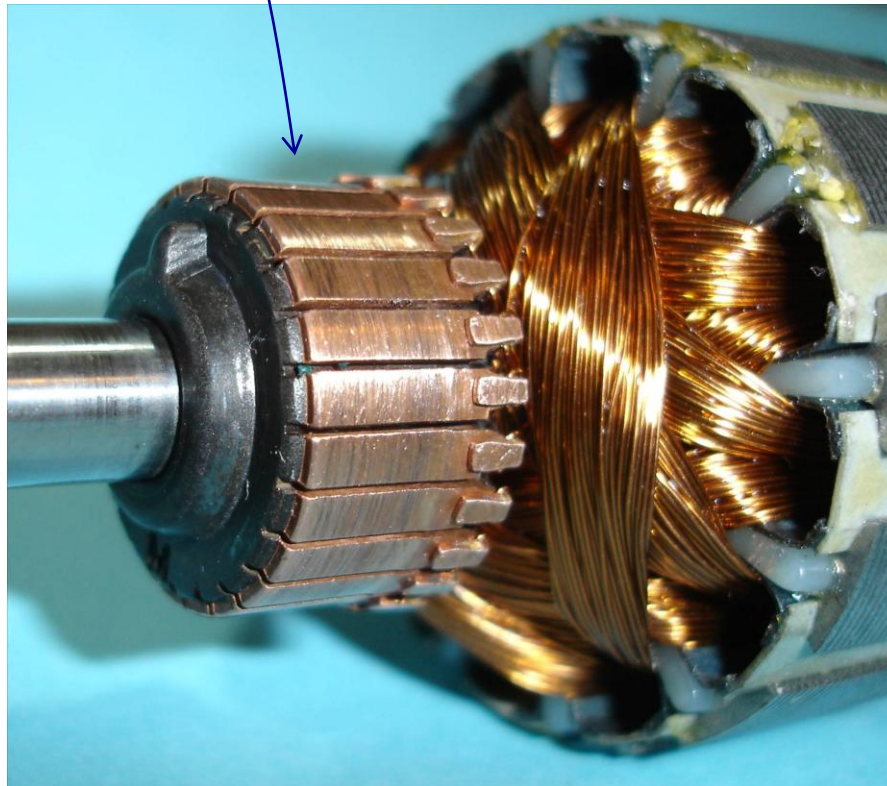
DC Motor Construction



DC Motor Construction

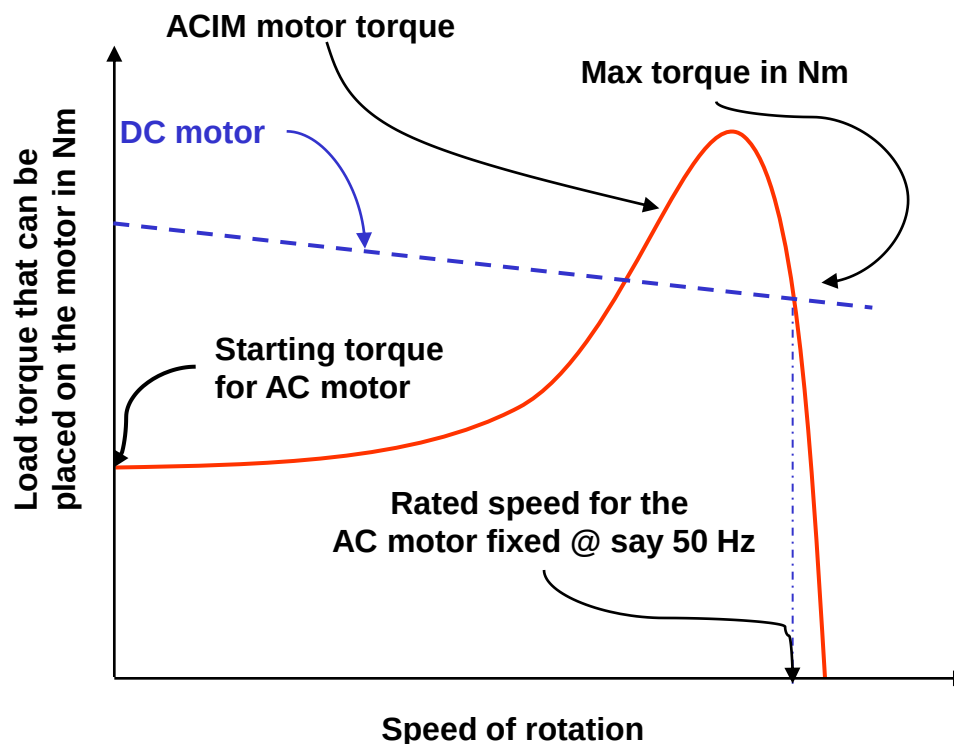
The armature has many windings and commutators.

In practice the coils are energized in such a way that their magnetic axis is perpendicular to the magnetic axis of the field winding



DC Motor Basics

- From the previous discussions, DC motor has an almost ideal torque speed characteristics
- This is not the case with AC motors



Three Phase Motors

- **Advantages**
 - Cheap
 - Brushless
 - High speed
 - Good power to weight ratio

- **Disadvantages**
 - Hard to control
 - Coupled torque and speed

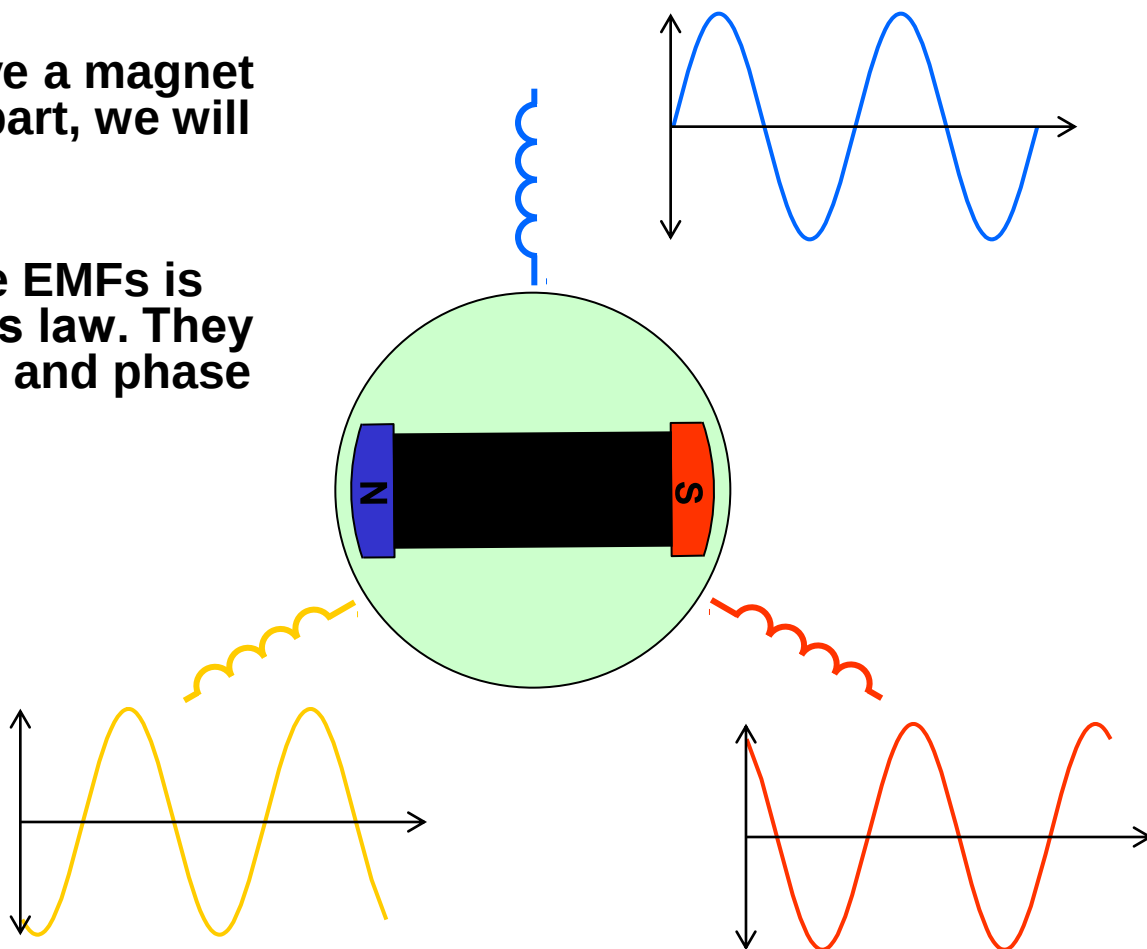
Rotating Magnetic Fields

- If we mechanically move a magnet near three coils 120° apart, we will induce three EMFs
- The magnitude of these EMFs is given by Faraday/Lenz's law. They are sinusoidal in shape and phase shifted by 120°

Therefore, if we excite these three coils with three EMFs 120° apart we will get a rotating magnetic field in the center

This is a 2 pole rotor i.e. for every 1 mechanical revolution there is 1 electrical revolution:

$$\omega_e = \omega_m \times \text{Pole Pairs}$$



Where ω_e = electrical angular frequency
 ω_m = mechanical angular frequency

Rotating Magnetic Fields

- If we had a 4 pole rotor (NSNS), then for every 1 mechanical revolution there would be 2 electrical revolutions:

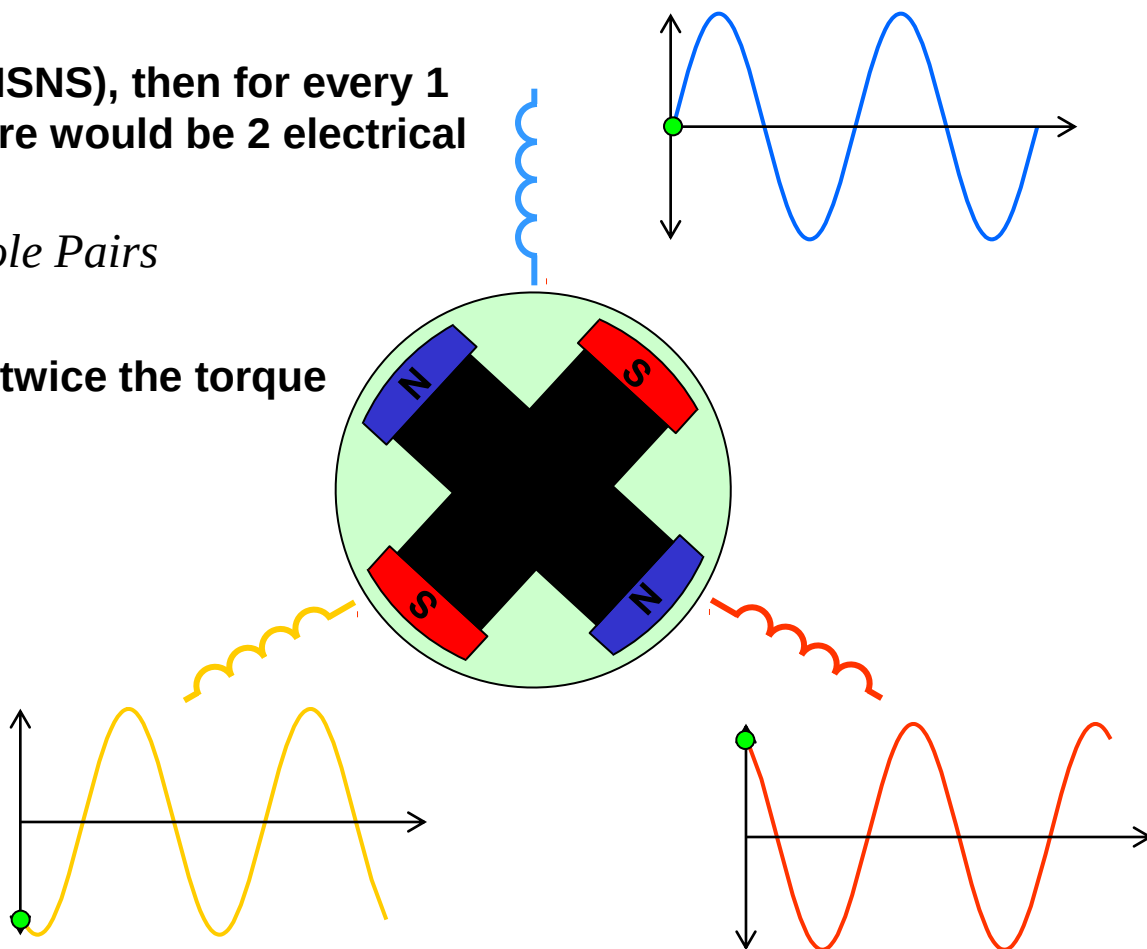
$$\omega_e = \omega_m \times \text{Pole Pairs}$$

- We get half the speed but twice the torque

From previous slides we know that:

$$\text{Power} = \text{Torque} \times \text{Speed}$$

Hence for the same power rating we can see that a 4 pole motor creates twice as much torque but it will travel at half the speed of a 2 pole motor



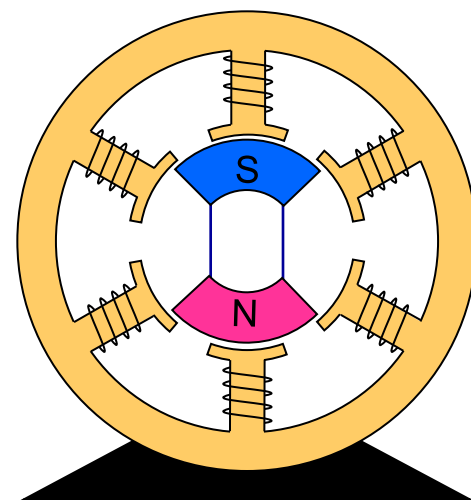
This is a simple permanent magnet synchronous motor

2. Motor Types

- **Beyond the simplest DC motors, we require some processing power for high performance control. We will now discuss the following popular types of 3 phase motors**
 - **BLDC**
 - **PMSM**
 - **Induction**

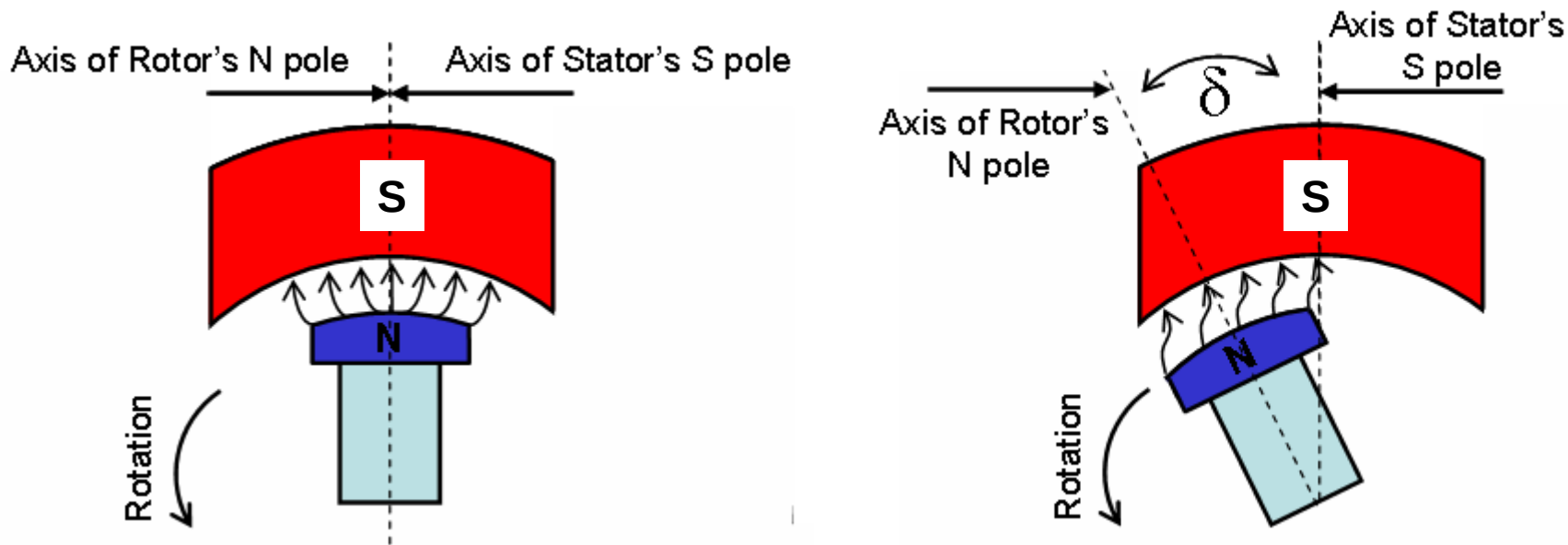
Synchronous Motors: PMSM & BLDC

- From the previous slides we know that if we excite the three stator coils with three-phase voltages we will get a rotating magnetic field at the centre
- All we have to do now to “invent” our PMSM (or BLDC) is to pivot a permanent magnet at the center



- The rotor will always rotate at exactly the same speed as the stator and that is why this type of machine is called “synchronous”

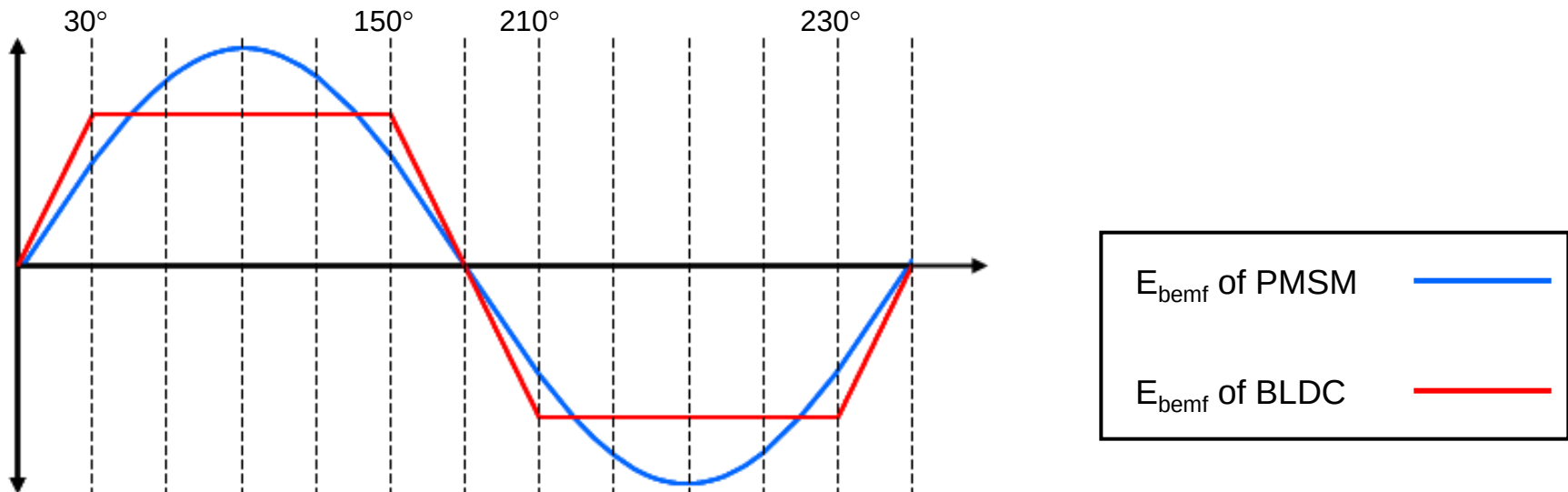
Torque Production in PMSM and BLDC Motors



- Torque is a function of Back-EMF, current and Torque Angle δ
- Without intelligent control as the load on the motor increases so does δ until it reaches 90°
 - Any load beyond that will result in the machine losing synchronisation and fail
- With intelligent control (i.e. field orientation) we can force the torque angle to be 90 degrees at all times
 - This means more torque per amp and therefore better efficiency

The Difference between PMSM & BLDC Motor

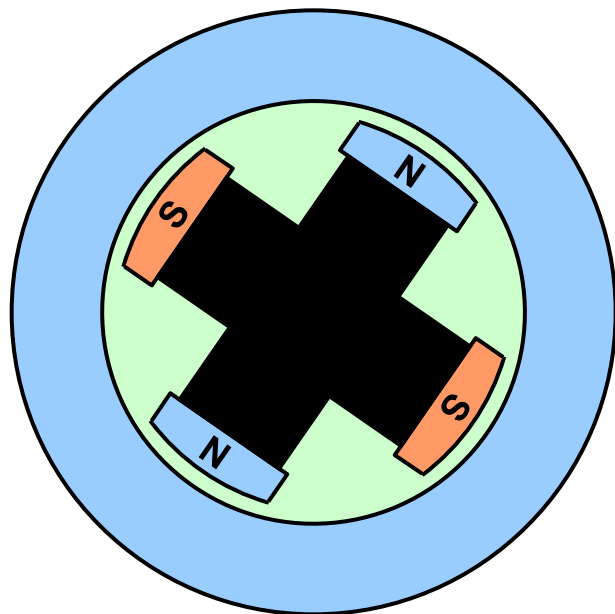
- The Torque is a function of current and Back EMF - E_{bemf}
 - E_{bemf} is a function of how the stator has been wound
- PMSM: stator with distributed windings will have a sinusoidal E_{bemf}
 - In order to achieve constant torque we inject a sinusoidal current
- BLDC: stator with concentrated windings will have a trapezoidal E_{bemf}
 - In order to achieve constant torque we inject a square wave current



Which Synchronous? BLDC vs. PMSM

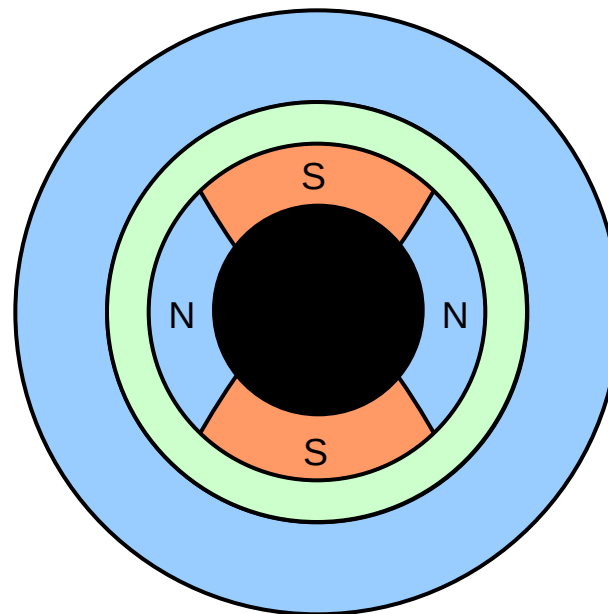
- **BLDC Motors**
 - Easier to control (6 Step) and only DC currents required
 - Torque ripple at commutations
 - Lower cost but poor performance
 - Higher efficiency, higher Torque
 - Needs Hall sensors
 - sensorless is possible
- **PMSM Motors**
 - Very commonly used in servo drives with integrated shaft encoder
 - Sensorless is possible, and usually used for speed control
 - More complex control (needs 3 phase sinusoidal PWM)
 - No torque ripple at commutation
 - Higher cost but high performance

PMSM & BLDC Rotor Types



Salient

- Added reluctance torque
- Easy to make $\therefore$ cheaper



Non-Salient

- Less windage $\therefore$ higher speeds
- More expensive
- More robust as magnets won't fly off!

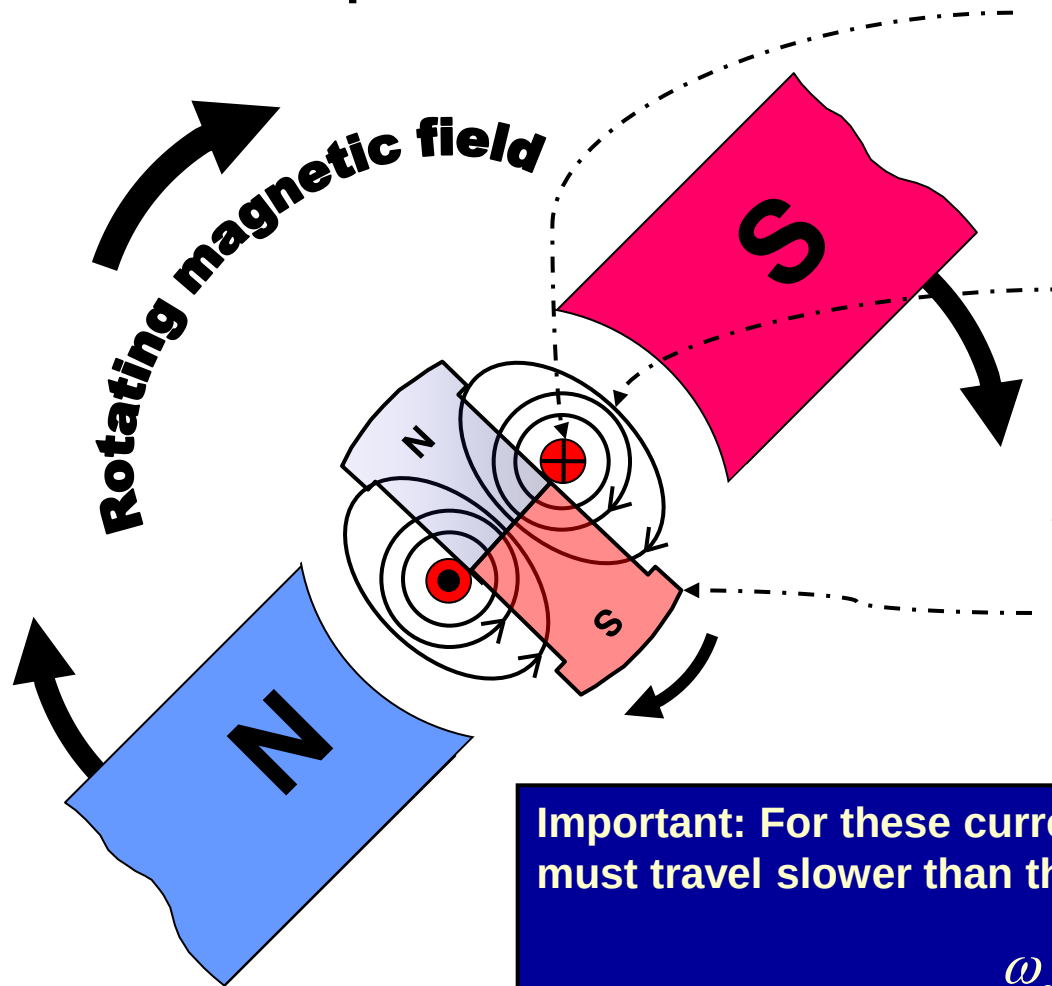
PMSM & BLDC Applications

- **Servo motors**
- **White goods**
- **PC fans**
- **Automotive fans, fuel pumps power steering**
- **Textile and spinning machines**
- **Electric vehicles**
- **Solar pumps**

Induction Motors

- Invented in 1888 by Nikola Tesla
- Elegantly simple and extremely robust and cheap
- The most common type of motor used in industry
 - Used to be used in unglamorous tasks (fixed speed fans / pumps)
 - Increasingly being used for speed control but requires sophisticated control
 - Sensorless control is possible
 - Position control possible but not very common
- Stator winding is the same as PMSM
 - i.e. rotating magnetic field when energized with 3 phase sinusoidal currents
- Rotor consists of aluminum bars in which currents are induced due to the rotating magnetic field

Induction Motor Operation



1. The rotating magnetic field in the stator, induces a current in the rotor
2. This current will have a magnetic field associated with it
3. This magnetic field makes the rotor behave like a magnet which will then follow the stator's rotating magnetic field

Important: For these currents to be induced the rotor must travel slower than the stator; this is called slip:

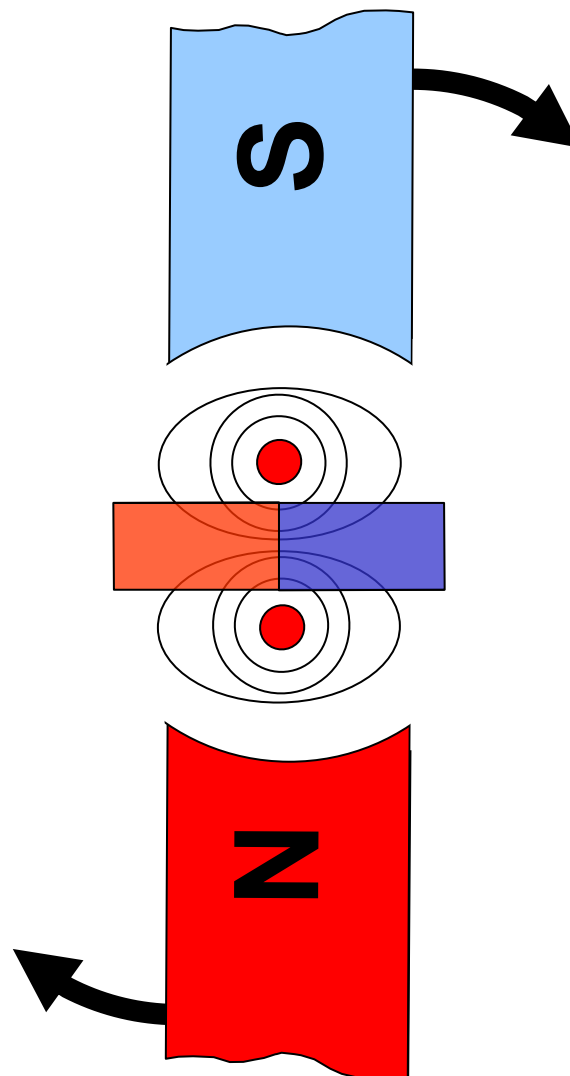
$$s = \frac{\omega_e - \omega_r}{\omega_e}$$

Induction Motor Operation

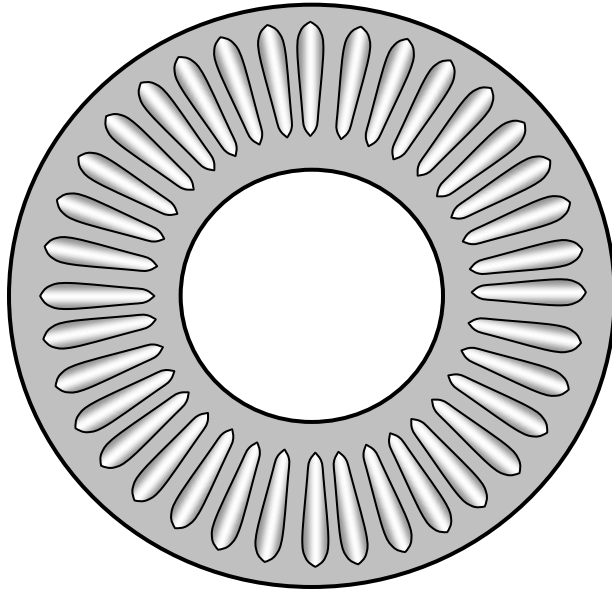
Note 1 - Rotor travels slower than the stator i.e. slip

$$s = \frac{\omega_e - \omega_r}{\omega_e}$$

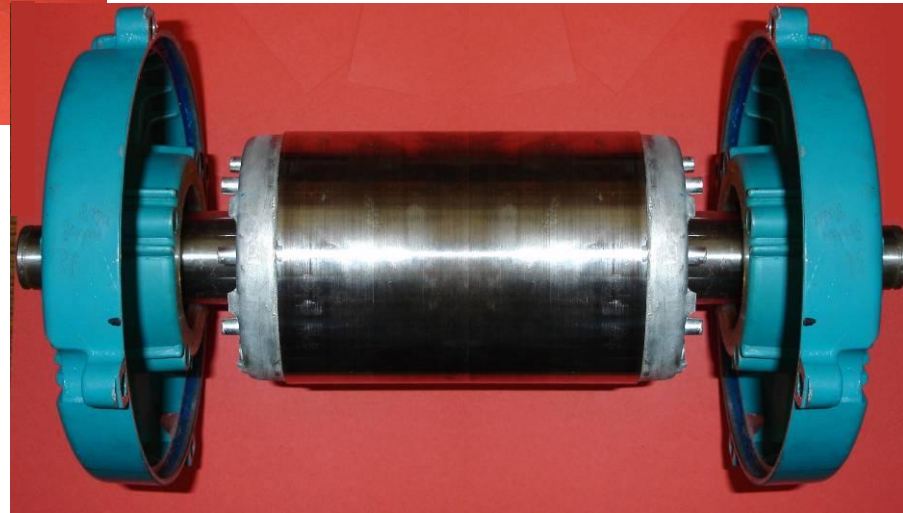
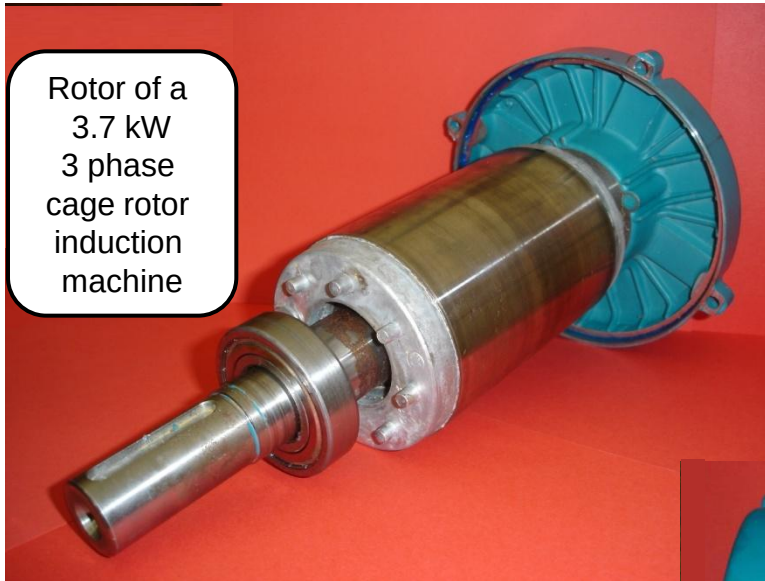
Note 2 - Rotor **MUST** travel slower than the stator or there will be no currents induced in the rotor bars $\therefore$ no torque



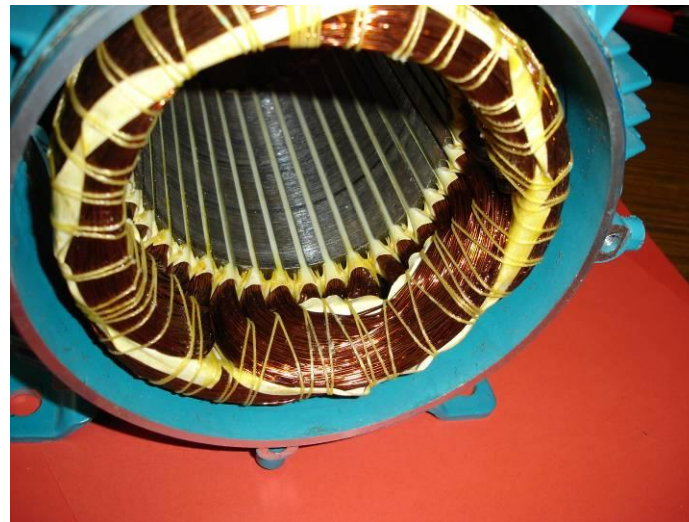
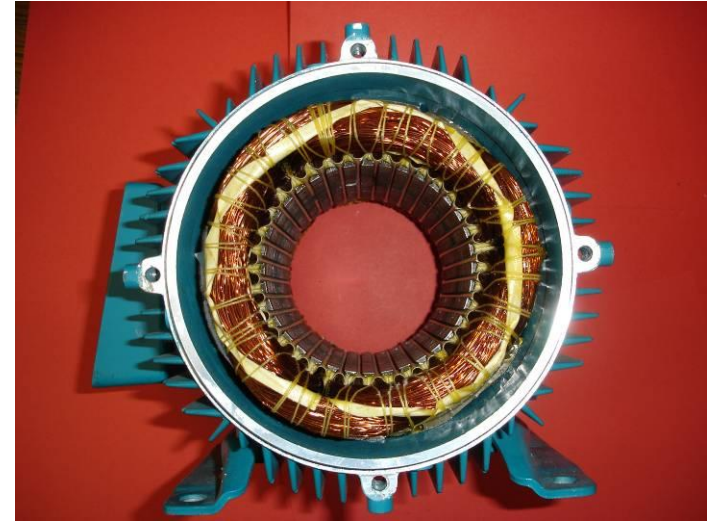
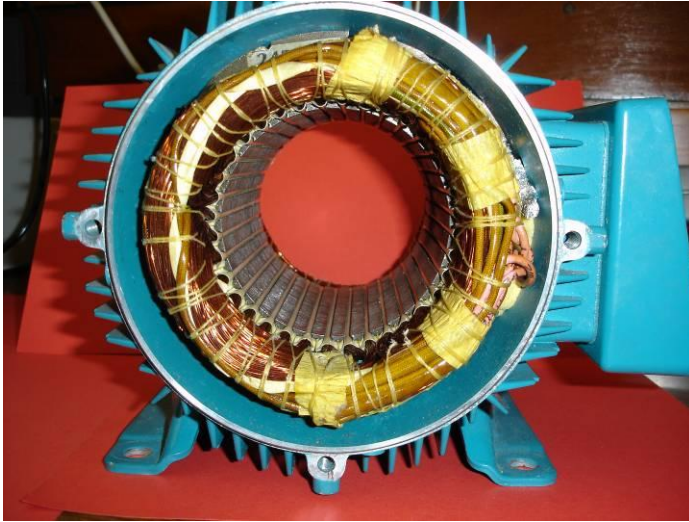
Induction Motor Construction



Induction Motor Construction



Induction Motor Construction



3. Control of 3 Phase Motors

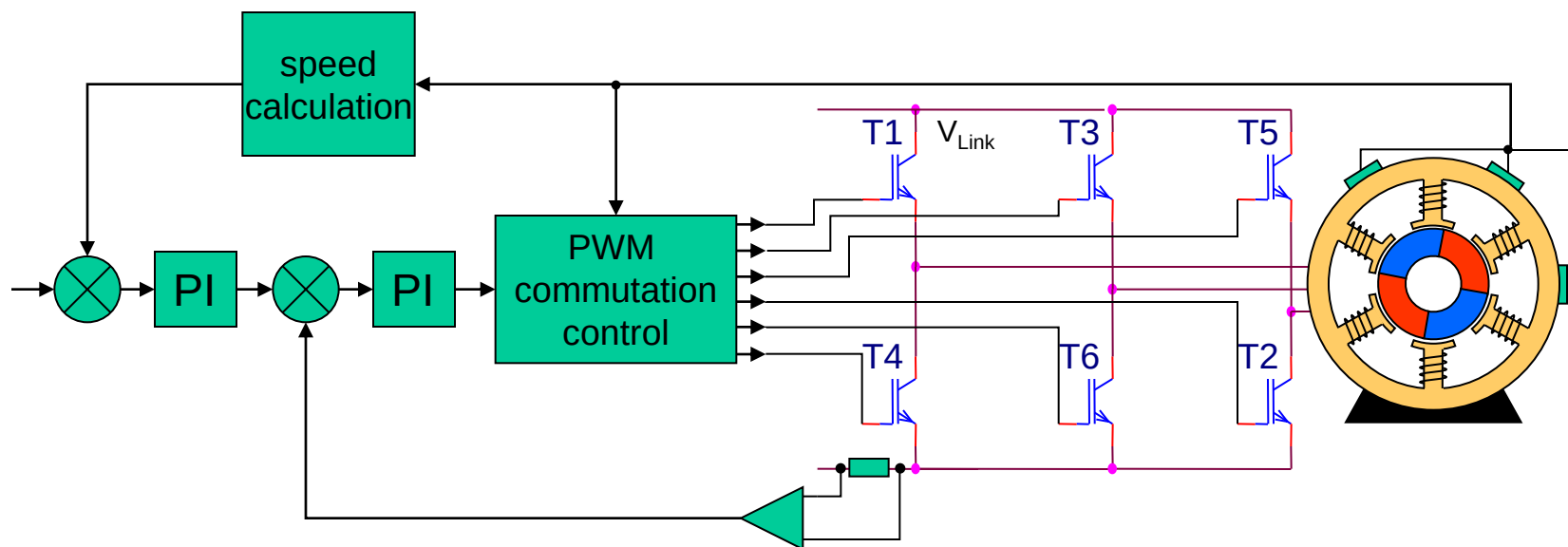
- Now that we have become familiar with various motor types, their operation and their applications, we will discuss intelligent control of these motors for optimum performance
 - BLDC Control
 - Hall Effect
 - Sensorless
 - ACIM Control
 - Scalar
 - Field Orientation
 - PMSM Control

Control of Brushless DC Motors

- In order to keep in synchronism we need to energize the coils at the correct point in time
- Also for maximum torque per amp we need to align the stator magnetic field at 90° to the rotor magnetic field i.e. vector control
 - Hence we need to identify the position of the rotor flux
- Identifying the position of the rotor flux is not very difficult
 - In a BLDC machine this is usually achieved using Hall effect transducers
 - Alternatively provided that the processor has enough bandwidth this can be done in a sensorless manner
- TI's C2000 family is ideally suited for these applications; TI provides complete libraries for both Hall effect sensed and sensorless control of BLDC motors
 - High Voltage Motor Control Kit TMDSHVMTRPFCKIT
 - Low Voltage Multi-Motor Control Kit: DRV8412-C2-KIT
 - Free SW and System Documentation through: www.ti.com/controlSUITE

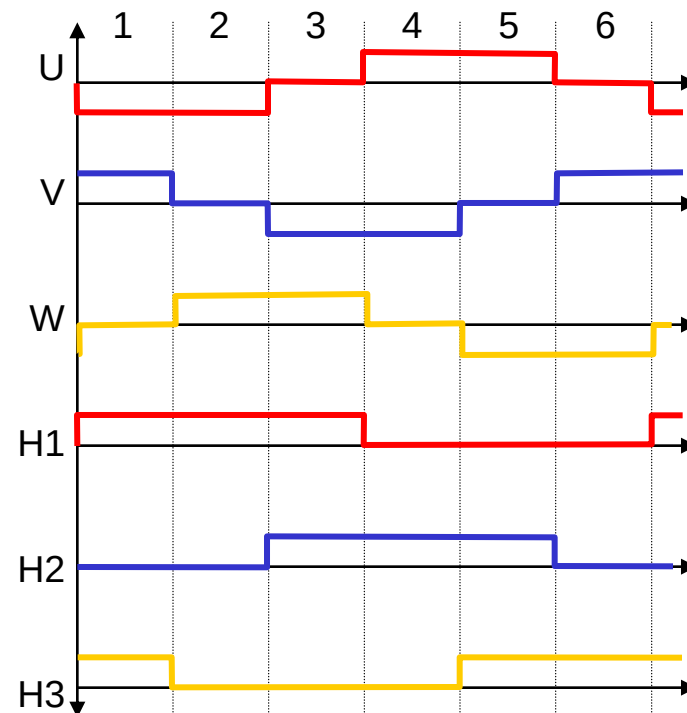
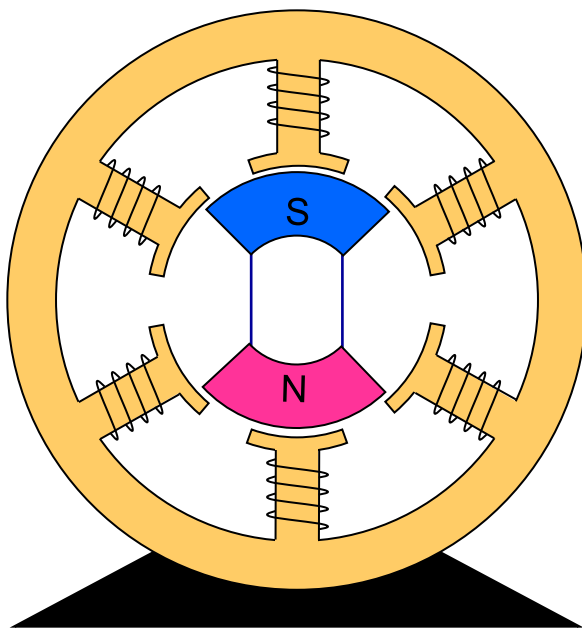
Hall Effect Control of BLDC Motors

- For better performance we use a closed loop system:
 - The Hall sensors and associated electronics will generate the signals necessary for correct commutation
 - Information from the Hall sensors are also used to calculate and feed back the velocity
 - Only two phases conduct at any one time; for current feedback DC-link current is measured and fed back



Hall Effect Control of BLDC Motors

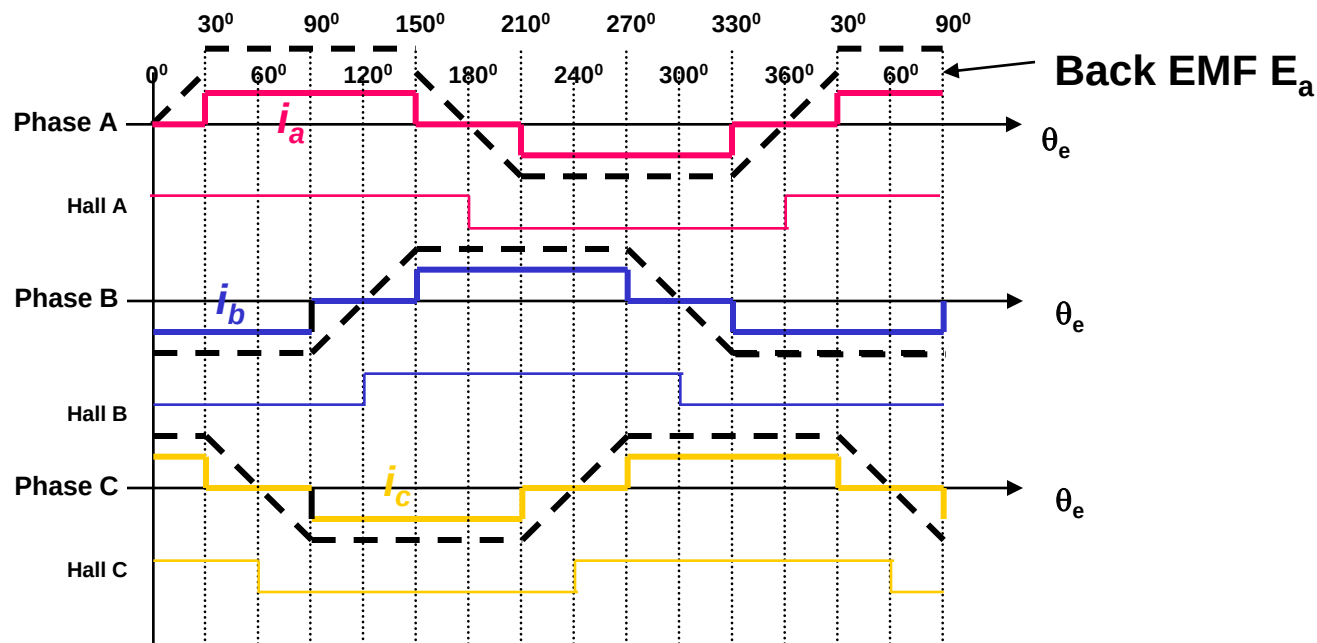
- Only two out of the three phases are energized at any one time
- The phases are energized in a 6 step manner
- The Hall sensors and the associated electronics will generate the signals necessary for correct commutation
- Current is then injected when the $E_{b\text{emf}}$ of each phase has reached its flat portion. This will ensure constant torque



Hall Effect Control of BLDC Motors

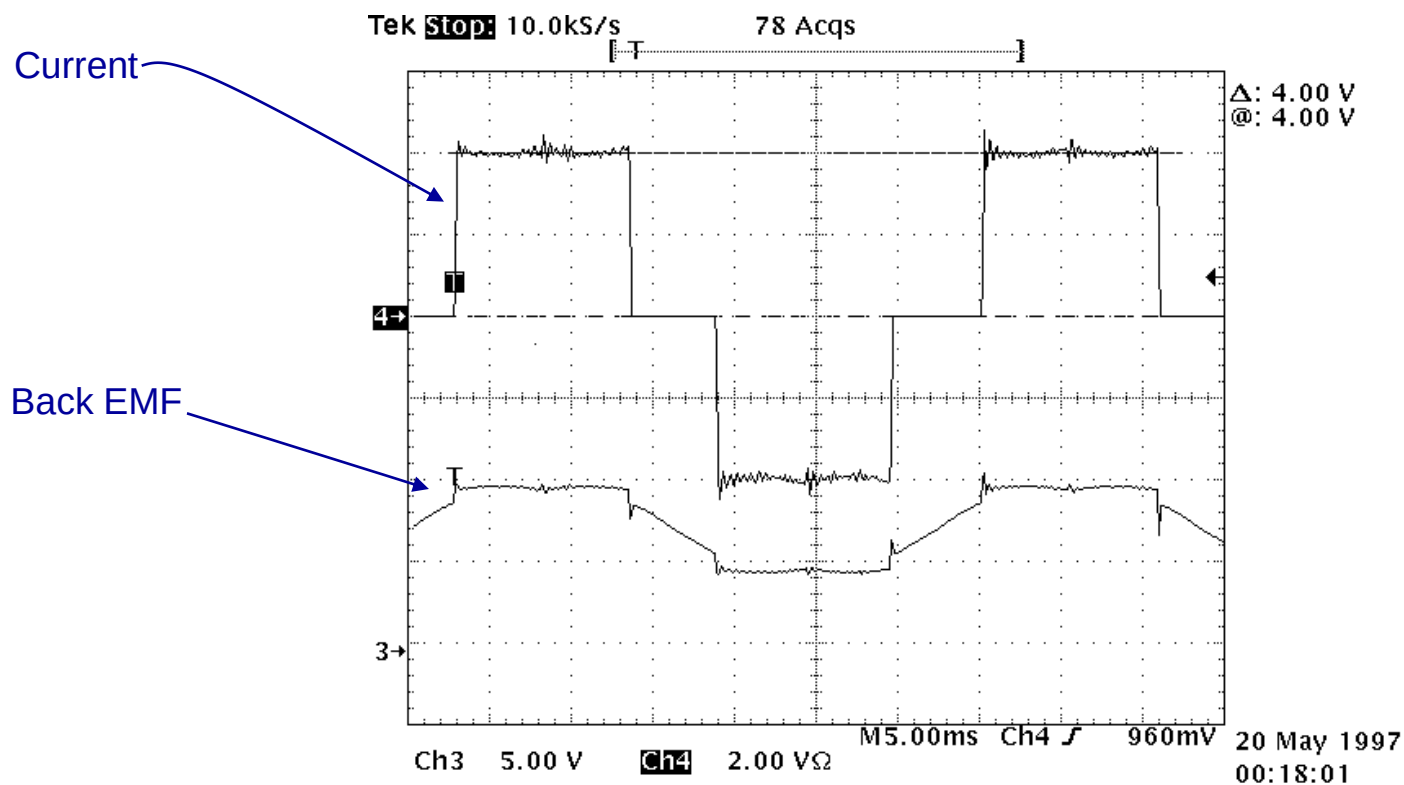
- In the previous slide, we stated that we use the signal from the Hall effect sensor to inject a DC current when the Back EMF reaches its flat region
 - Flat current & flat Bemf $\rightarrow$ Flat torque (i.e. constant torque)
 - No need for complex PWM (i.e. Slow switching)

Back EMF of BLDC Motor



Hall Effect Control of BLDC Motors

- Actual measured current and Back EMF



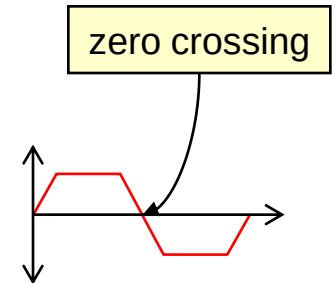
- **Modular DMCLib blocks are connected together in software**
- **www.ti.com/controlSUITE choose High Voltage Kit**



Sensorless Control of BLDC Machines (E_{bemf})

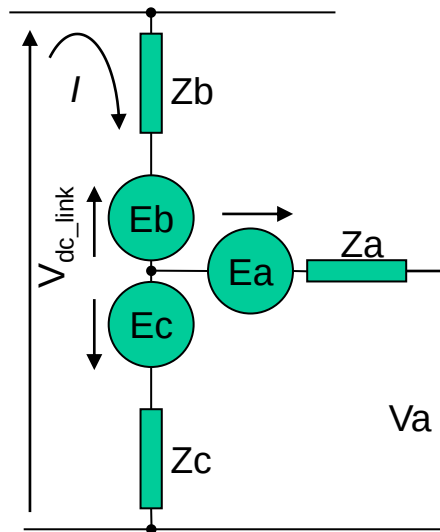
For sensorless control:

- The back-emf E_{bemf} waveform is directly related to the position of the rotor.
- If we could detect the zero crossing of the back-emf waveform we could deduce the position of the rotor
- We will then need to wait for 30° for the E_{bemf} to reach its constant region and then turn on the current in that phase
- The waiting time needed is dependant on the motor speed and can be deduced by continuously measuring the previous E_{bemf} zero crossings.
- Note that this is essentially a feed forward system and hence sudden changes to torque will cause timing errors.
- But $E_{bemf} \propto \omega \therefore$ at low speeds i.e. $\omega \Rightarrow 0$ E_{bemf} will disappear
- Usually operated open loop at low speeds and when E_{bemf} becomes large enough to estimate accurately the loop is closed



Sensorless Control of BLDC Machines (E_{bemf})

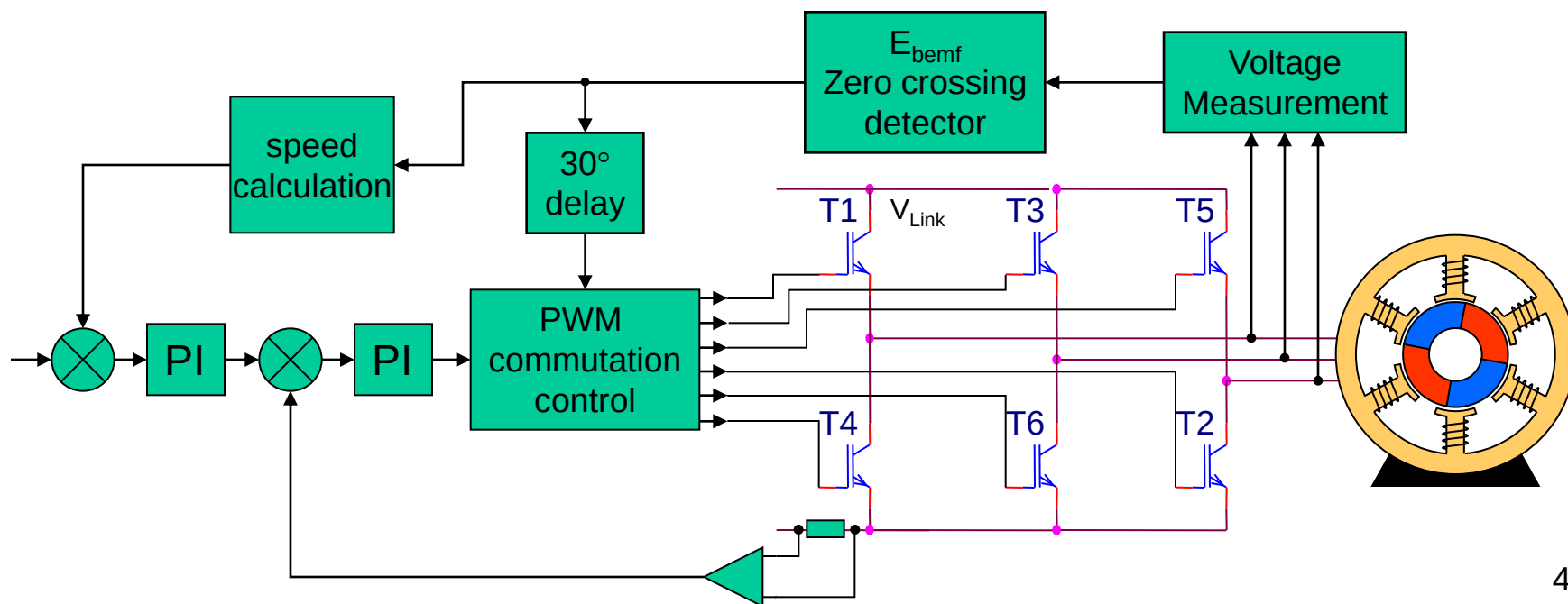
- In a BLDC system only two coils are “on” at any moment in time. The equivalent circuit of the motor with only two phases “on” is shown below
- It has been shown that* E_{bemf} at the unconnected phase is crossing its zero point when the terminal voltage at that phase is equal to $V_{dc_link}/2$
- In other words if we measure V_a , when $V_a = V_{dc_link}/2$ the $E_{bemf} = 0$



* “Microcomputer Control of Sensorless Brushless Motor”, K. Iizuka et.al, IEEE Transactions on Industry Applications, Vol IA-21, No4, May/June 1985, pp. 595 - 601

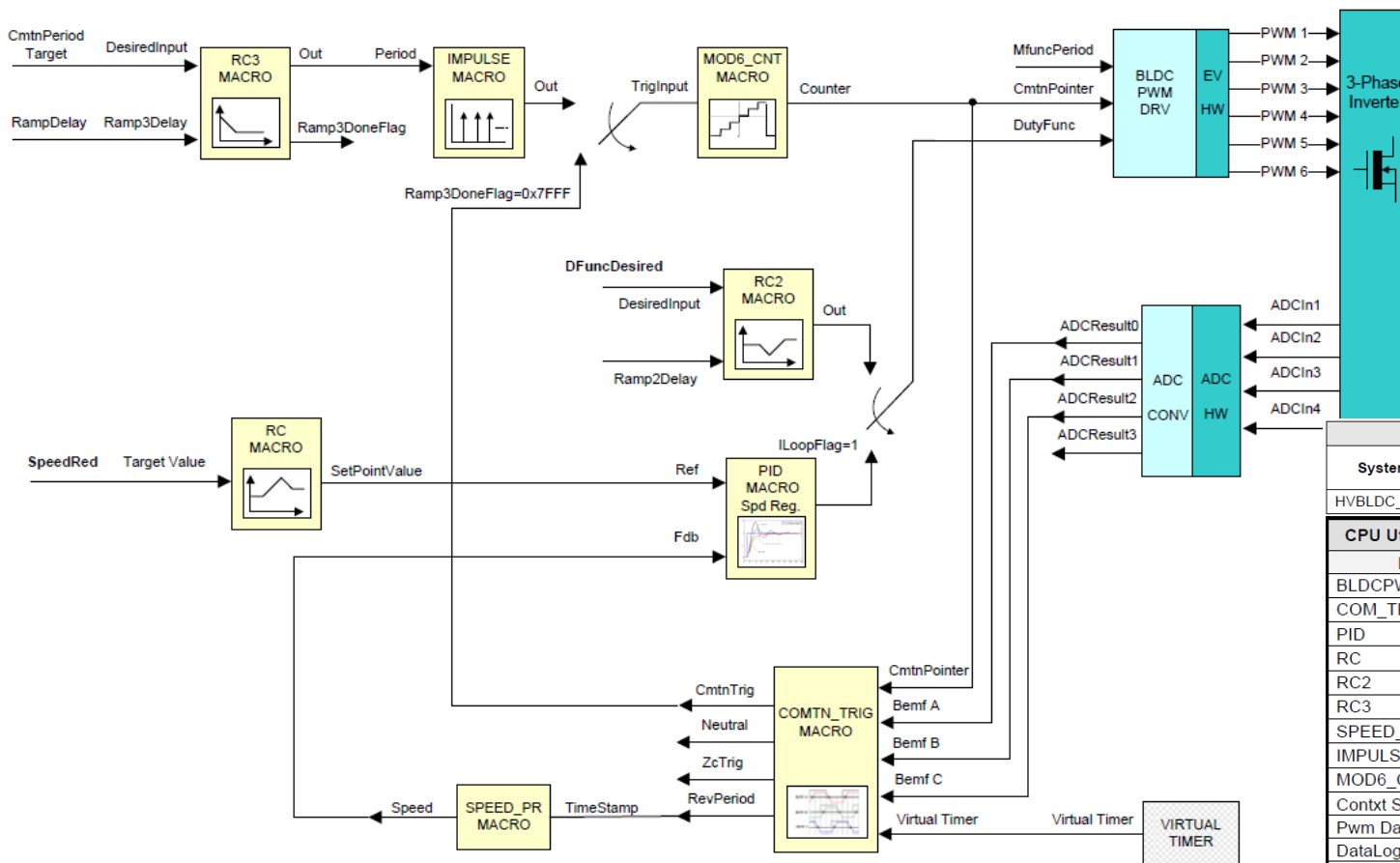
Sensorless Control of BLDC Motors (E_{Bemf})

- Full block diagram of the closed loop system
 - The Hall sensors have been eliminated giving a cheaper system at the expense of performance
 - To start the system, the stator frequency is slowly increased until rotor locks into synchronism
 - As the speed is increased E_{bemf} becomes significant and is used for sensorless closed loop control



TI Systems for Sensorless Control of BLDC Machines

- Modular DMCLib blocks are connected together in software
- www.ti.com/controlSUITE choose High Voltage Kit

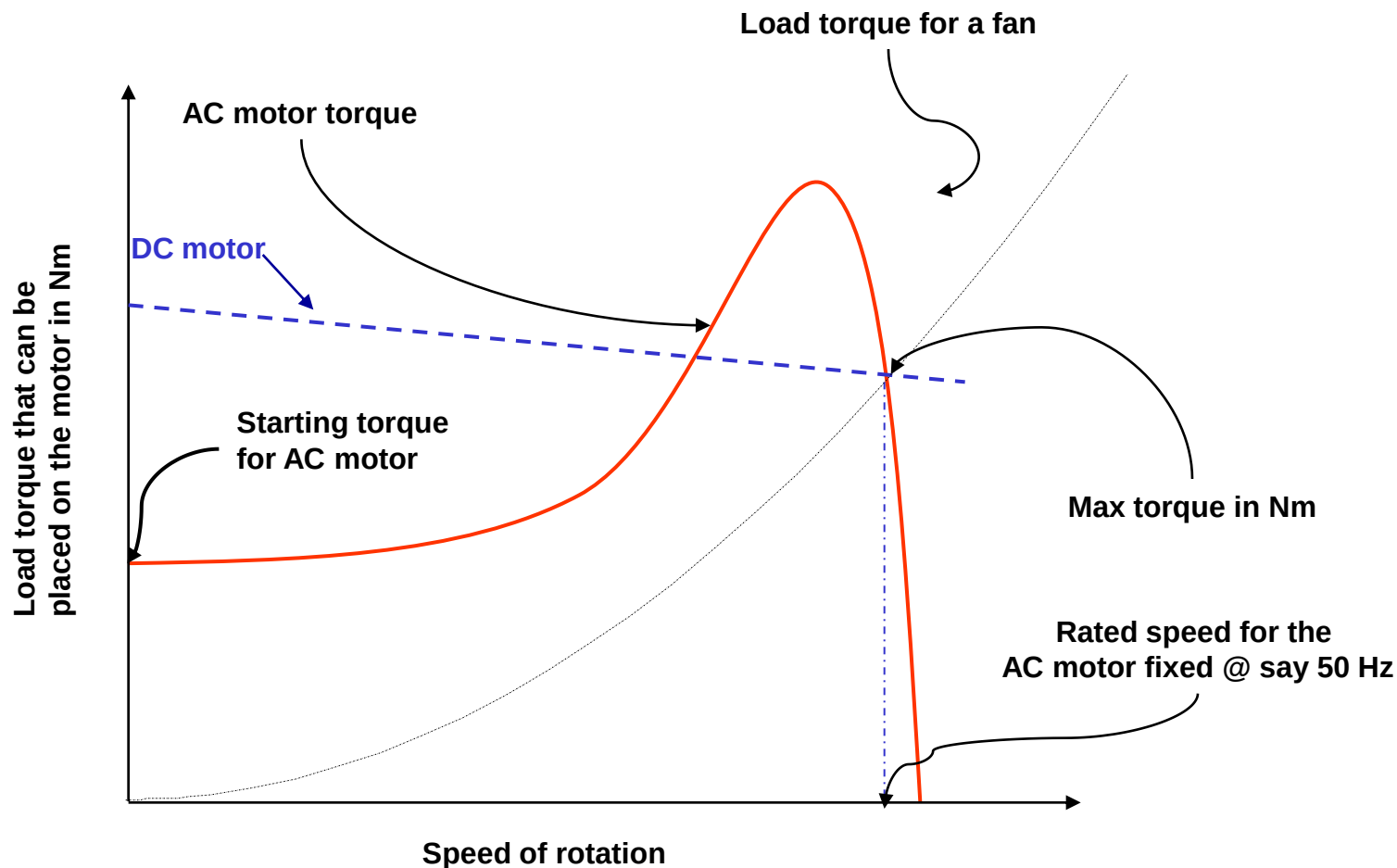


20 KHz PWM

C Framework		
System Name	Program Memory Usage 2803x	Data Memory Usage ¹ 2803x
HVBLDC_Sensorless	3699 words ²	1200 words
CPU Utilization of Trapezoidal BLDC Control (Sensorless)		
Name of Modules *	Number of Cycles	
BLDCPWM / PWM DAC	119	
COM_TRIG	87	
PID	47	
RC	29	
RC2	26	
RC3	26 ✓	
SPEED_PR	42	
IMPULSE	17 ✓	
MOD6_CNT	20	
Contxt Save, Virtual Timer etc.	153	
Pwm Dac (optional)		
DataLog (optional)		
Total Number of Cycles	548 **	
CPU Utilization @ 60 Mhz	18.3%	
CPU Utilization @ 40 Mhz	27.4%	

Scalar Control of Induction Motors

ACIM Torque Characteristics:



Scalar Control of Induction Motors

Constant V/f Control

$$V_s = I_s R_s + E_{bemf}$$

by Faraday's law:

$$E_{bemf} = -\frac{d\phi}{dt} = -\frac{d(BAN \cos(\omega t))}{dt} = k B \omega \sin(\omega t)$$

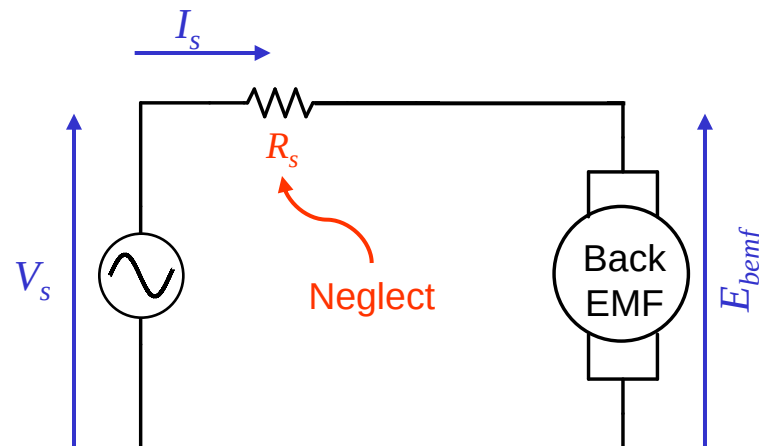
$\therefore$

$$E_{bemf_rms} = \frac{k_1 B \omega}{\sqrt{2}} = k_2 B f$$

at high speeds $I_s R_s \ll \frac{d\phi}{dt}$

$\therefore$

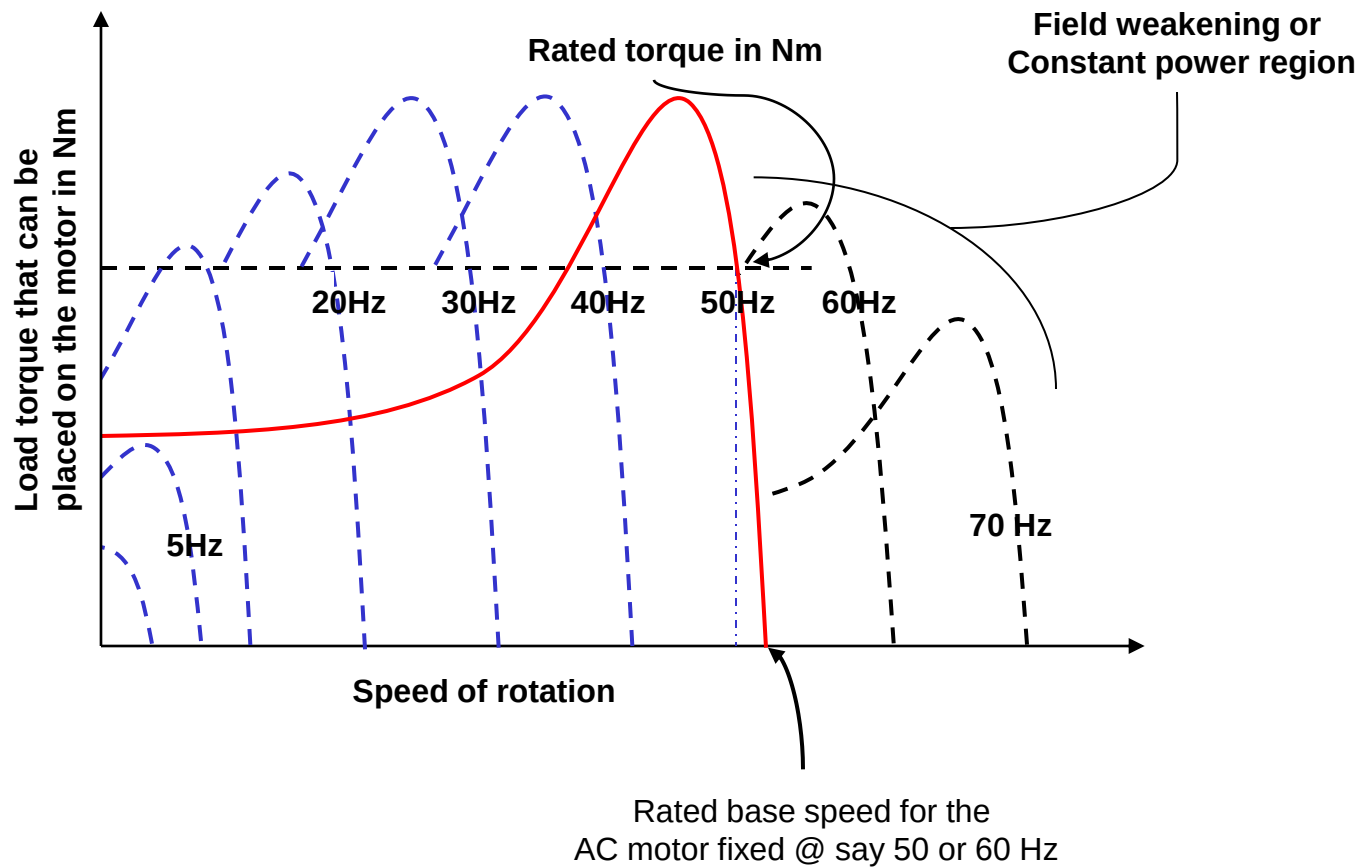
$$V_{s_rms} \cong k_2 B f \quad \text{or}$$



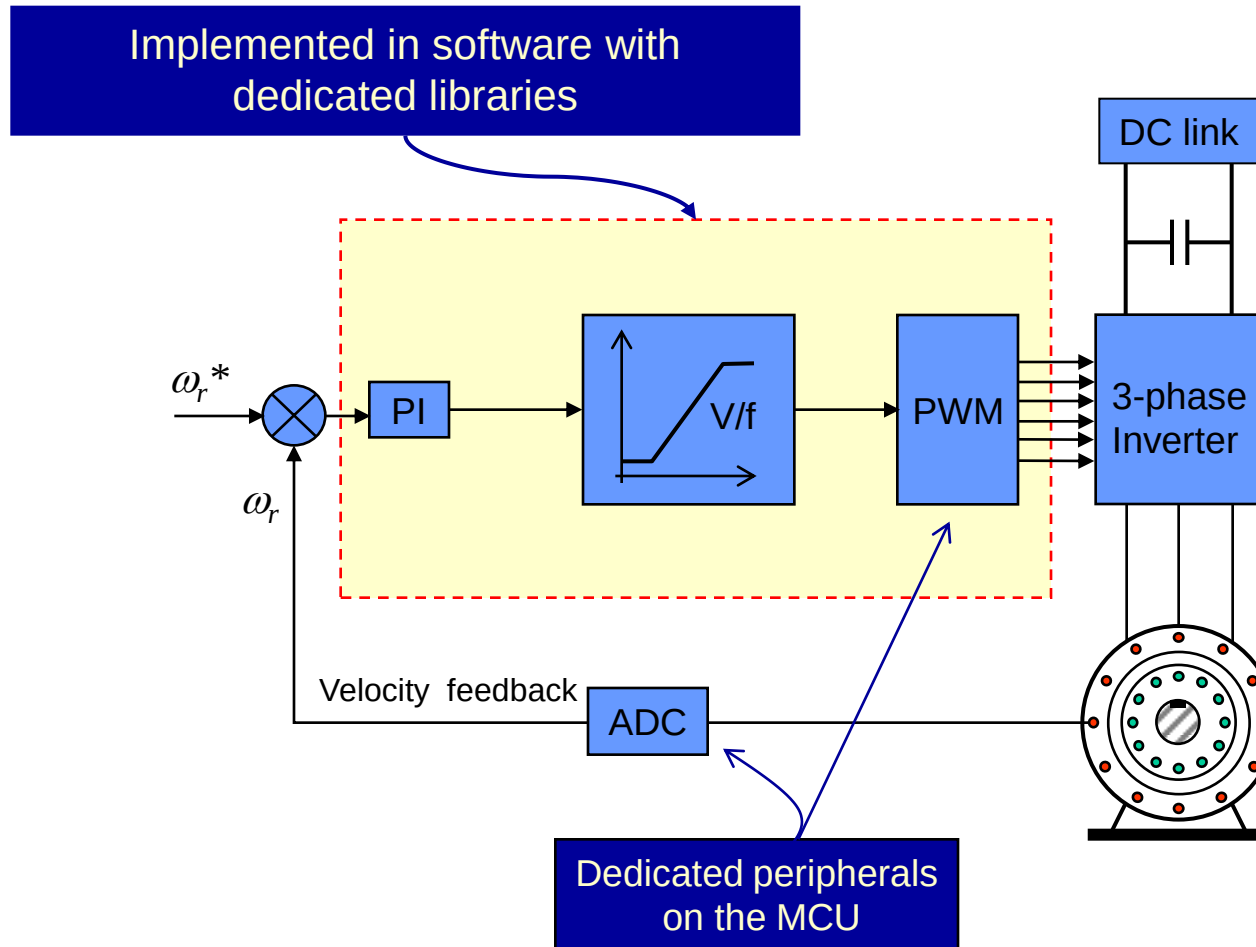
$$B \propto \frac{V}{f}$$

Important: This implies that if we change either f or V on their own the magnetic field will change thus changing the torque. If we keep this ratio constant then we will keep the magnetic field and hence the torque constant.

Induction Motor Torque Characteristics Under V/f Control



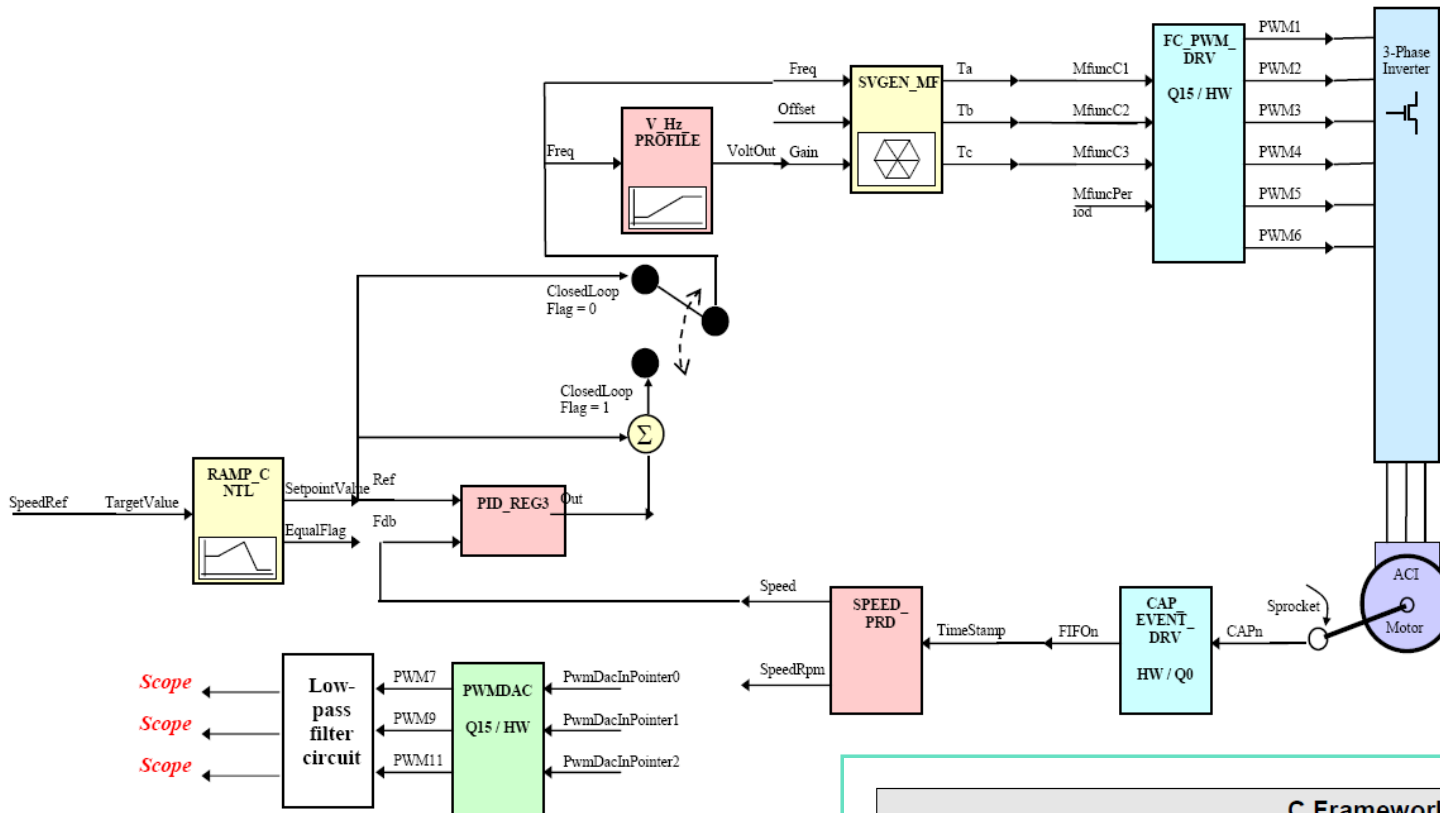
V/f Control Block Diagram



Note: velocity feedback is not essential for V/f control

V/f Control of Induction Machines

- www.ti.com/controlSUITE choose High Voltage Kit



10 KHz PWM

C Framework		
System Name	Program Memory Usage 2803x	Data Memory Usage ¹ 2803x
HVACI_Scalar	3284	1132

¹ Excluding the stack size

Summary of Constant V/f Control

- Torque is proportional to the flux density B
- $B \propto V/f$ from previous slide
- Increasing f on its own reduces B
- Increasing V on its own increases B
- Must keep V/f constant to keep T constant
- Advantages:
 - Cheap, simple, low development time, low processing power
 - No need for position information
- Disadvantages:
 - Poor performance especially at low speeds
 - Sluggish response

Induction Motor Applications (direct online or scalar control)

- **Fans**
- **Pumps**
- **Industrial Drives**
- **White goods (with belt drives/gearbox)**
- **High speed applications**
- **Compressors**
- **Air conditioning units**

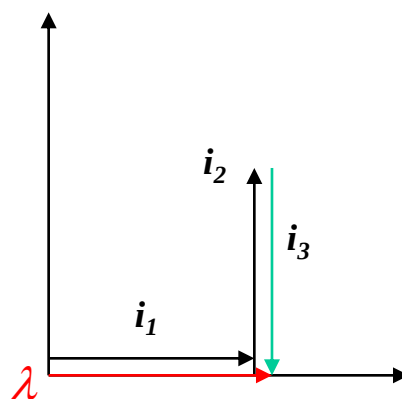
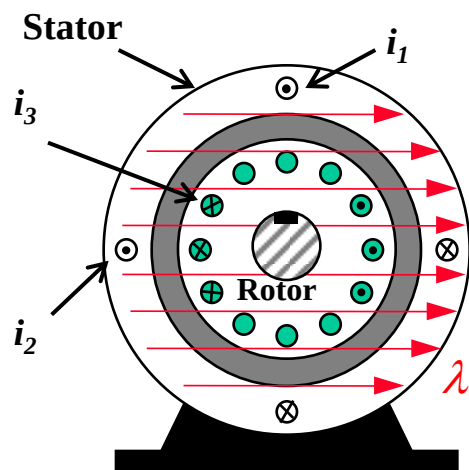
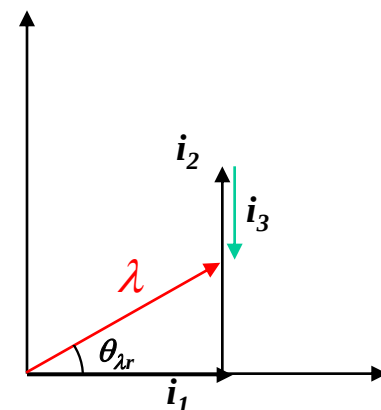
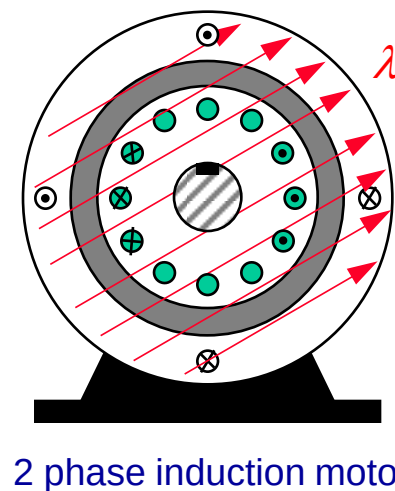
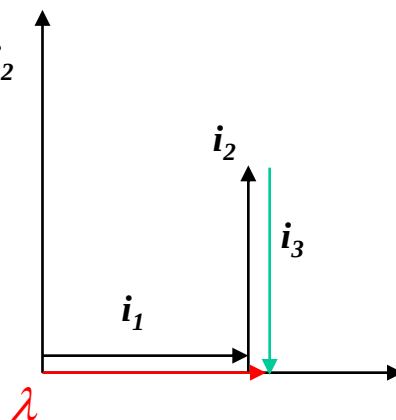
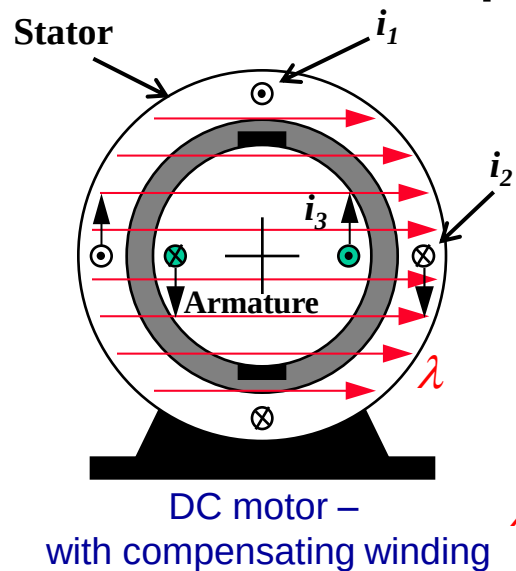
Vector Control of Induction Motors

- **The theory of operation:**
 - In essence the stator is a rotating magnet. The rotor is also a rotating magnet. If we could align these two magnets such that the stator magnetic field is always at 90° to the rotor magnetic field, we should get performance like a DC motor
- **Problems:**
 - We need to somehow identify the position of rotor flux
 - We have no access to the rotor currents and must estimate them
 - **Therefore we must have a mathematical model of the motor, i.e. need equivalent circuit**
 - The rotors currents are induced as a result of the moving stator flux
 - **Therefore, if we would like to manipulate the rotor currents, we have to manipulate the stator currents in such a way that the net effect would result in our desired state of the rotor current**
 - The angle of the stator flux does not remain constant
 - The stator and rotor are turning at different speeds i.e. slip

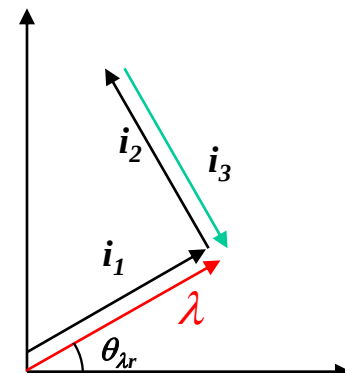
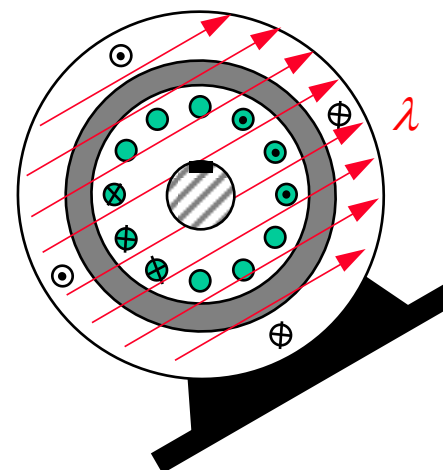
Vector Control Simplified

- In essence the vector controller runs a mathematical model of the 3 phase induction motor
 - Using this model it estimates the position of rotor magnet and then calculates where to position the stator's rotating magnet so that its magnetic bearing is at 90° to the rotor magnet
- The problem is that the mathematical model of a 3 phase motor is very complex
 - We could physically build a 2 phase motor using 2 stator coils at 90° to one another and energize them by a sine and a cosine wave which would give us exactly the same performance as the 3 phase motor
 - This would greatly simplify the mathematics
 - For simplicity in the next slide we have used a 2 phase induction motor
 - Later on we show how to convert a 3 phase motor into a 2 phase motor mathematically (Clarke Transform)

Vector Control Simplified



2 phase induction motor



2 phase induction motor

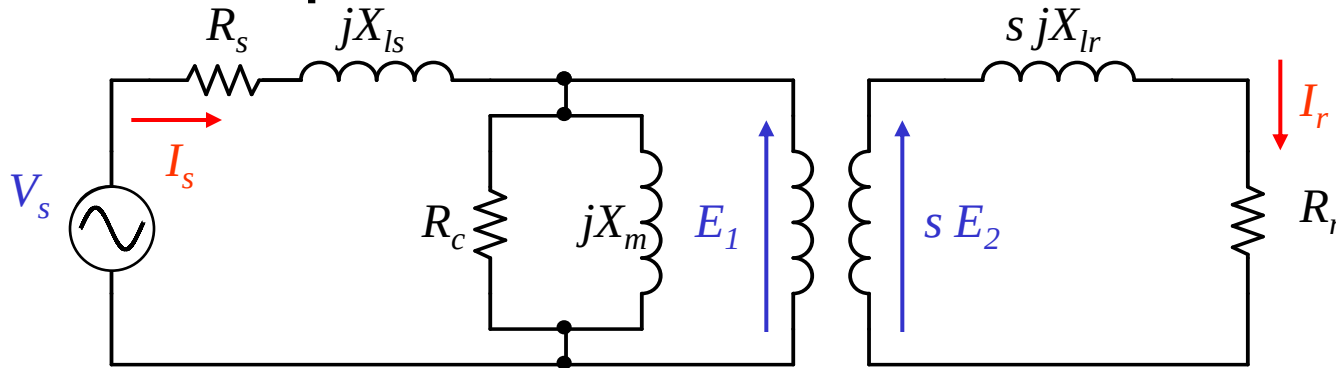
Vector Control Operations

Starting from equivalent circuit:

- **Task 1:** Transform the three-phase quantities (120° apart) to into two-phase quantities (90°) such that the resultant vector is the same
- **Task 2:** Identify the value of $\theta_{\lambda r}$ which would result in the correct orientation of the relevant vectors
- **Task 3:** Rotate the relevant vectors with $\theta_{\lambda r}$

Note: in the following slides we will describe the operation of vector control as applied to induction machines. The procedure in synchronous machines is almost identical; the only difference being that the identification of the position of rotor flux in a synchronous machine is much simpler.

Induction Motor Equivalent Circuit



R_s = stator resistance

X_{ls} = stator leakage reactance

R_c = core losses (ignore)

X_m = mutual reactance

X_{lr} = rotor leakage inductance

R_r = rotor resistance

V_s = stator input voltage

E_1 = back emf at stator

E_2 = per phase voltage induced in rotor at standstill

sE_2 = per phase voltage induced in rotor as it rotates

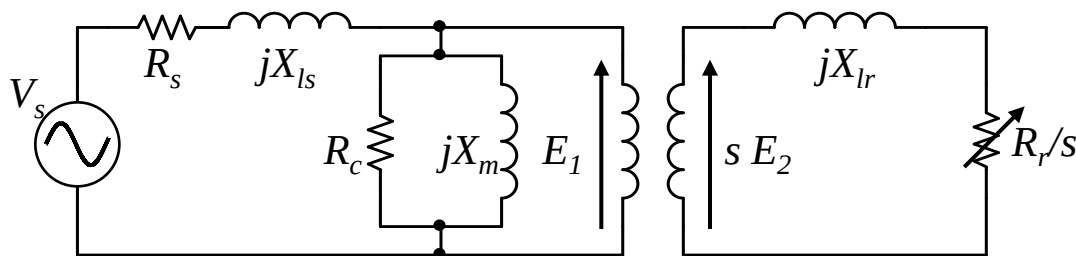
I_r = rotor current

I_s = stator current

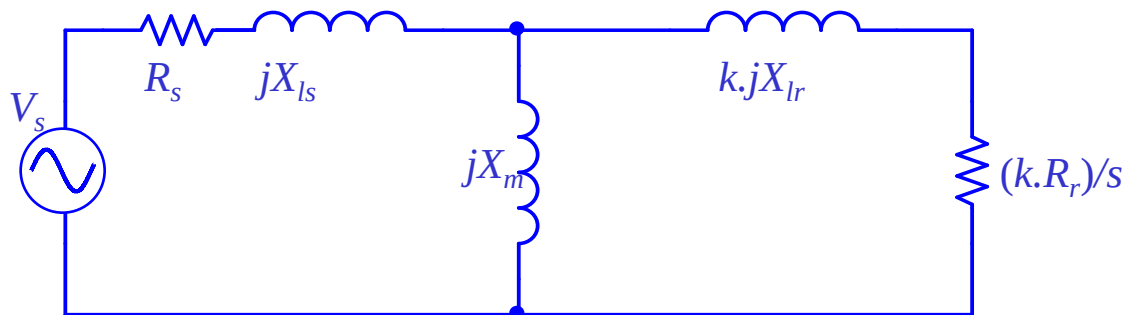
Note: as rotor starts to rotate less conductors will be cut by the stator's rotating magnetic field. Therefore, the magnitude of the induced emf will reduce $\Rightarrow s E_2$

Induction Motor Equivalent Circuit

From the previous slide:
$$I_r = \frac{s E_2}{\sqrt{R_r^2 + (s X_{lr})^2}} \therefore I_r = \frac{E_2}{\sqrt{\frac{R_r^2}{s} + X_{lr}^2}}$$

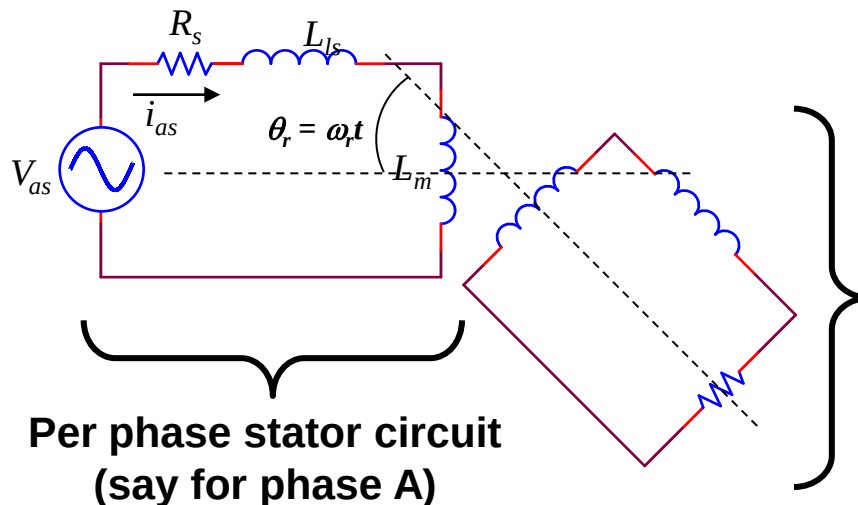


Finally we can refer all values to the stator side:



Note: R_c has been ignored

Deriving the Voltage Equations:



IMPORTANT: the rotor circuit is rotating at an angular velocity of $\omega_r \therefore$ the relative position of the rotor is changing with respect to the stator. Hence the mutual inductance between stator and rotor is a function of θ_r e.g. $L_m \cos \theta$

e.g. Phase A stator voltage V_{as} = the sum of:

Resistive voltage drop due to stator resistance R_s and phase a current i_{as}

Bemf due to i_{as} and phase A self inductance L_s

Bemf due to i_{bs} and the mutual inductance between stator phase A & B L_{ms}

Bemf due to i_{cs} and the mutual inductance between stator phase A & C L_{ms}

Bemf due to i_{ar} and the mutual inductance between stator and rotor

Bemf due to i_{br} and the mutual inductance between stator and rotor

Bemf due to i_{cr} and the mutual inductance between stator and rotor

$$V_{as} = i_{as} R_s + L_s \frac{di_{as}}{dt} + L_{ms} \frac{di_{bs}}{dt} + L_{ms} \frac{di_{cs}}{dt} + L_{ms} \cos \theta \frac{di_{ar}}{dt} + L_{ms} \cos \left(\theta + \frac{2\pi}{3} \right) \frac{di_{br}}{dt} + L_{ms} \cos \left(\theta - \frac{2\pi}{3} \right) \frac{di_{cr}}{dt}$$

Deriving the Voltage Equations:

$$V_{as} = i_{as} R_s + L_s \frac{di_{as}}{dt} + L_{ms} \frac{di_{bs}}{dt} + L_{ms} \frac{di_{cs}}{dt} + L_{ms} \cos \theta \frac{di_{ar}}{dt} + L_{ms} \cos \left(\theta + \frac{2\pi}{3} \right) \frac{di_{br}}{dt} + L_{ms} \cos \left(\theta - \frac{2\pi}{3} \right) \frac{di_{cr}}{dt}$$

Complete stator voltage equations:

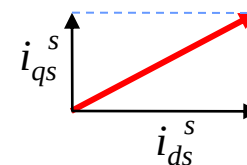
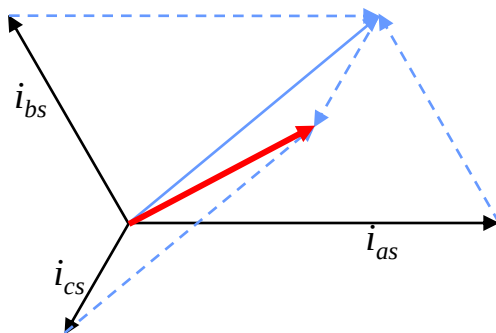
$$\begin{pmatrix} v_{as} \\ v_{bs} \\ v_{cs} \end{pmatrix} = \begin{pmatrix} i_{as} \\ i_{bs} \\ i_{cs} \end{pmatrix} \cdot R_s + \begin{pmatrix} L_s & L_{ms} & L_{ms} \\ L_{ms} & L_s & L_{ms} \\ L_{ms} & L_{ms} & L_s \end{pmatrix} \cdot \begin{pmatrix} \dot{i}_{as} \\ \dot{i}_{bs} \\ \dot{i}_{cs} \end{pmatrix} + \begin{pmatrix} L_m \cos \theta & L_m \cos \left(\theta + \frac{2\pi}{3} \right) & L_m \cos \left(\theta - \frac{2\pi}{3} \right) \\ L_m \cos \left(\theta - \frac{2\pi}{3} \right) & L_m \cos \theta & L_m \cos \left(\theta + \frac{2\pi}{3} \right) \\ L_m \cos \left(\theta + \frac{2\pi}{3} \right) & L_m \cos \left(\theta - \frac{2\pi}{3} \right) & L_m \cos \theta \end{pmatrix} \cdot \begin{pmatrix} \dot{i}_{ar} \\ \dot{i}_{br} \\ \dot{i}_{cr} \end{pmatrix}$$

Complete rotor voltage equations:

$$\begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} = \begin{pmatrix} i_{ar} \\ i_{br} \\ i_{cr} \end{pmatrix} \cdot R_r + \begin{pmatrix} L_r & L_{mr} & L_{mr} \\ L_{mr} & L_r & L_{mr} \\ L_{mr} & L_{mr} & L_r \end{pmatrix} \cdot \begin{pmatrix} \dot{i}_{ar} \\ \dot{i}_{br} \\ \dot{i}_{cr} \end{pmatrix} + \begin{pmatrix} L_m \cos \left(\theta + \frac{2\pi}{3} \right) & L_m \cos \left(\theta - \frac{2\pi}{3} \right) & L_m \cos \theta \\ L_m \cos \left(\theta - \frac{2\pi}{3} \right) & L_m \cos \theta & L_m \cos \left(\theta + \frac{2\pi}{3} \right) \\ L_m \cos \left(\theta + \frac{2\pi}{3} \right) & L_m \cos \left(\theta - \frac{2\pi}{3} \right) & L_m \cos \theta \end{pmatrix} \cdot \begin{pmatrix} \dot{i}_{as} \\ \dot{i}_{bs} \\ \dot{i}_{cs} \end{pmatrix}$$

Converting Our 3 Phase to an equivalent 2 phase motor

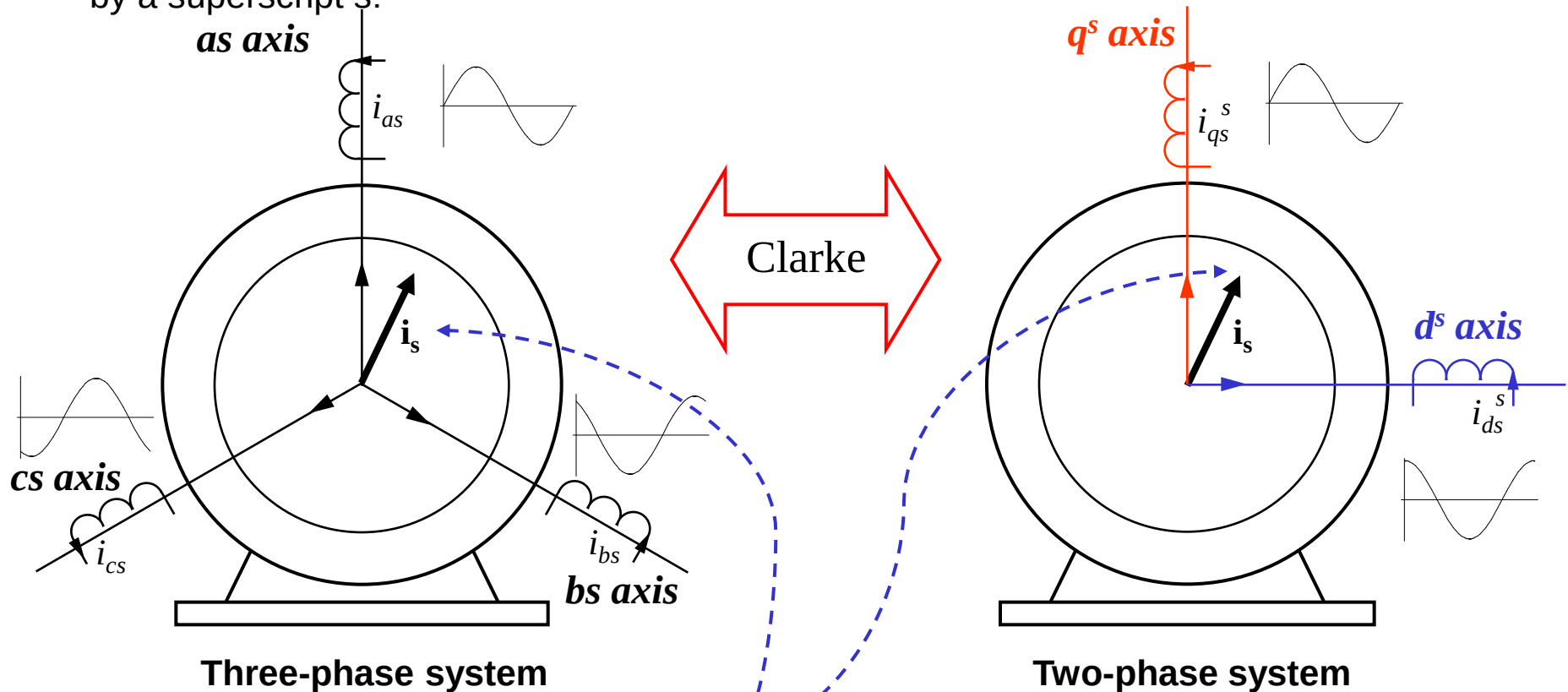
- Imagine three sinusoidal currents (i_{as} , i_{bs} & i_{cs}) in a balanced 3-phase system
- The interaction of these three currents will result in one resultant current vector shown in red:
- However, exactly the same resultant vector (shown in red) can be generated using only 2 currents
- We will call these new currents i_{qs}^s and i_{ds}^s



- These new currents are acting along two new axes at 90° which we will call the direct d (*field*) and quadrature q (*armature*) axis.
- The process above is called the Clarke Transform. We will talk about this in the next slide

Stationary Three-Phase to Stationary Two-Phase Transform (Clarke Transform)

This transforms our **balanced** three-phase quantities to two-phase stationary quantities. After the transformation we will be in the **“Stationary Reference Frame”** this is denoted by a superscript s:



Note: the resultant current vector is the same

Important: i_{qs} and i_{ds} axes are sometimes named β and α axes

Clarke Transform

Aligning the qs axis with the as axis so that $\theta = 0$ and and resolving we will have:

Clarke Transform:

$$f_{qs}^s = K \cdot \left(f_{as} - \frac{1}{2} f_{bs} - \frac{1}{2} f_{cs} \right) = K \cdot \frac{3}{2} f_{as}$$

$$f_{ds}^s = K \cdot \left(-\frac{\sqrt{3}}{2} f_{bs} + \frac{\sqrt{3}}{2} f_{cs} \right)$$

Inverse Clarke Transform:

$$f_{as} = f_{qs}^s$$

$$f_{bs} = -\frac{1}{2} f_{qs}^s - \frac{\sqrt{3}}{2} f_{ds}^s$$

$$f_{cs} = -\frac{1}{2} f_{qs}^s + \frac{\sqrt{3}}{2} f_{ds}^s$$

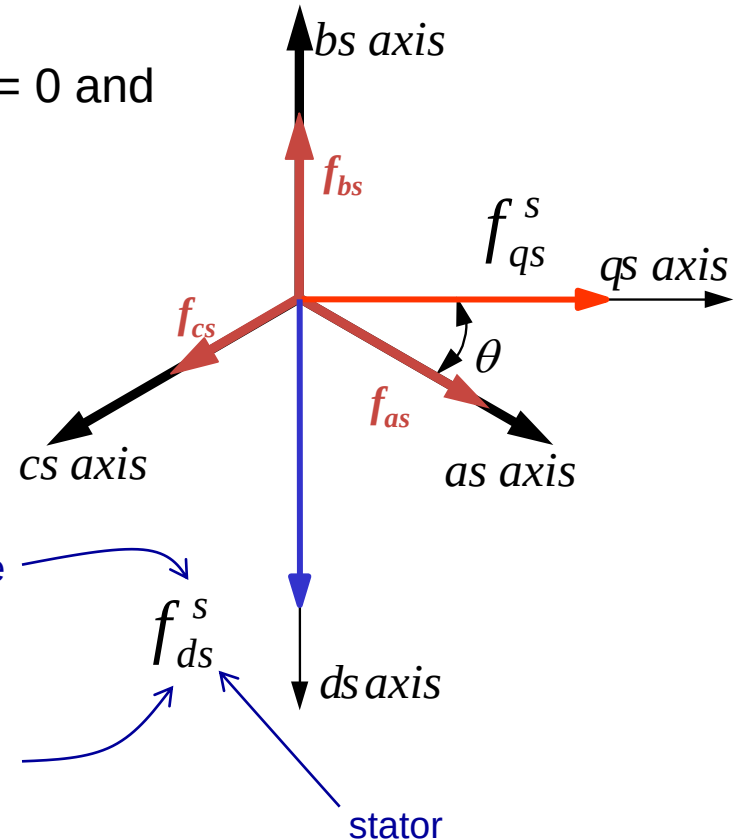
Where:

$$K = \frac{2}{3}$$

$$f_{as}, f_{bs}, f_{cs} = \text{Three phase stator quantities.}$$

$$f_{ds}^s, f_{qs}^s = \text{Stator } d^s\text{-}q^s \text{ quantities.}$$

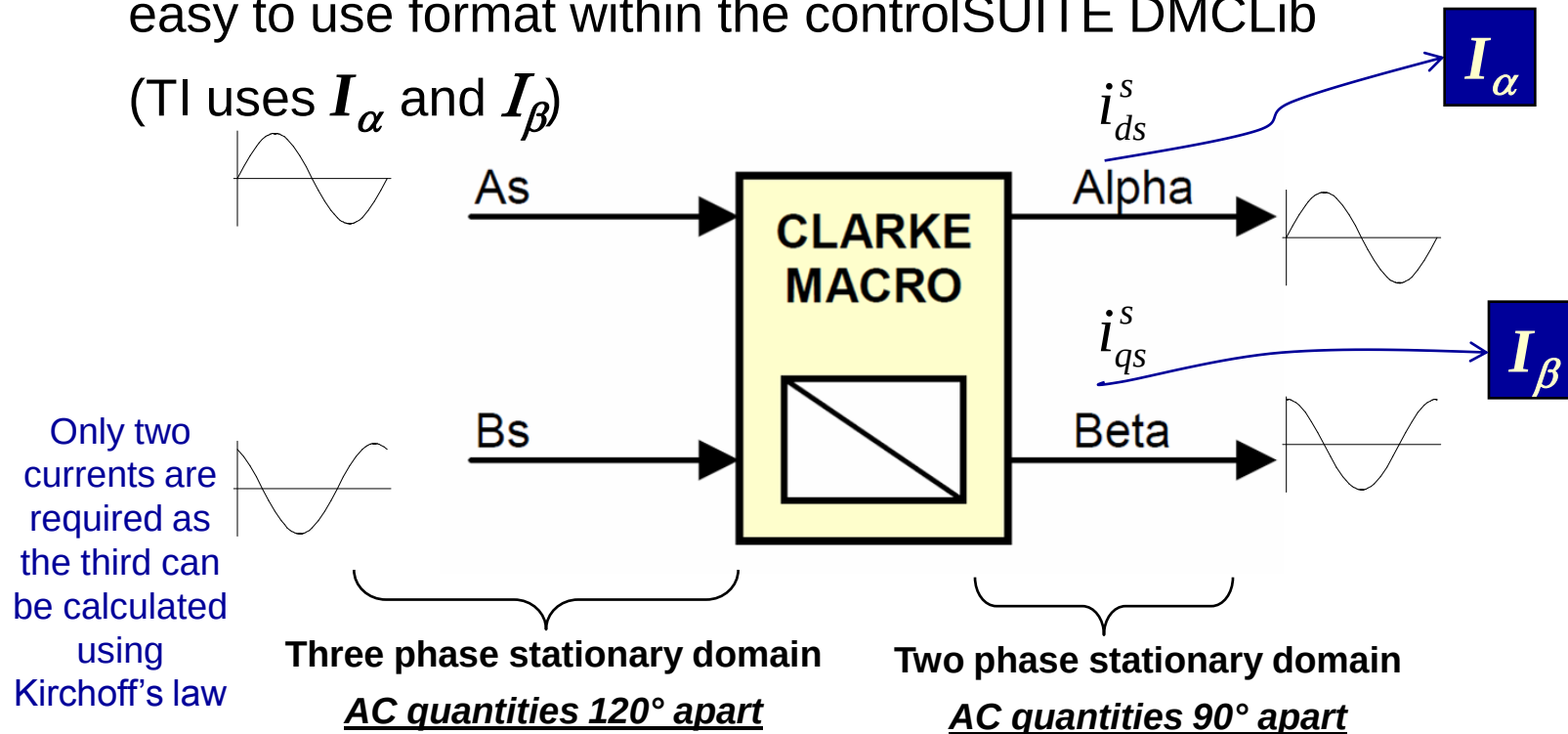
$$\theta = \text{An arbitrary angle.}$$



Clarke Transform Made Easy

- TI provides this transform (and its inverse in the SVGEN macro) in an easy to use format within the controlSUITE DMCLib

(TI uses I_α and I_β)



If we now apply the Clarke transform to our 3 phase mathematical model, the entire machine model can be reduced to 4 simple differential equations – please see next slide

Induction Machine Model in the Stationary Reference Frame

$$v_{qs}^s = \frac{d\lambda_{qs}^s}{dt} + i_{qs}^s R_s \quad (\text{equ. 1})$$

$$v_{ds}^s = \frac{d\lambda_{ds}^s}{dt} + i_{ds}^s R_s \quad (\text{equ. 2})$$

$$0 = \frac{d\lambda_{qr}^s}{dt} + i_{qr}^s R_r - \lambda_{dr}^s \omega_r \quad (\text{equ. 3})$$

$$0 = \frac{d\lambda_{dr}^s}{dt} + i_{dr}^s R_r + \lambda_{qr}^s \omega_r \quad (\text{equ. 4})$$

Task 1 completed
i.e. we can convert
our 3 phase
motor to a 2 phase motor

Where:

$$\lambda_{qs}^s = L_s i_{qs}^s + L_m i_{qr}^s = \text{qs axis stator flux linkage} \quad (\text{equ. 5})$$

$$\lambda_{ds}^s = L_s i_{ds}^s + L_m i_{dr}^s = \text{ds axis stator flux linkage} \quad (\text{equ. 6})$$

$$\lambda_{qr}^s = L_r i_{qr}^s + L_m i_{qs}^s = \text{qs axis rotor flux linkage} \quad (\text{equ. 7})$$

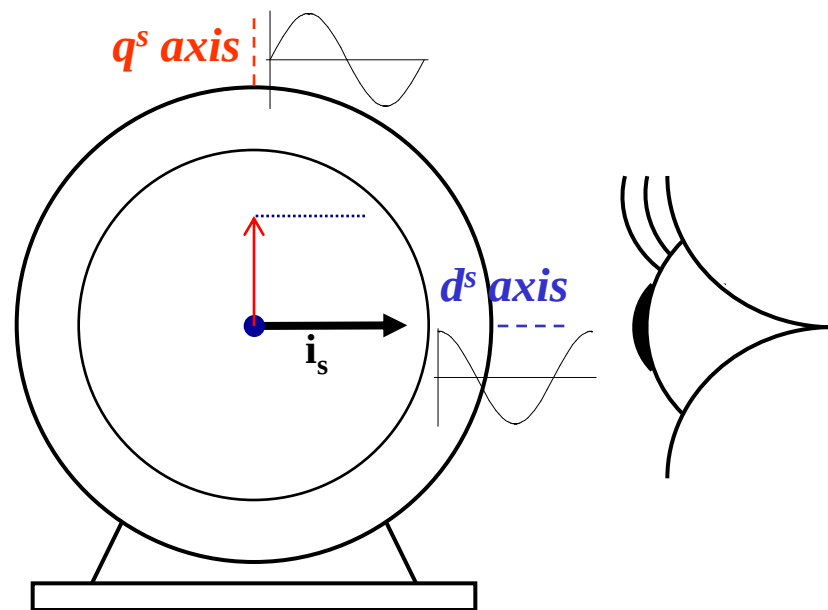
$$\lambda_{dr}^s = L_r i_{dr}^s + L_m i_{ds}^s = \text{ds axis rotor flux linkage} \quad (\text{equ. 8})$$

Park Transform: Stationary (dq^s) to Rotating (dq^e) Reference Frame Transform:

- Using the Clarke Transform we have obtained a 2 phase mathematical model for our motor
 - This was the first task of the vector controller
 - This resulted in a 2 phase motor with 90° phase shift between the d^s and q^s axes (also known as α and β axes)
- Assuming that we know the rotation angle $\theta_{\lambda r}$ for now (Task 2) we would like to be able to vector rotate our 2 phase quantities by this angle
- This is done using the Park Transform

Park Transform

- Consider the **resultant** current vector ($\mathbf{i}_s$) of our 2 phase motor as a result of our two phase currents (sin & cos) that we are injecting into our 2 phase stator
 - This vector will rotate at a stator angular frequency of ω_e
 - The stationary observer will see the projection of this vector along the vertical (i.e. $\mathbf{i}_s \times \sin \omega_e t$). Therefore at $t = 0$ the observer will see nothing
 - Let's now assume that after a certain time our vector has rotated by 45° ; the observer will see $\mathbf{i}_s \times \sin \omega_e t$
 - If now the observer also moves by exactly 45° there is no relative movement between the vector on the rotating side (rotor) and the observer on the stationary side (stator) and the observer will see nothing again
 - We can do this operation mathematically and is known as the Park Transform



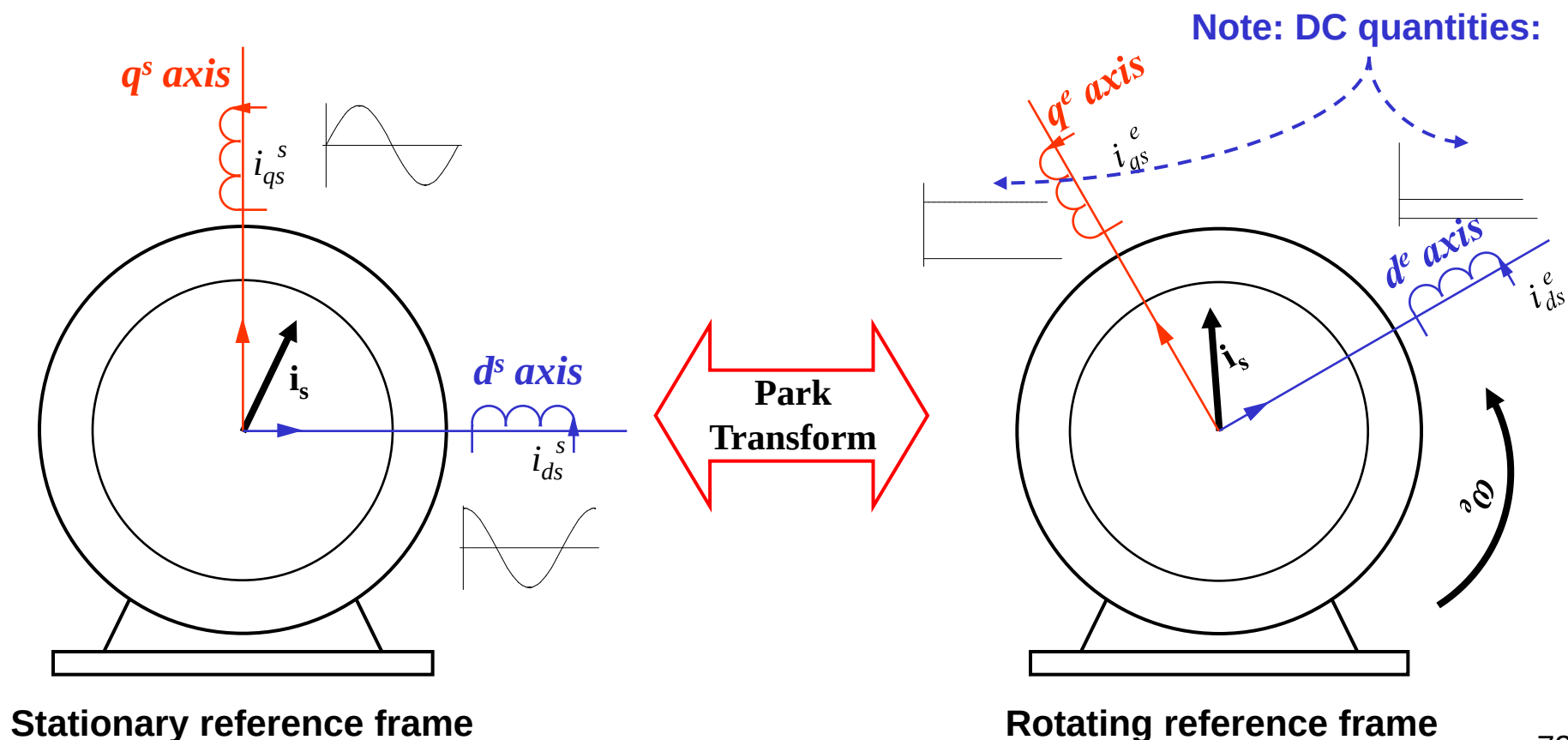
Stationary reference frame

Park Transform

- It follows from the previous slide that if the observer runs around the motor at exactly the same speed as the rotor, then there will be no relative movement between the observer and the rotor
 - Therefore all quantities on the rotor will appear as constants (i.e. DC to the observer)
 - Under this condition we say that we are operating in the **rotating reference frame**
 - Of course in practice we do not run around the motor!! We use mathematical vector rotation algorithms to achieve this objective
- This operation is known as the Park Transform and a vector controller calculates the Park algorithm at every sampling interval by rotating the relevant vectors with $\theta_{\lambda r}$
- In the following slides we show the details of the Park Transform and the available TI libraries

Park Transform: Stationary (dq^s) to Rotating (dq^e) Reference Frame Transform:

This transforms our two-phase stationary quantities to a two-phase reference frame rotating synchronously with the stator quantities, i.e. it allows our sinusoidal two-phase quantities to be **rotated** by a known angle. This is denoted by a superscript e (dq^e).



Stationary to Rotating Reference Frame

Again the transformation equations can be simply derived by resolving f_s , along the desired axis:

Rotating
ref frame

dq^s to dq^e transform:

$$f_{qs}^e = f_{qs}^s \cos(\omega_e t) - f_{ds}^s \sin(\omega_e t)$$

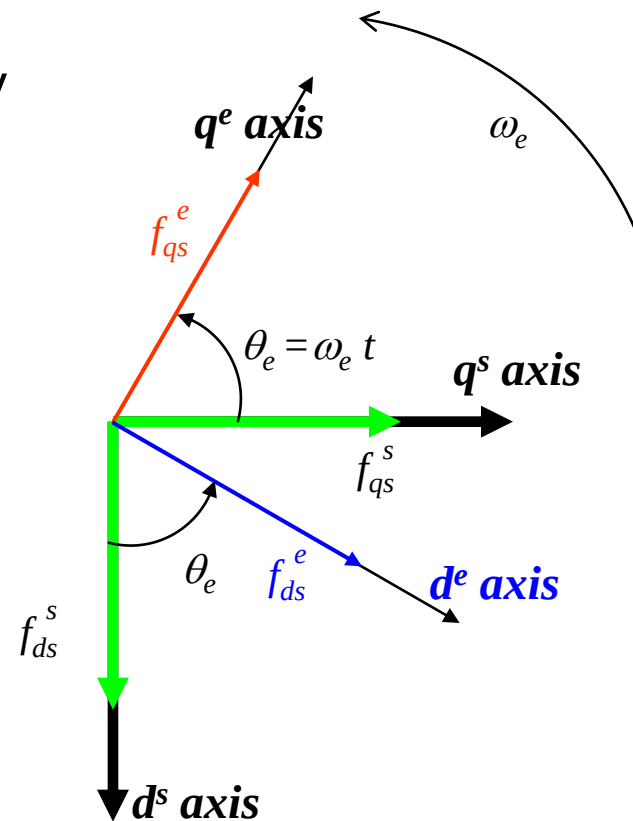
$$f_{ds}^e = f_{qs}^s \sin(\omega_e t) + f_{ds}^s \cos(\omega_e t)$$

q
axis
stator

Inverse dq^s to dq^e transform:

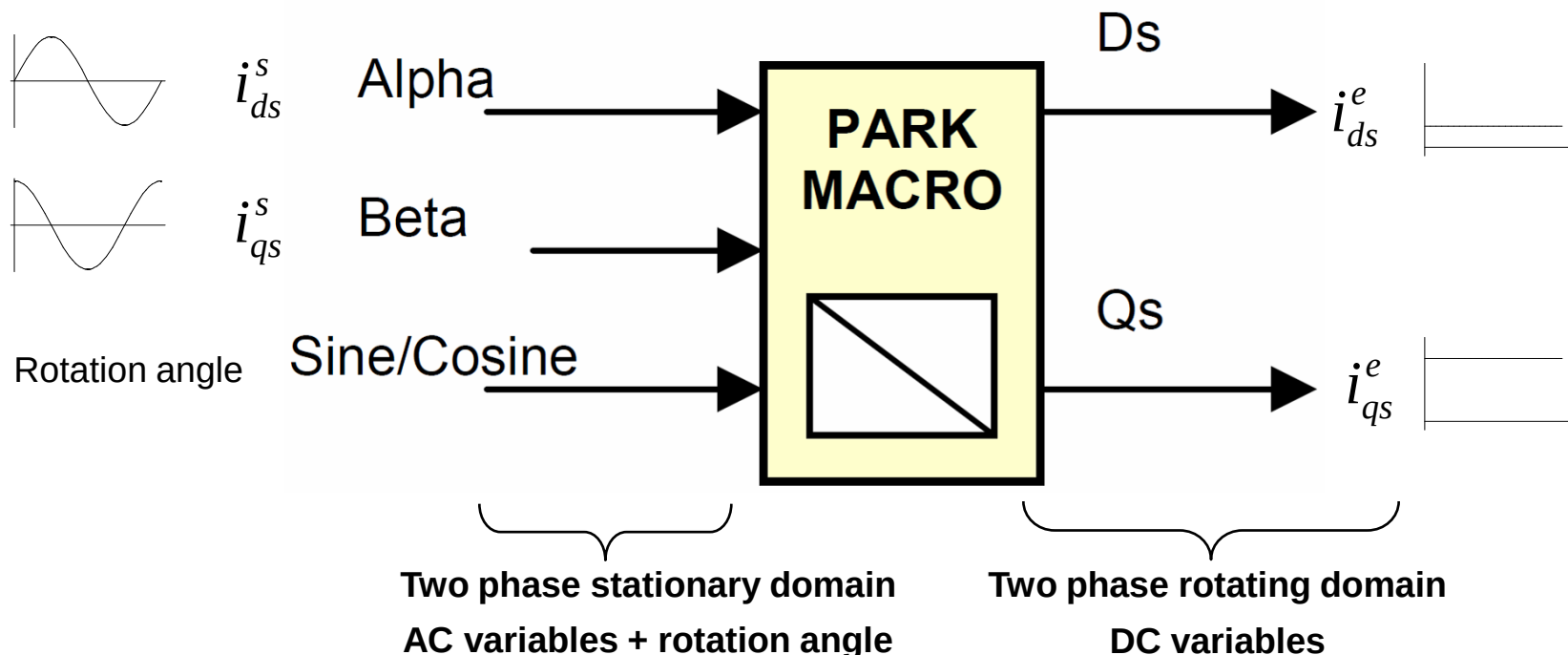
$$f_{qs}^s = f_{qs}^e \cos(\omega_e t) + f_{ds}^e \sin(\omega_e t)$$

$$f_{ds}^s = -f_{qs}^e \sin(\omega_e t) + f_{ds}^e \cos(\omega_e t)$$



Park Transform Made Easy

- TI provides this transform (and its inverse) in an easy to use format within the controlSUITE DMCLib



If we now apply the Park transform to our 2 phase mathematical model in the stationary reference frame, the entire machine model in the rotating reference frame will only have DC quantities – please see next slide

Induction Machine Model in Rotating Reference Frame

$$v_{qs}^e = \frac{d\lambda_{qs}^e}{dt} + i_{qs}^e R_s + \lambda_{ds}^e \omega_e \quad (\text{equ. 13})$$

$$v_{ds}^e = \frac{d\lambda_{ds}^e}{dt} + i_{ds}^e R_s - \lambda_{qs}^e \omega_e \quad (\text{equ. 14})$$

$$0 = \frac{d\lambda_{qr}^e}{dt} + i_{qr}^e R_r + \lambda_{dr}^e (\omega_e - \omega_r) \quad (\text{equ. 15})$$

$$0 = \frac{d\lambda_{dr}^e}{dt} + i_{dr}^e R_r - \lambda_{qr}^e (\omega_e - \omega_r) \quad (\text{equ. 16})$$

Task 3 completed
We can now
rotate vectors with a known
angle

Where:

$$\lambda_{qs}^e = L_s i_{qs}^e + L_m i_{qr}^e = \text{qe axis stator flux linkage} \quad (\text{equ. 17})$$

$$\lambda_{ds}^e = L_s i_{ds}^e + L_m i_{dr}^e = \text{de axis stator flux linkage} \quad (\text{equ. 18})$$

$$\lambda_{qr}^e = L_r i_{qr}^e + L_m i_{qs}^e = \text{qe axis rotor flux linkage} \quad (\text{equ. 19})$$

$$\lambda_{dr}^e = L_r i_{dr}^e + L_m i_{ds}^e = \text{de axis rotor flux linkage} \quad (\text{equ. 20})$$

Identifying the Position of Rotor Flux

- In order to achieve our final task we need to manipulate the machine model equations to get an equation for $\theta_{\lambda r}$
- Our equations must be in terms of **Stator** currents and voltages only as we do not have access to rotor currents
- There are two main ways of doing this which leads to two different types of vector control systems:
 - Indirect vector control
 - Direct vector control
- We will talk about these next

Indirect Vector Control

- The objective is to derive an equation for the rotor flux $\theta_{\lambda r}$ in terms of stator quantities only
- We know that: $\theta_{\lambda r} = \theta_{sl} + \theta_r \Rightarrow \theta_{\lambda r} = \int_0^t \omega_{sl} dt + \theta_r$ (equ. 25.a)
- By manipulating the machine model equations we can show that:

$$\omega_{sl} = \frac{i_{qs}^e R_r}{i_{ds}^e L_r} \quad (\text{equ 34})$$

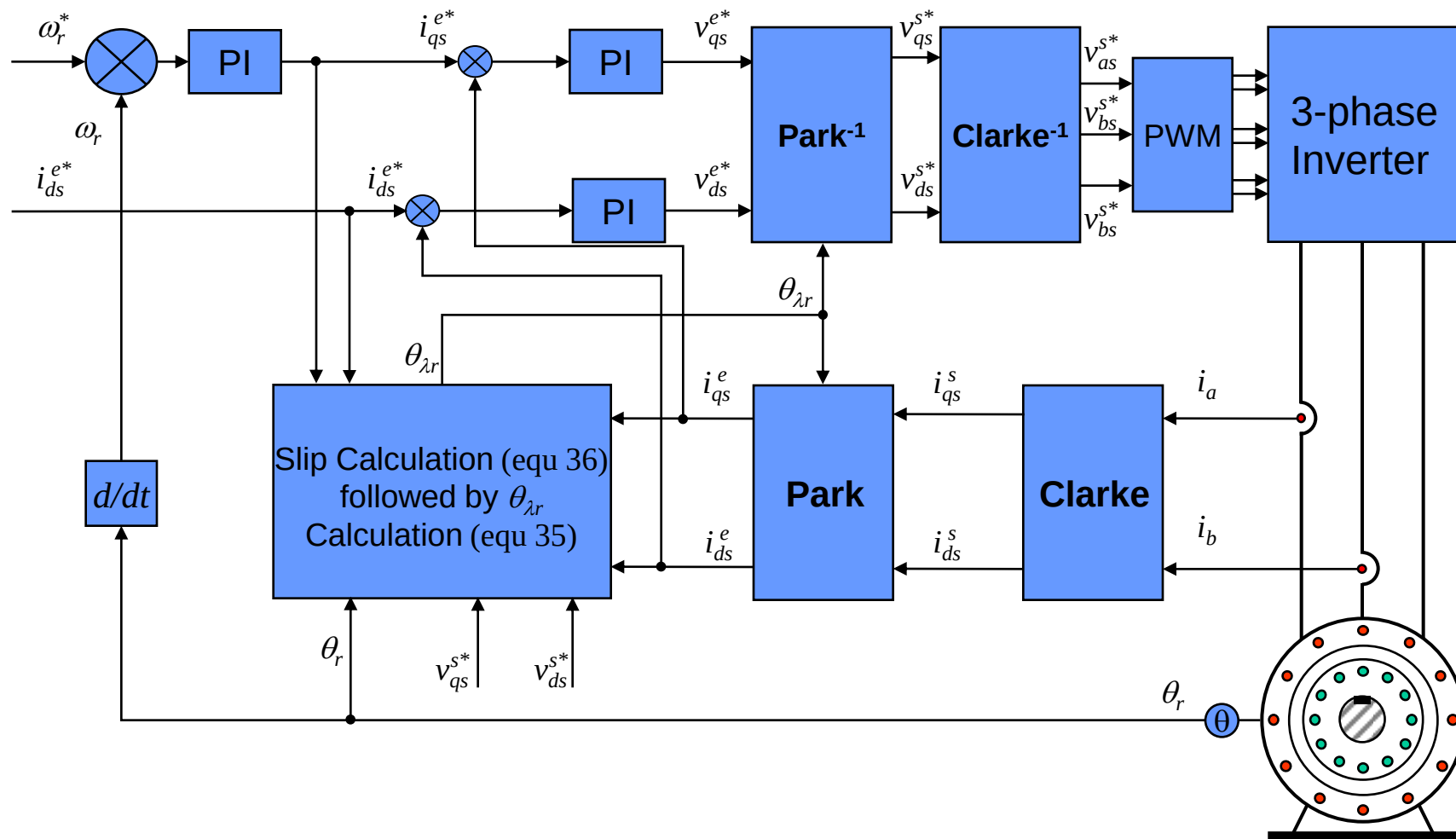
- By using the demand value of stator currents we can enforce the slip such that we get the correct orientation :

$$\theta_{\lambda r}^* = \int_0^t (\omega_{sl}^* + \omega_r) dt \quad (\text{equ 35})$$

Where: $\omega_{sl}^* = \frac{i_{qs}^{e*} R_r}{i_{ds}^{e*} L_r}$ (equ 36)

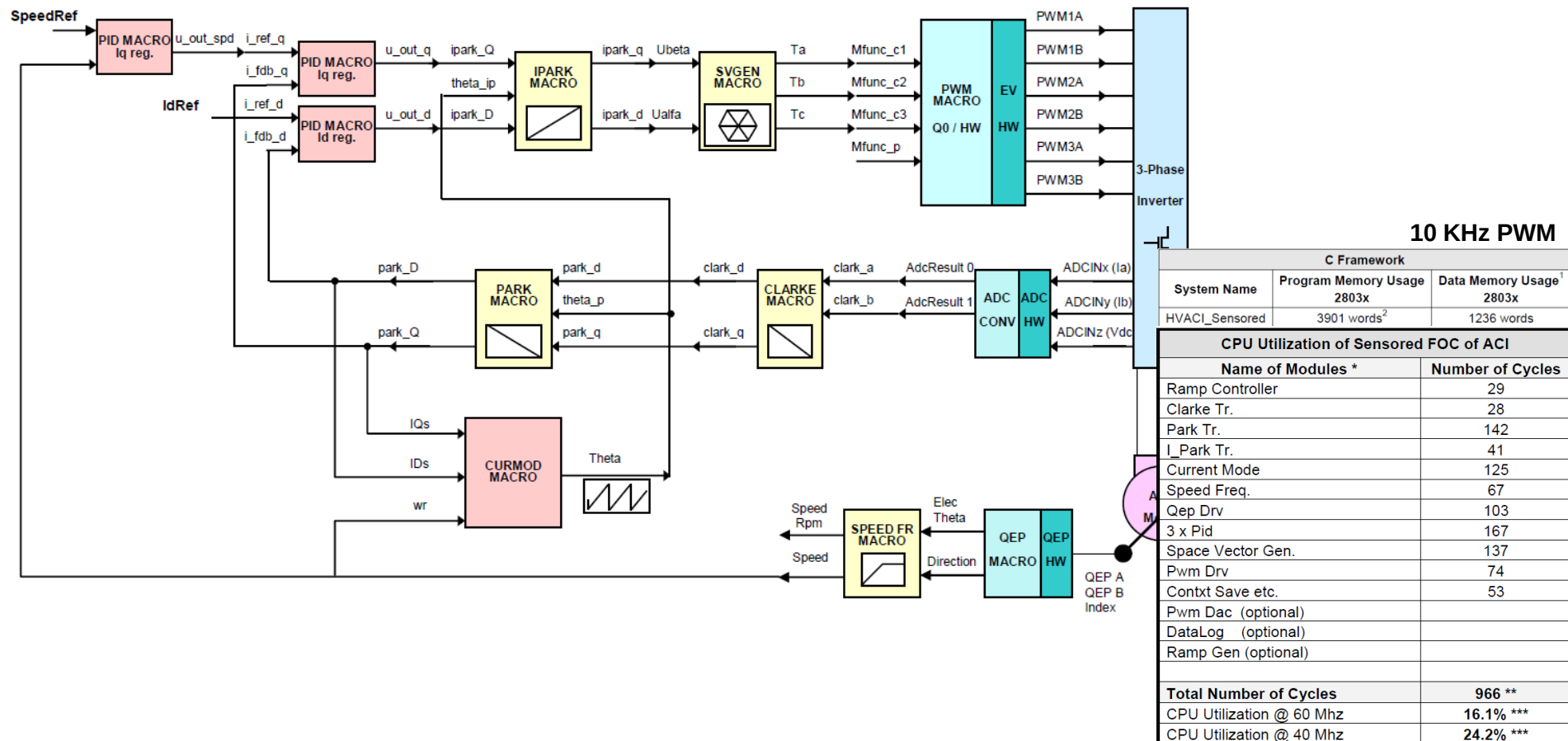
Indirect Vector Control Step by Step

Task 2 completed



TI Systems for Sensored Indirect FOC of ACI

- www.ti.com/controlSUITE and choose High Voltage Kit



Direct Vector Control

- Again the objective is to derive an equation for the rotor flux $\theta_{\lambda r}$ in terms of stator quantities only
- By manipulating the machine model equations we can show that:

$$\lambda_{qr}^s = L_r \left(\frac{\lambda_{qs}^s - L_s i_{qs}^s}{L_m} \right) + L_m i_{qs}^s \quad (\text{equ. 39})$$

$$\lambda_{dr}^s = L_r \left(\frac{\lambda_{ds}^s - L_s i_{ds}^s}{L_m} \right) + L_m i_{ds}^s \quad (\text{equ. 40})$$

– Where:

$$\lambda_{qs}^s = \int v_{qs}^s - i_{qs}^s R_s dt \quad (\text{equ. 37})$$

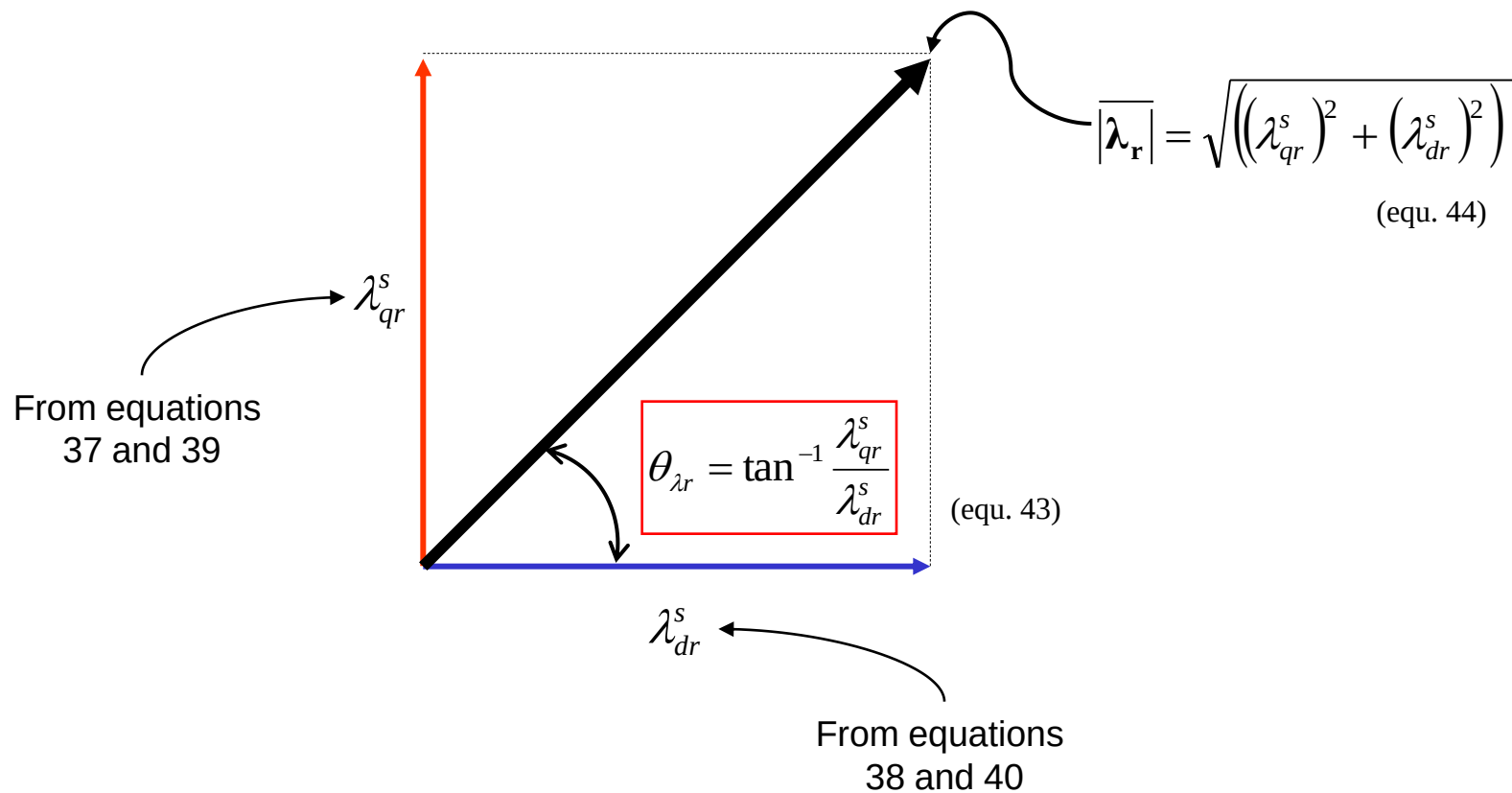
$$\lambda_{ds}^s = \int v_{ds}^s + i_{ds}^s R_s dt \quad (\text{equ. 38})$$

- Finally: $\theta_{\lambda r} = \tan^{-1} \frac{\lambda_{qr}^s}{\lambda_{dr}^s} \quad (\text{equ. 43})$

Note that the above are all in stator quantities. Using the above we can calculate the position of the rotor flux as shown in the next slide

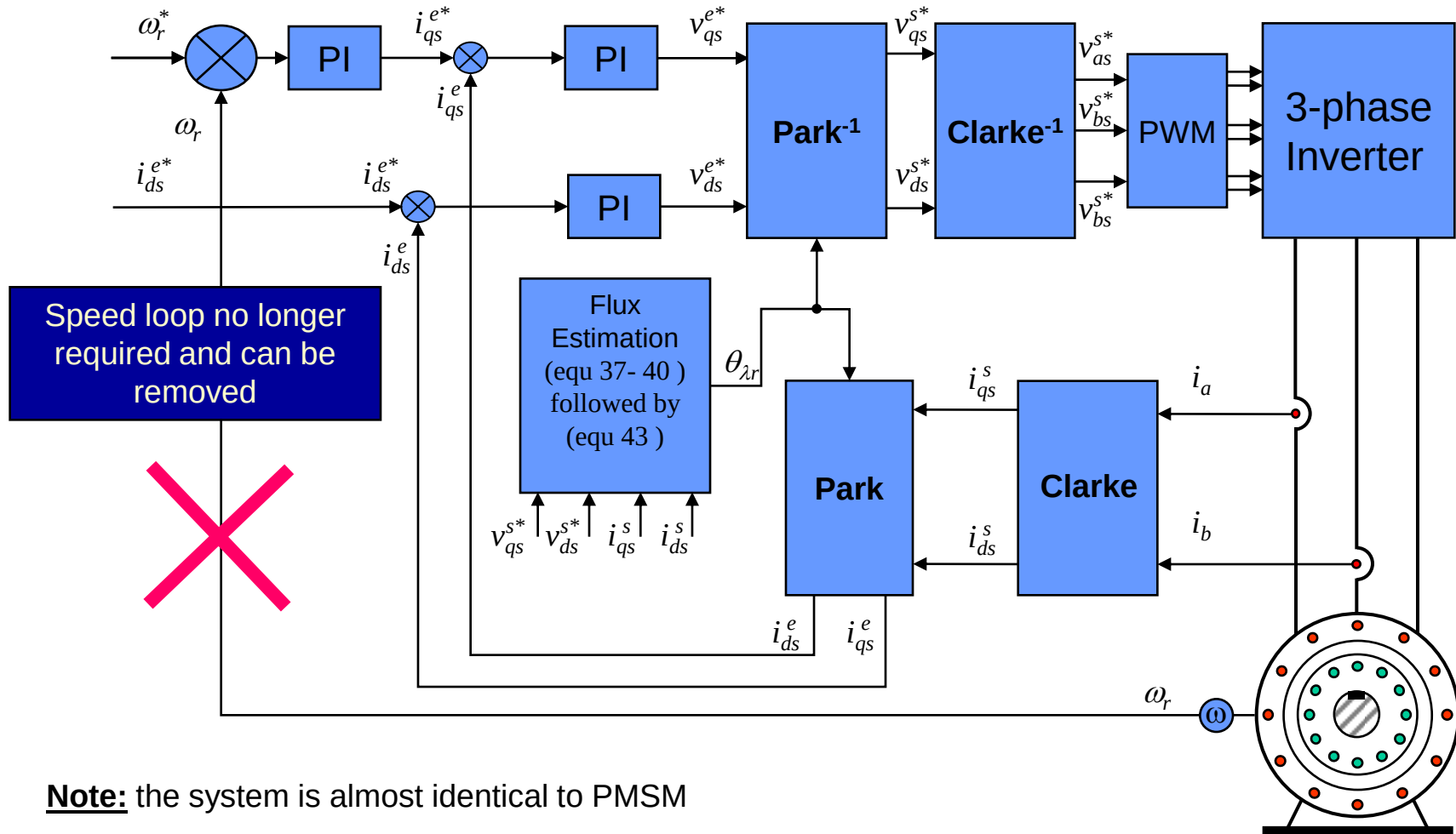
Direct Vector Control

We can now calculate the position of the rotor flux θ_{λ_r} :



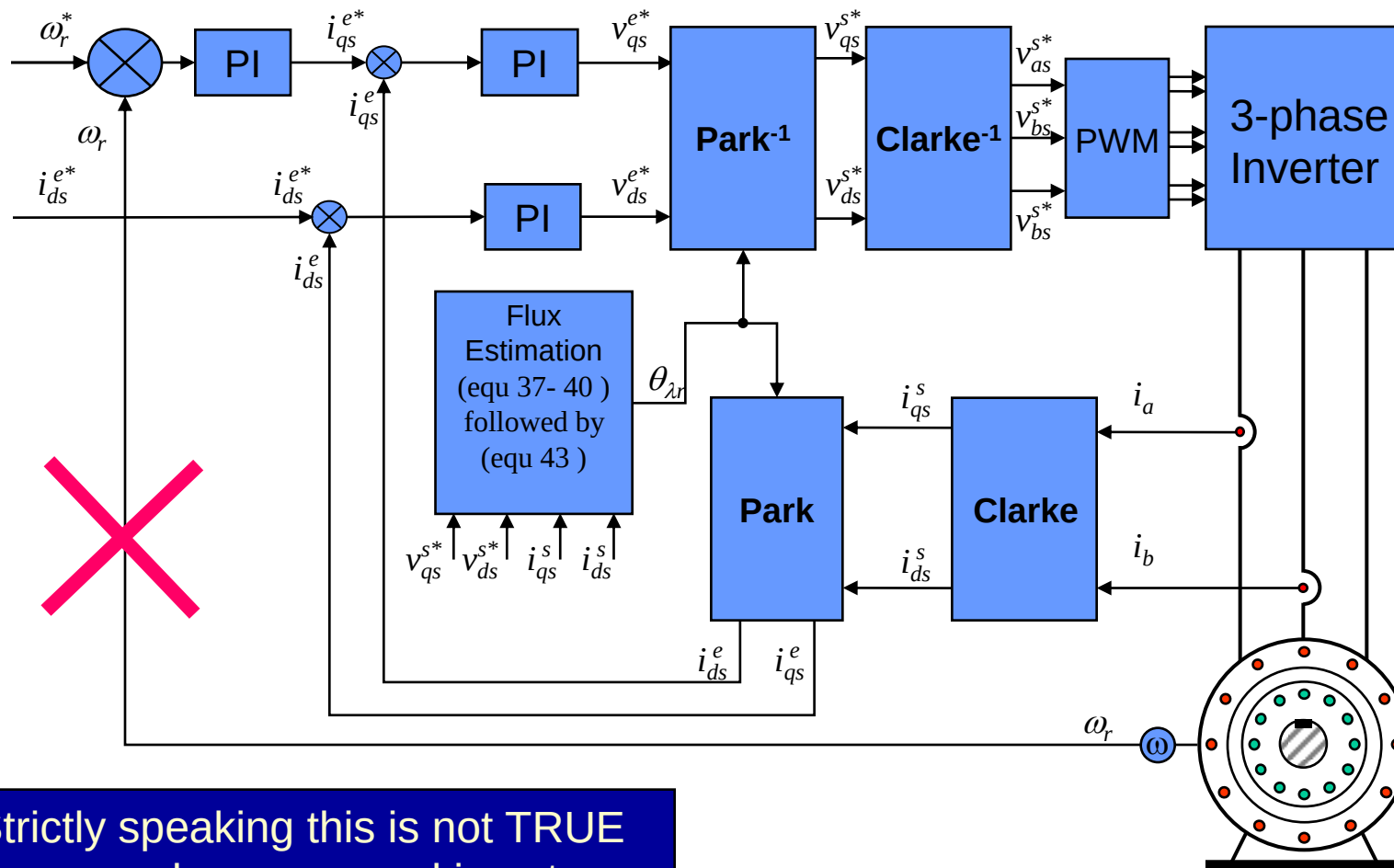
Direct Vector Control Block Diagram

Task 2 completed



Sensorless Vector Control Block Diagram

- In many applications a stiff speed loop is not essential and hence the feedback loop can be removed



Strictly speaking this is not TRUE sensorless as speed is not estimated and fed back

True Sensorless Direct Vector Control

- For true Sensorless speed control we also need to evaluate ω_r . There are many different ways of doing this:
 - Manipulation of machine model equations (Bemf)
 - MRAS based
 - Observer based
 - Kalman Filters
- A common and easy way is to differentiate θ_{λ_r} to get ω_e and then subtract out the estimate of slip:

$$\omega_r = (\omega_e - \omega_{sl})$$

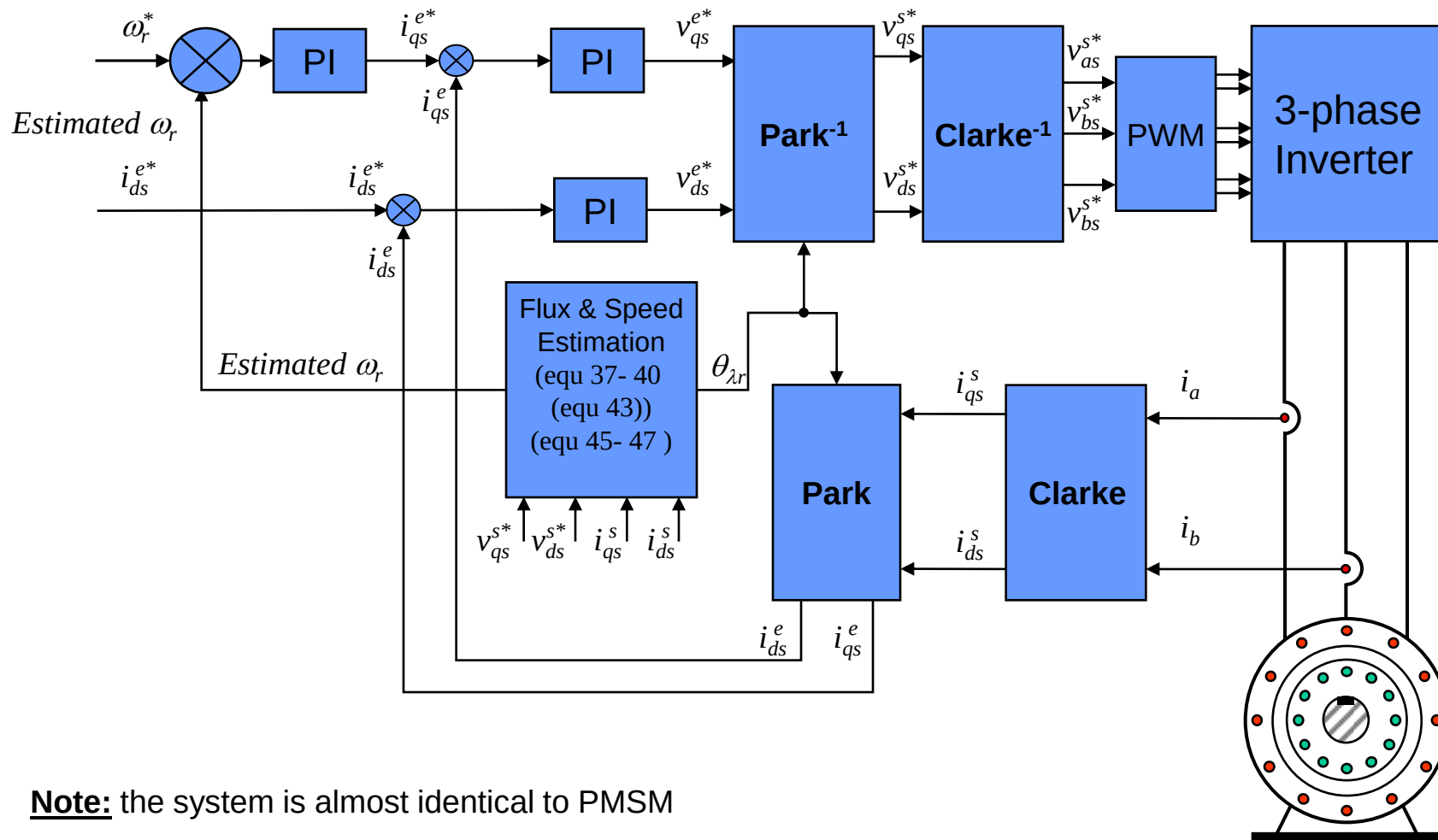
Where

$$\omega_e = \frac{d\theta_{\lambda_r}}{dt}$$

$$\omega_{sl} = \frac{1}{|\lambda_r|} \frac{L_m R_r}{L_r} (\lambda_{dr}^s i_{qs}^s - \lambda_{qr}^s i_{ds}^s)$$

**This is the method used in the
ACI_SE block within TI DMCLib**

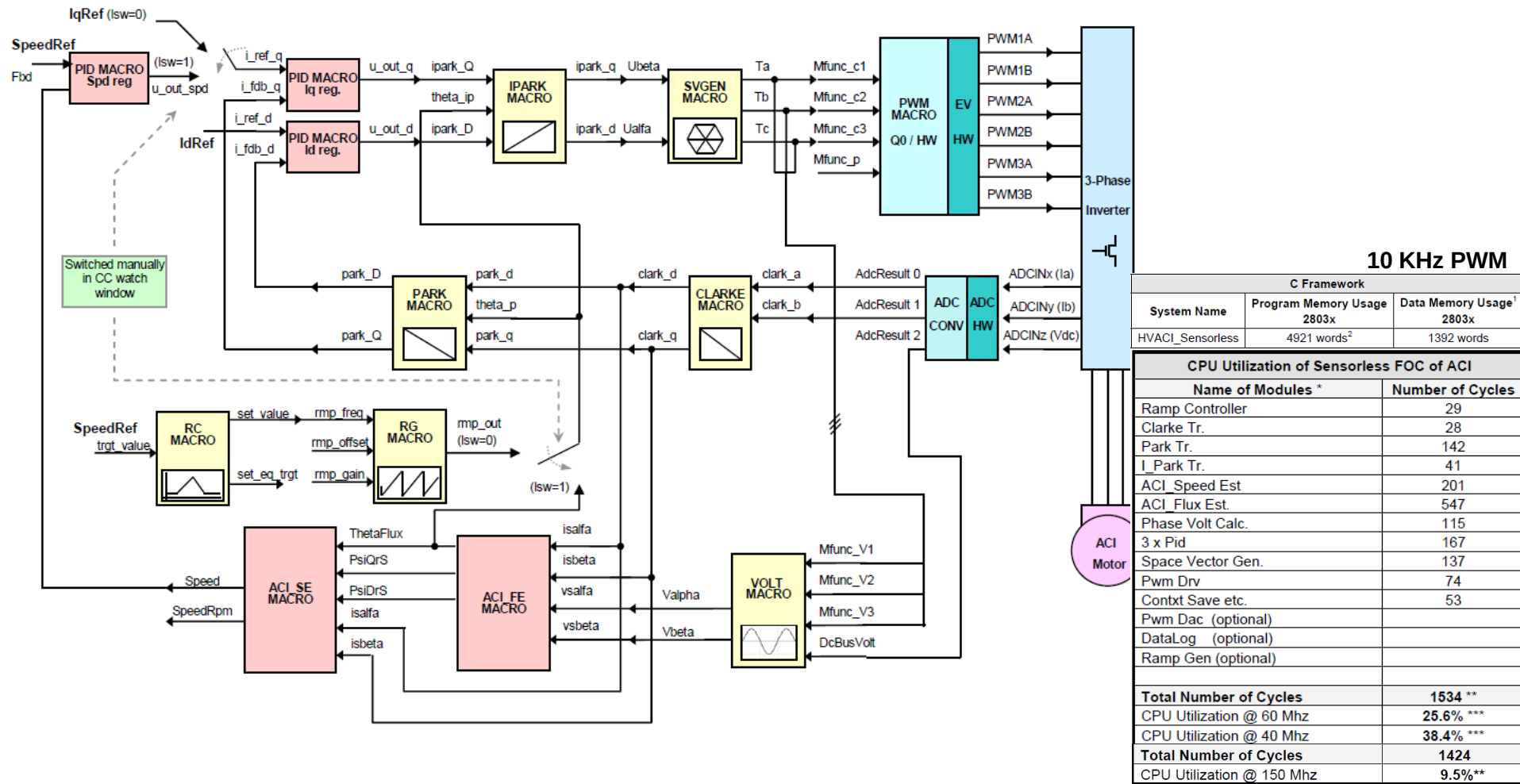
True Sensorless Vector Control Block Diagram



Note: the system is almost identical to PMSM

TI Systems for Sensorless Direct FOC of ACI

- Available for Piccolo (fixed) and Delfino (float)
- www.ti.com/controlSUITE and choose High Voltage Kit

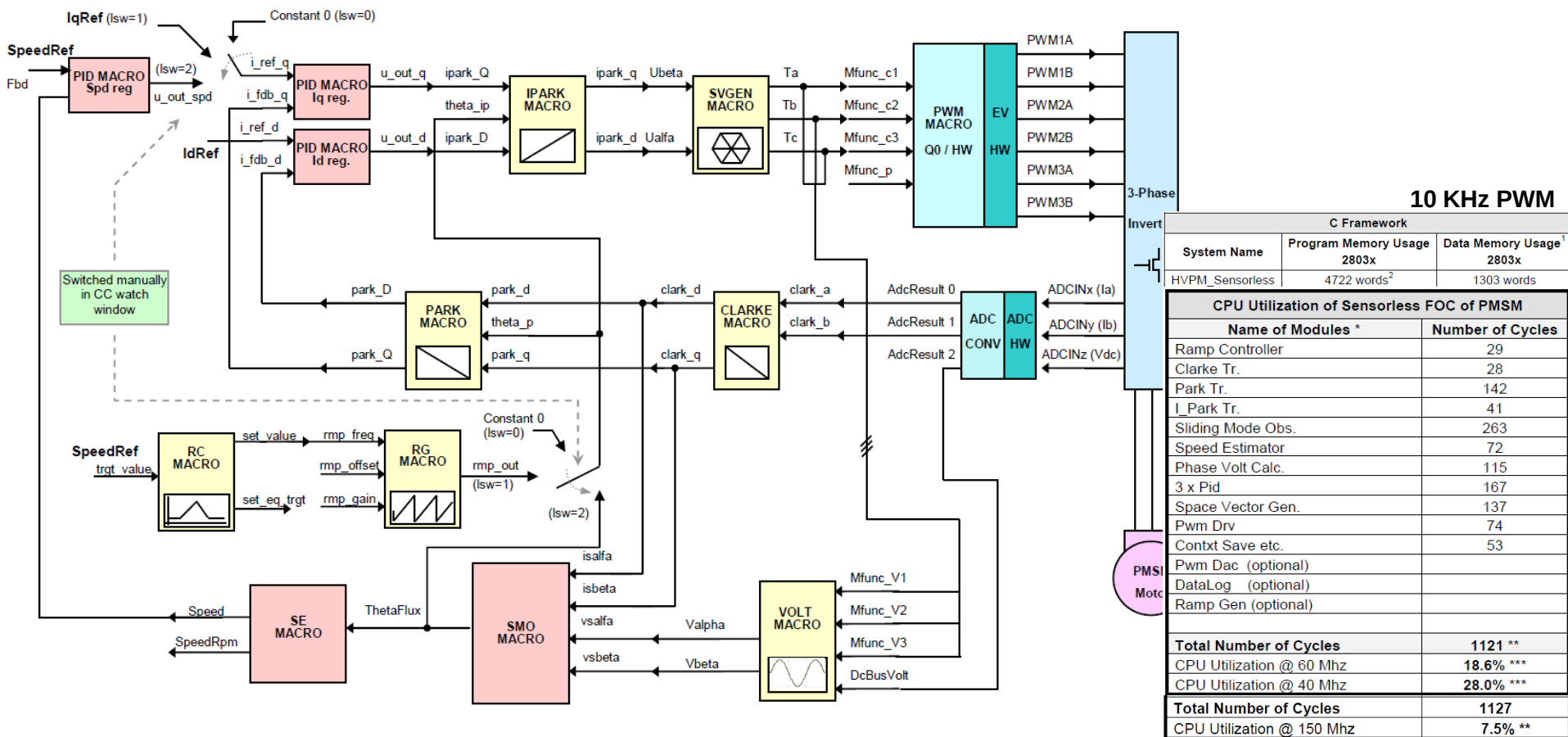


Vector Control for PMSM

- Our description of vector control has been based on induction motors, for PMSM the principles stay EXACTLY the same i.e. we are still trying to locate the position of rotor flux $\theta_{\lambda r}$.
- The block diagrams are almost identical, but for a **permanent magnet** machine the position of the rotor flux is fixed with respect to θ_r
 - i.e. if we know θ_r then we also know $\theta_{\lambda r}$
- For sensed vector control all we need is θ_r
 - Piccolo and Delfino (float) versions available in controlSUITE with High Voltage Kit
 - Multiple Legacy F281x/0x versions (speed/position with encoder, resolver) available at www.ti.com/c2000dmc
- For sensorless vector control we need to estimate the position of the rotor flux:
 - This can be done in a few different ways: machine model equations, sliding mode controller, Kalman filters, MRAS etc.
 - Sliding Mode Observer for Piccolo & Delfino in controlSUITE with High Voltage Kit and Dual Axis Low Voltage Kit (kit used for this DMC Workshop)

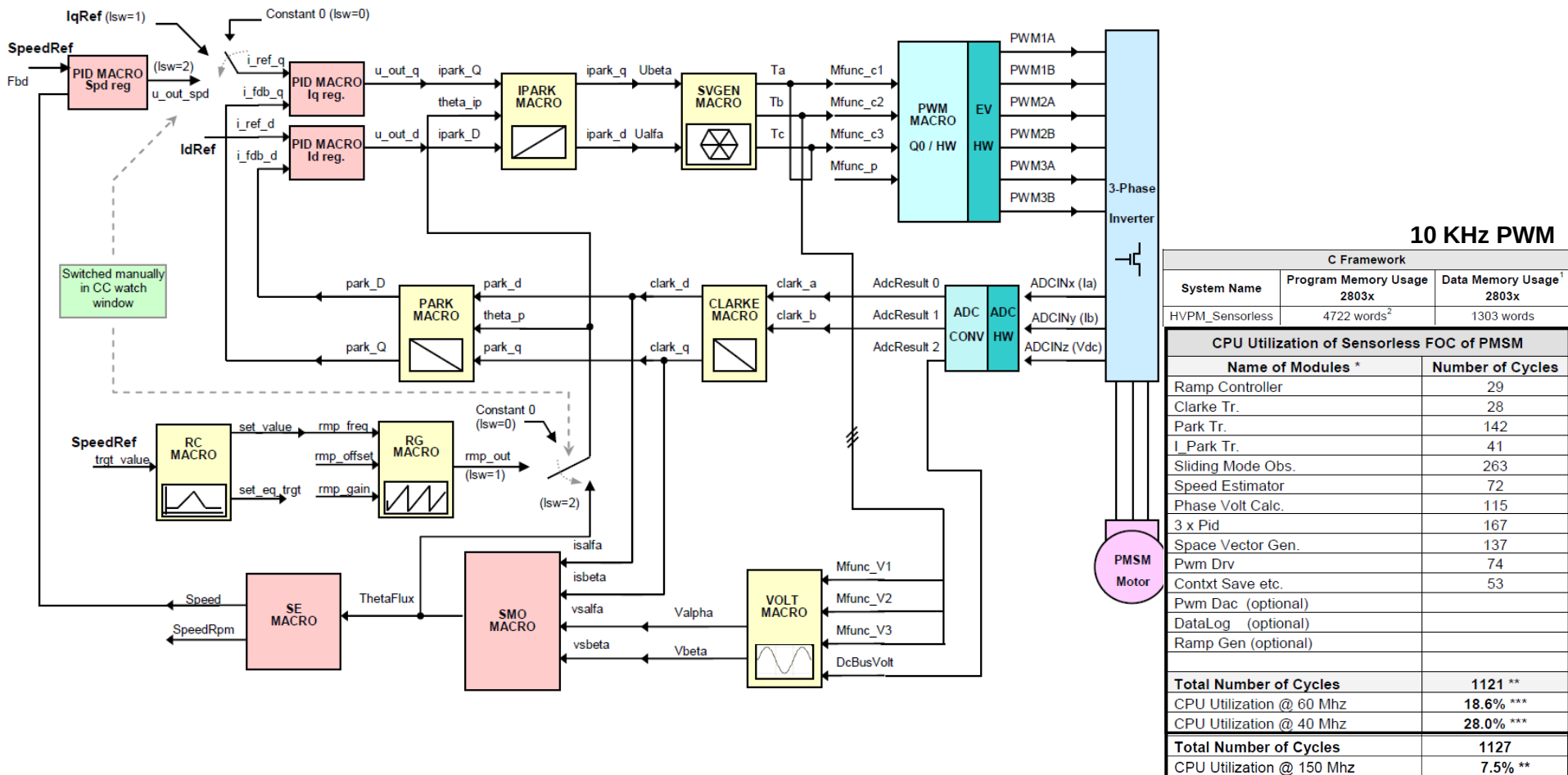
TI Systems for Sensored (Encoder) FOC of PMSM

- Available for Piccolo (fixed) and Delfino (float)
www.ti.com/controlSUITE and choose High Voltage Kit
- Legacy Resolver system at www.ti.com/c2000dmc



TI Systems for Sensorless (SMO) FOC of PMSM

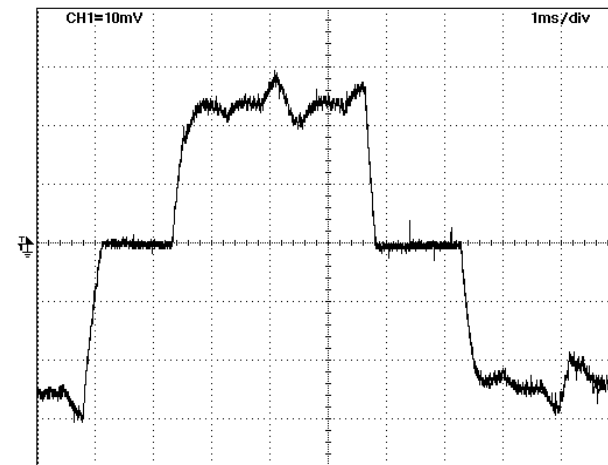
- Available for Piccolo (fixed) and Delfino (float)
- www.ti.com/controlSUITE and choose High Voltage Kit or Low Voltage Kit



Using BLDC Motors with Field Orientation

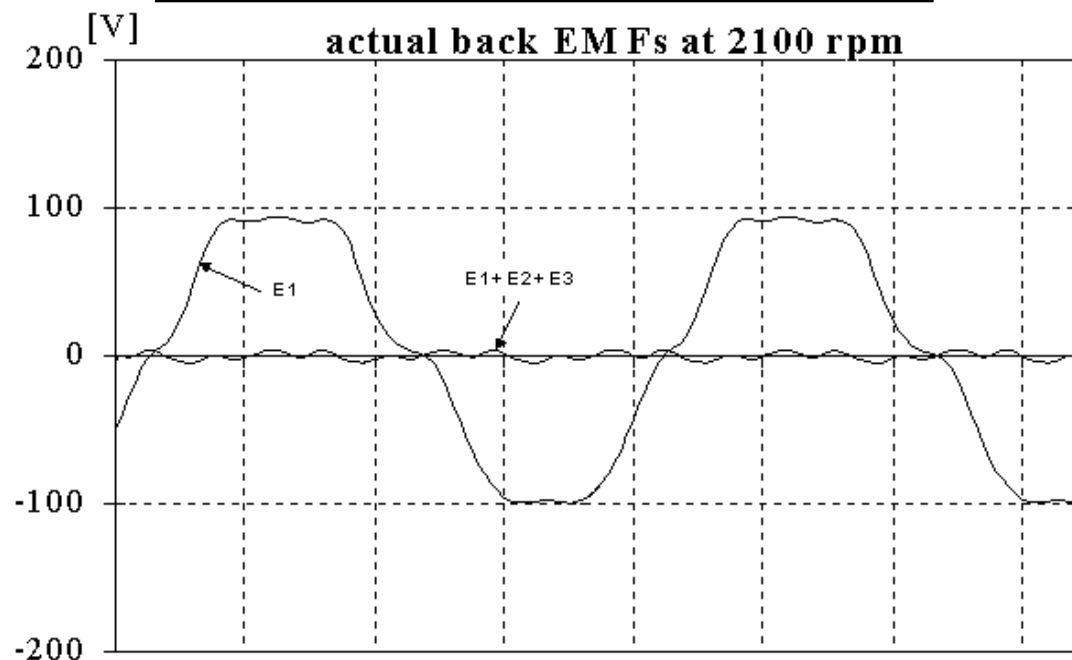
- As mentioned in the previous slides BLDC motors have a trapezoidal B_{emf} and are driven with DC currents
- There will be more acoustic noise as these waveforms are at frequencies within the audible range
- In many cases it is possible operate our BLDC motor under field orientation with sinusoidal currents to resolve these issues
 - Many cheap BLDC motors have a sinusoidal B_{emf} with a flat top
 - Some manufacturers in fact call their motors BLDC despite the fact that the motor has a sinusoidal B_{emf}
- Always check the shape of the B_{emf} with an oscilloscope

BLDC current with hall effect control



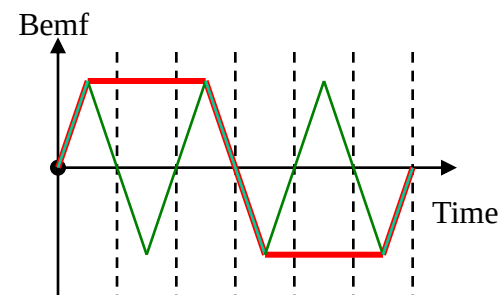
When Can We Use BLDC with Sinusoidal Currents?

Many BLDC motors have a sinusoidal B_{emf}. Hence can be operated with sinusoidal waveforms



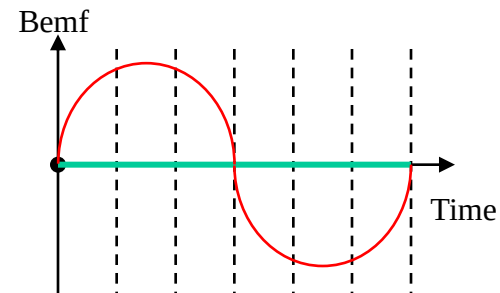
Trapezoidal B_{emf}:

$$E_1 + E_2 + E_3 \neq 0$$



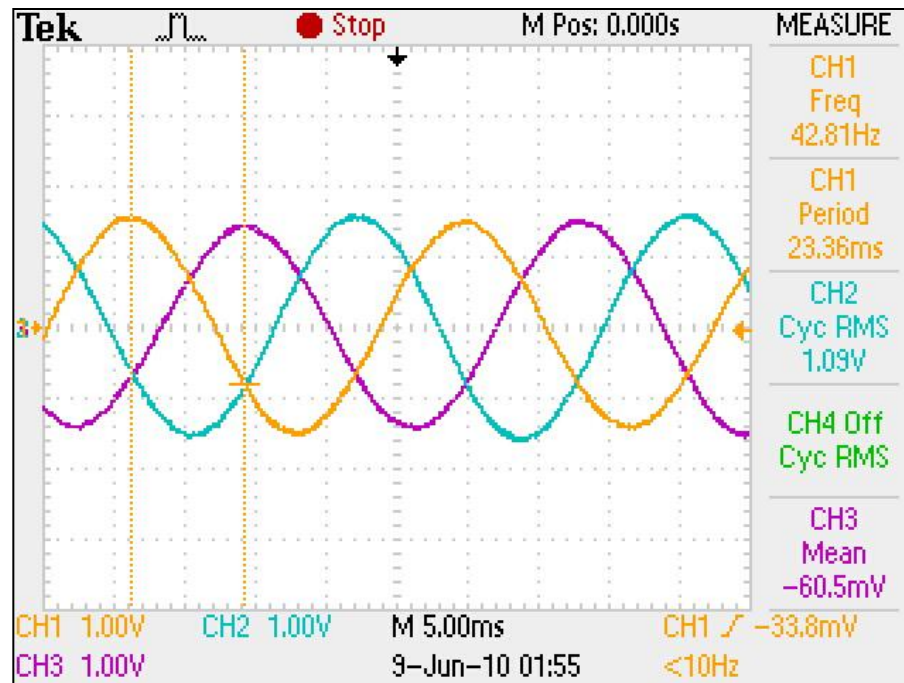
Sinusoidal B_{emf}:

$$E_1 + E_2 + E_3 = 0$$



The B_{emf} of Our Workshop BLDC Motor

- The B_{emf} of the workshop motor was observed by attaching a hand-drill to the shaft and viewing the generated waveforms on an oscilloscope
 - As can be seen we have perfect sinusoids 120° apart – hence we can use sinusoidal waveforms to operate it under vector control



4. Motor Drive Implementation

- **3 Phase Electronic Inverters:**
 - Pulse width modulation
 - 3rd harmonic injection
 - Space vector modulation
- **C2000 Family of MCUs**
 - Architecture
 - Dedicated motor control peripherals
 - Hardware/software and development tools
- **Lab Exercises**
 - Implementing Field Oriented (Vector) Control on a synchronous motor using TI's Piccolo F28035 MCU, Dual Axis Low Voltage Kit, and controlSUITE Digital Motor Control Library

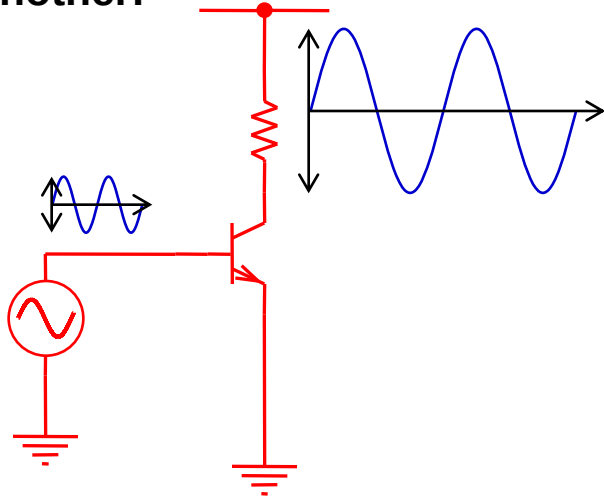
Making Our Workshop Motor Turn

- From our discussions so far, we know how to operate our 3 phase motor – in the case of our workshop motor:
 - We excite our 3 phase motor with three generated sinusoidal currents which have a 120° phase shift with respect to each other
 - Our 3 phase currents will produce a rotating magnetic field in the center
 - The rotor of our motor is made of permanent magnets. This rotor will follow the rotating magnetic field and therefore we do not need commutators and brushes i.e. Brushless operation
 - All we need to do now is to generate our 3 phase currents

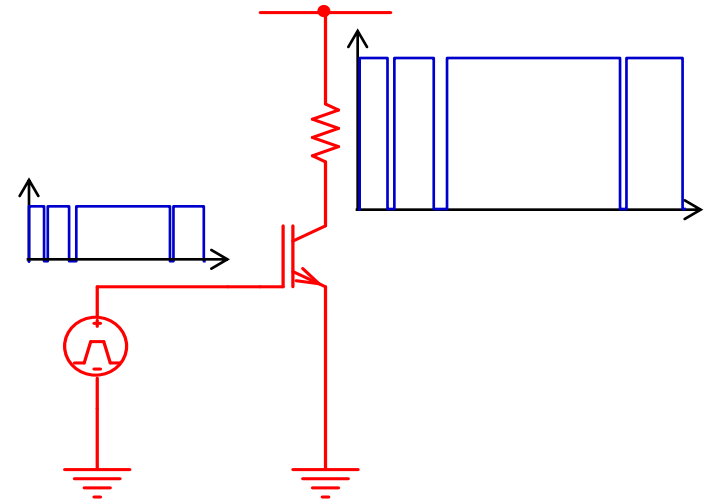
All we have to do to operate our workshop motor is to learn how to use the C2000 MCU to generate 3 phase currents

Generating 3-phase Currents and Pulse Width Modulation

We need to inject our motor with three sinusoidal currents 120° apart from one another:

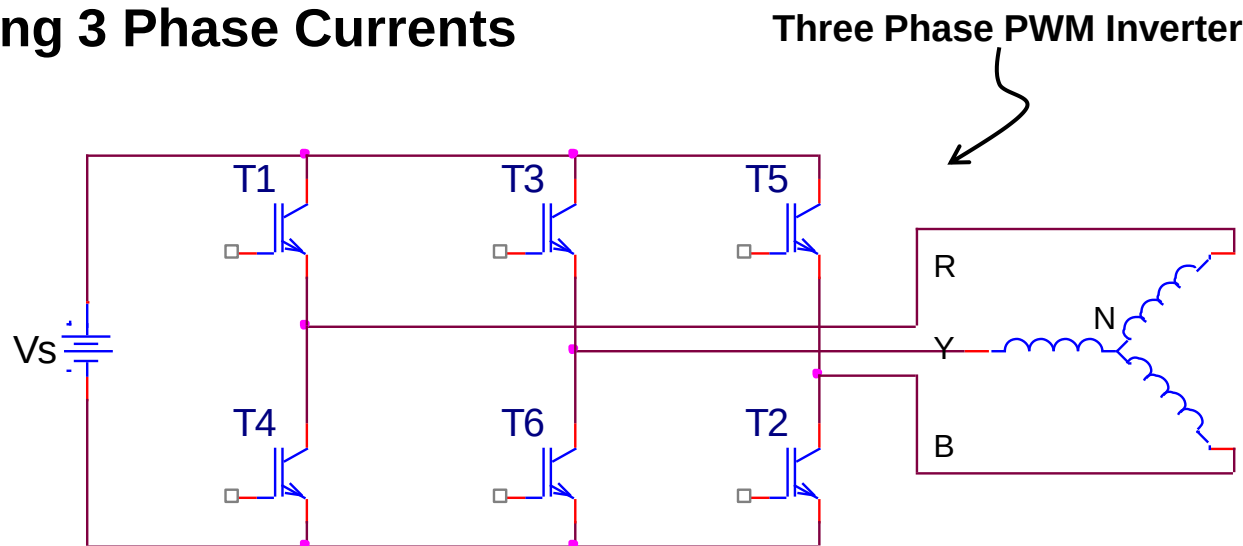


- Using a transistor in the linear region is not a good idea:
 - There will be excessive losses in the transistor
 - Microprocessors are not very good at generating the small sinusoidal signal needed for the base of the transistor



- Ideally we would like to use the transistor as a switch in the saturation region:
 - We will have much lower losses
 - Microprocessors are very good at generating the small square wave signal needed for the base of the transistor

Generating 3 Phase Currents



We can switch our transistors in a sinusoidal PWM manner in order to inject 3 sinusoidal currents into our motor. The high frequency square wave voltage has a low frequency sinusoidal fundamental component.

The motor is very inductive (i.e. a low pass filter) and therefore will filter out the high frequency square wave component. Hence the currents will be sinusoidal + some noise at the switching frequency

Therefore, we can operate our motor with digital waveforms instead of analog i.e. ideal for microprocessors

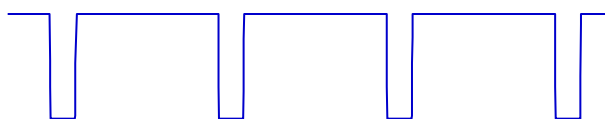
Pulse Width Modulation

We only want to switch the transistor/IGBT/Fet on or off. By keeping the frequency of our square wave into the Gate/Base constant and changing the pulse width (i.e. duty) we can control how much power is delivered into our system.

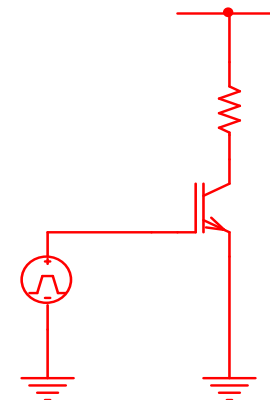
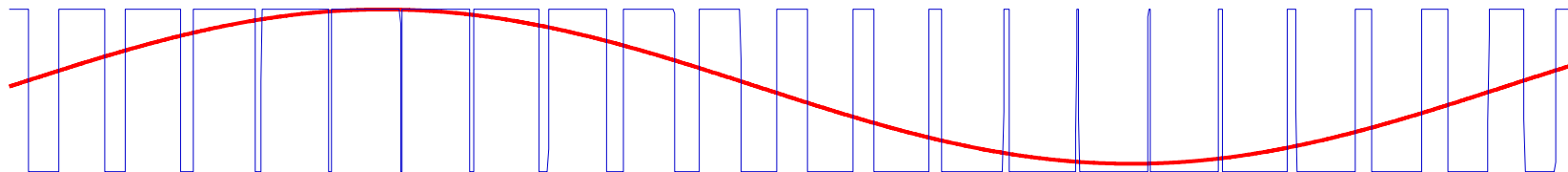
A 10% duty $\Rightarrow$ 10% power injected in to the system



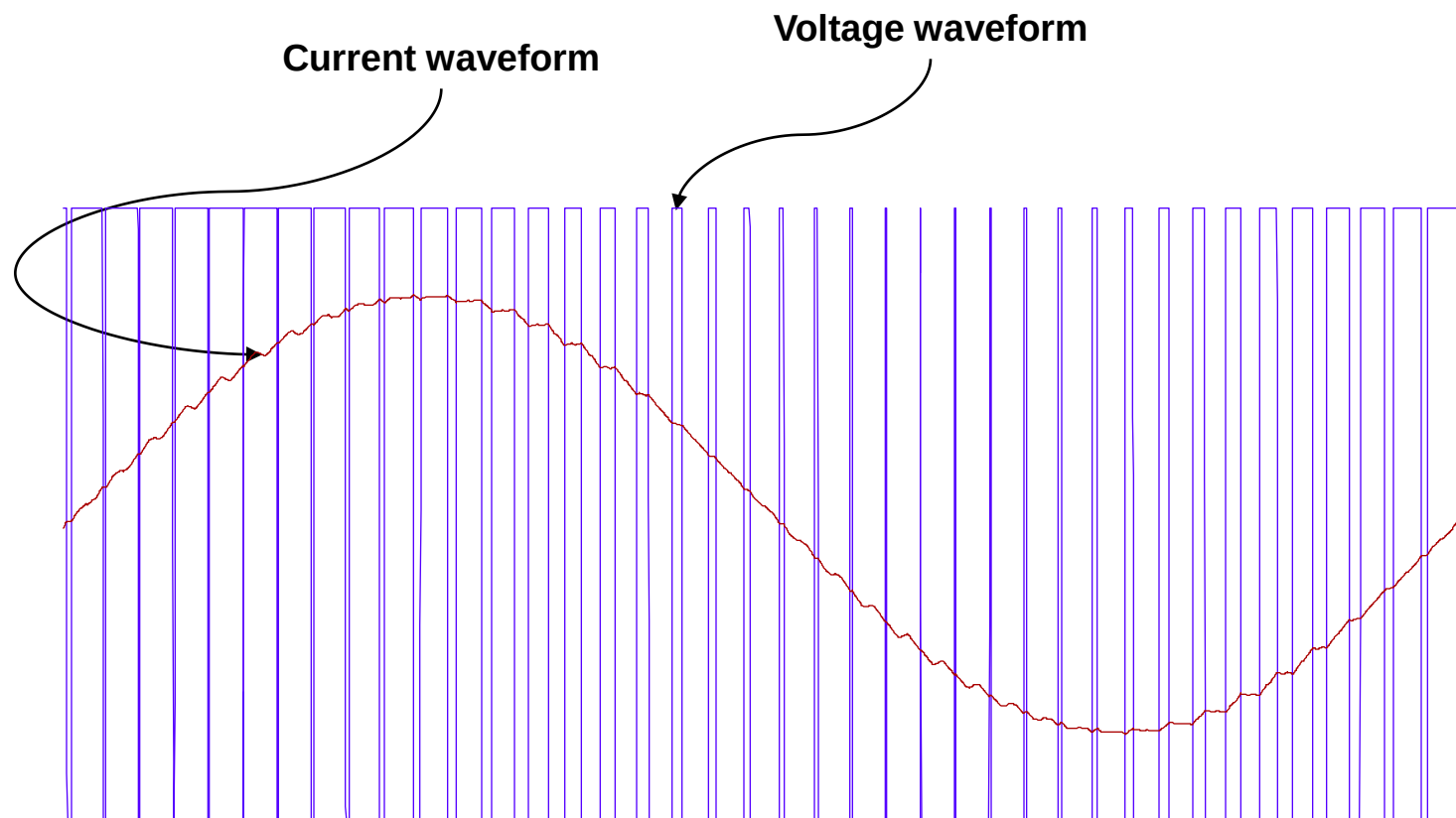
A 90% duty $\Rightarrow$ 90% power injected in to the system



and most importantly: varying our duty sinusoidally $\Rightarrow$ sinusoidal power injected into the system



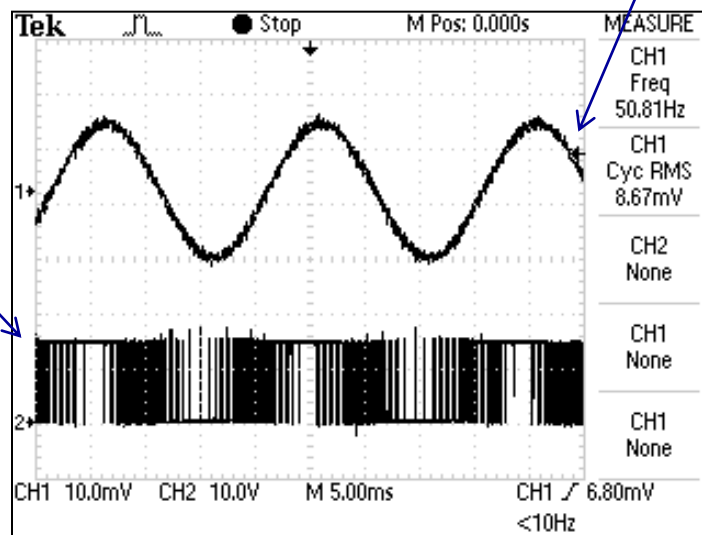
Three Phase PWM Inverter



This is a simulated waveform and is grossly exaggerated so that we can observe the high frequency voltage wave. An experimental waveform is shown in the next slide.

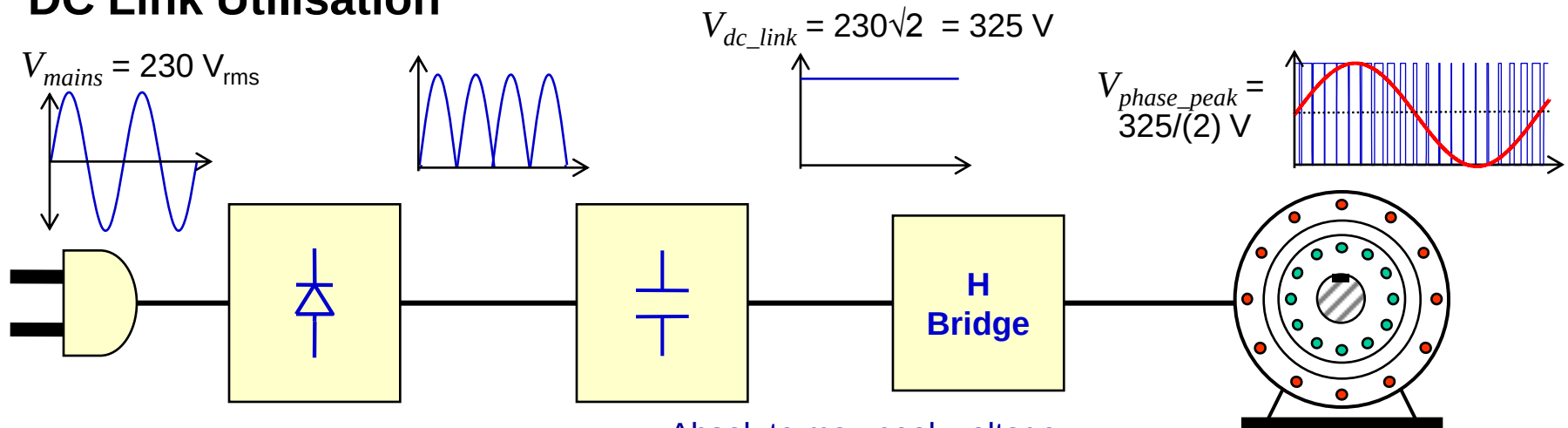
50H.

50Hz sine wave current – due to the filtering action of the inductor



BUT: this is not the optimum way of utilizing our DC link voltage as the maximum voltage amplitude is quite limited

DC Link Utilisation



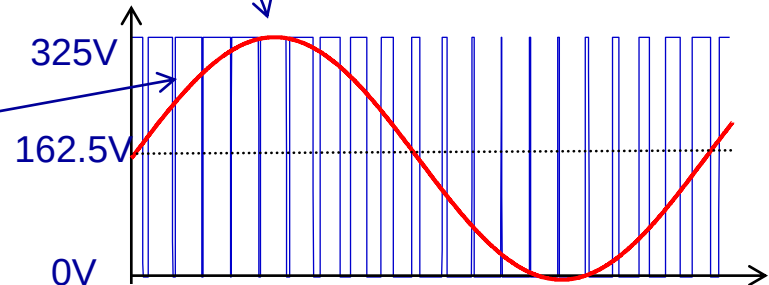
$$V_{dc_link} = \sqrt{2} V_{mains} = 325 \text{ V}$$

$$V_{Phase} = \frac{V_{dc_link}}{2\sqrt{2}} = \frac{325}{2\sqrt{2}} \approx 115 \text{ V!}$$

$$V_{Line} = \frac{\sqrt{3} V_{mains}}{2} = 0.866 \times V_{mains} \approx 200 \text{ V!}$$

Absolute max peak voltage

Motor **Maximum**
RMS V_{phase}



Standard PWM can create a maximum sine wave amplitude of only 86.6% of the mains voltage! This is not very good. It would be nice to be able to achieve a higher voltage on the motor terminals $\Rightarrow$ 3rd harmonic injection

Third Harmonic Injection

- We will show shortly that if, in addition to our fundamental sine wave, we also inject its third harmonic into our motor:
 - We can deliver the *same* amount of current that we inject into the motor with a *lower* DC link voltage
 - *Increase* the torque for the same DC link voltage
 - Utilizes the DC link better
- Our injected signal into the motor therefore becomes:

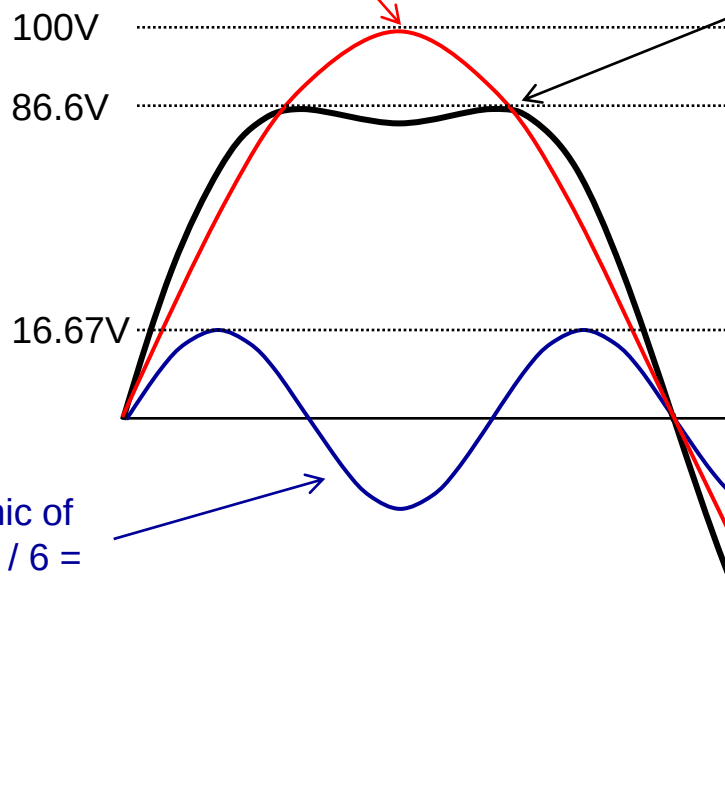
$$y = \sin \omega t + a \sin 3\omega t$$

- For best performance a is set to $1/6^{\text{th}}$ of the amplitude of the fundamental

$$y = \sin \omega t + (1/6) \sin 3\omega t$$

Third Harmonic Injection

Pure sine of height = maximum DC link voltage (say 100V)



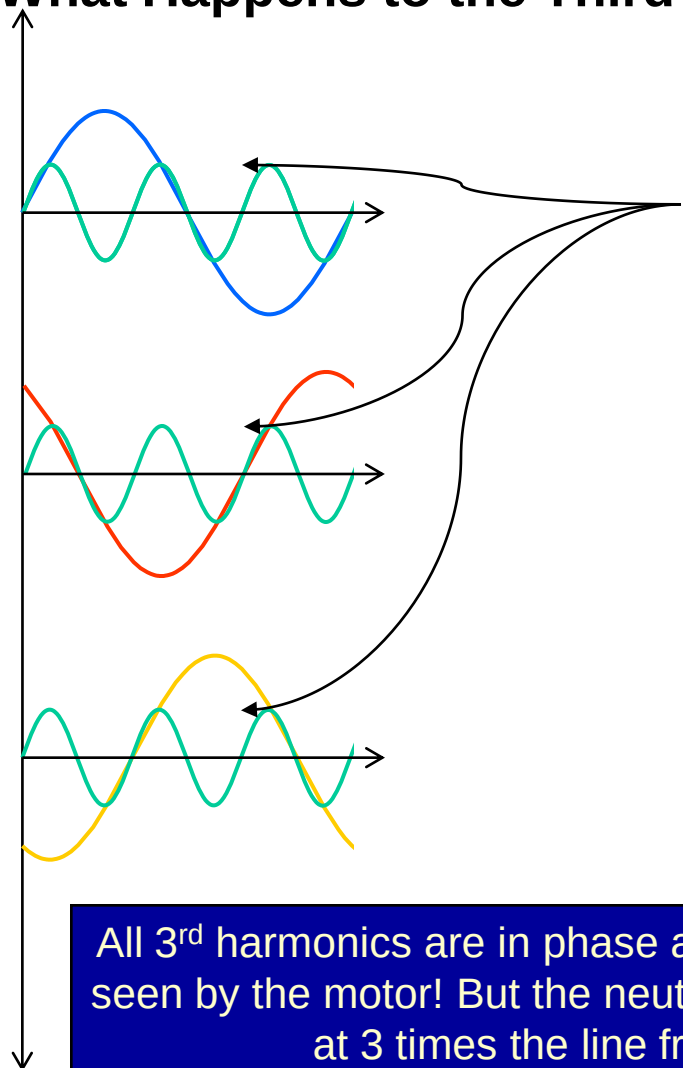
Its 3rd harmonic of height = $100V / 6 = 16.67V$

Resultant (i.e. sine wave + its third harmonic) will have a maximum value of $100 \times 0.866 = 86.6V$ but will deliver the same amount of power - we will explain this shortly

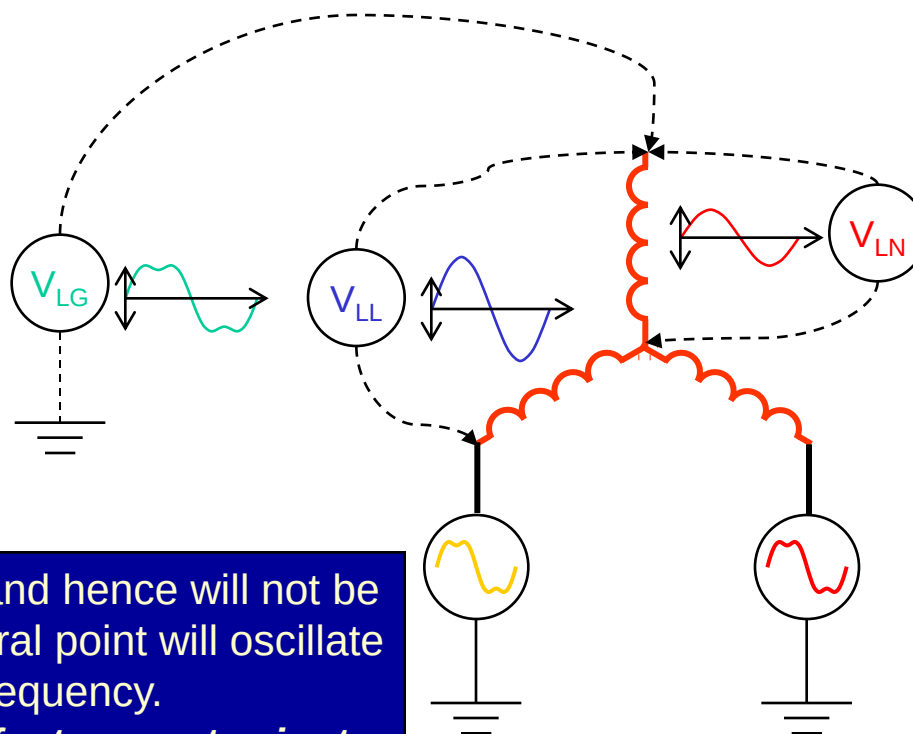
This means that we can increase the amplitude our injected signal (sine + 3rd harmonic) back up to DC link value and inject more current into our motor for the same DC link voltage

This is an “effective” DC link voltage increase of 15.5% - but will the injected 3rd harmonic cause issues with the motor’s operation?

What Happens to the Third Harmonic?



The 3rd harmonics of a three-phase balanced voltages are identical in amplitude and phase $\therefore$ provided that the Neutral point is not connected, they cancel each other out, leaving a sinusoidal $V_{\text{line-neutral}}$ and current

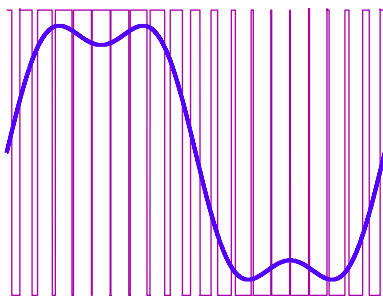


All 3rd harmonics are in phase and hence will not be seen by the motor! But the neutral point will oscillate at 3 times the line frequency.

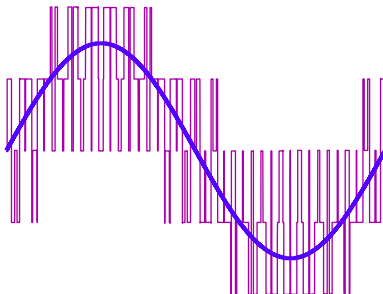
Therefore no detrimental effect on motor just more torque!

Voltage Waveforms for a Star Connected Motor

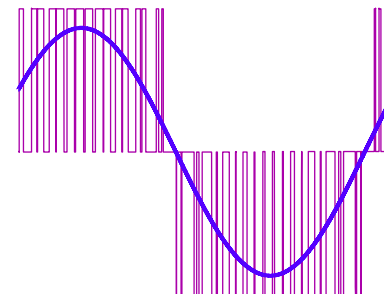
- Line to Ground voltage:
 - 3rd harmonic 86.6% of max DC link
 - So we can increase it by 15.5%



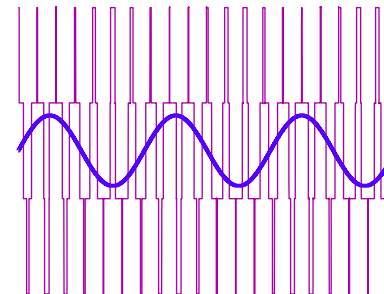
- Phase Voltage:
 - Note: no 3rd harmonic



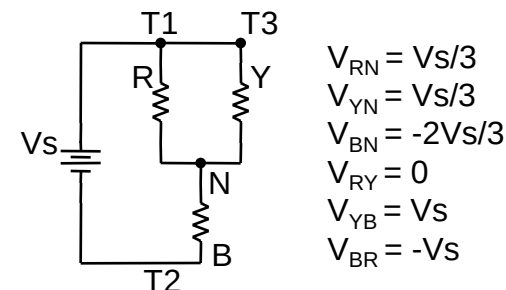
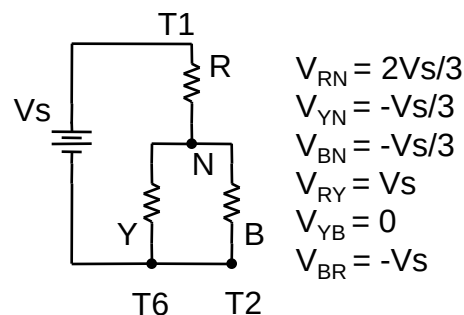
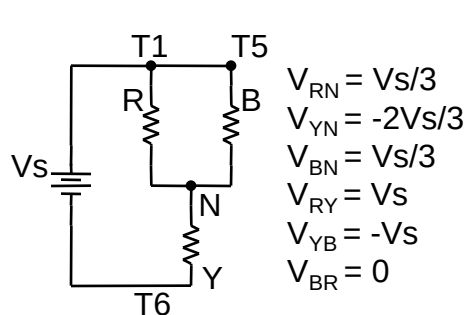
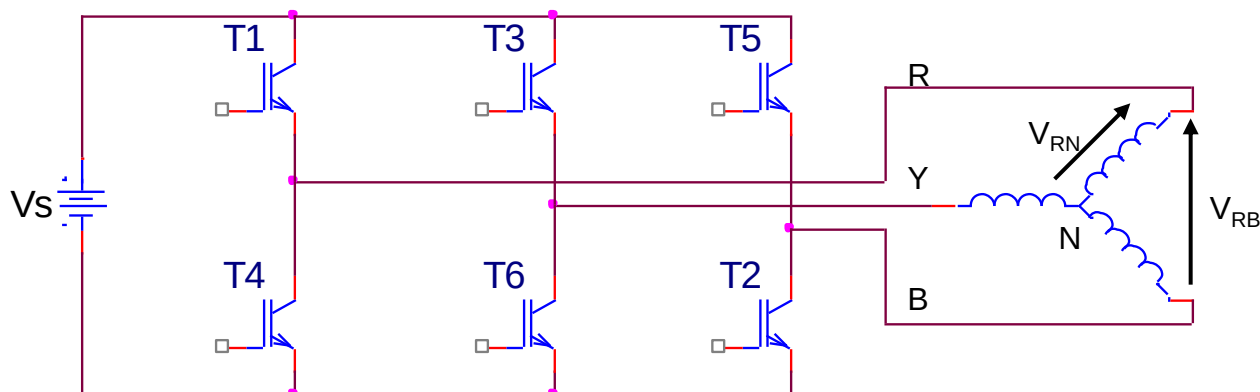
- Line voltage
 - Note: no 3rd harmonic
 - Amplitude = $\sqrt{3}$ × phase voltage



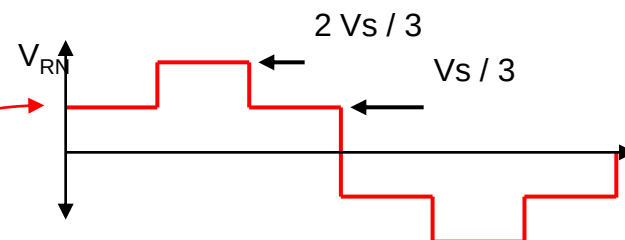
- Neutral to Ground voltage
 - Oscillates at 3 times the line frequency with respect to ground



Space Vector PWM



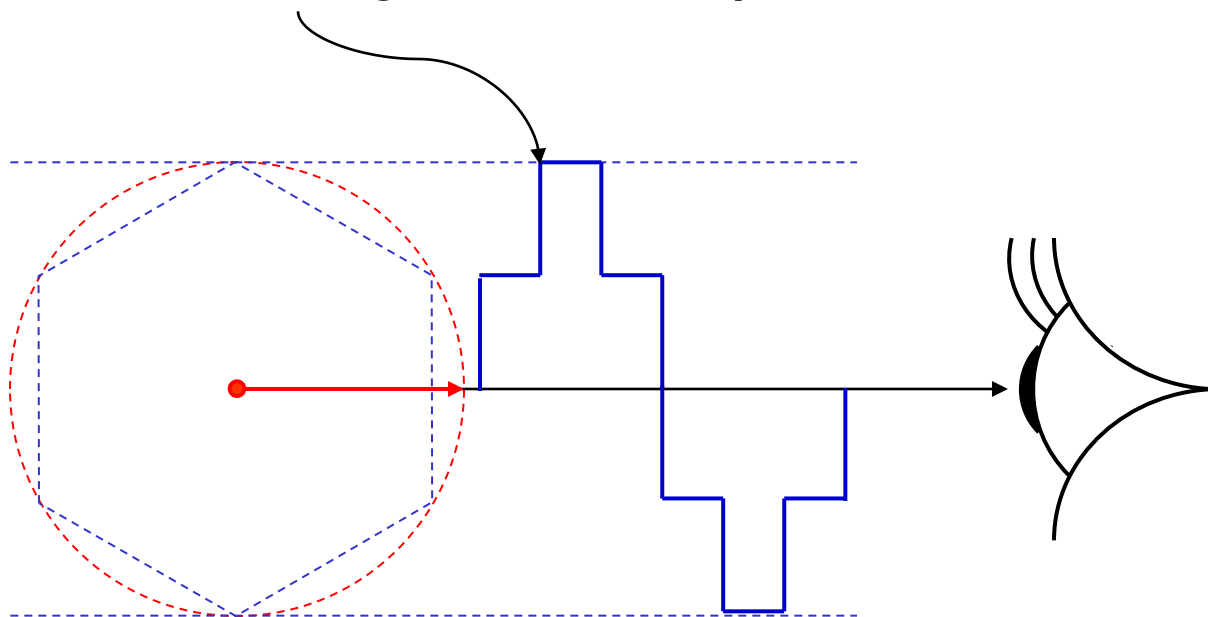
We can only generate 6 voltage levels: Vs , $\pm 2Vs/3$, $\pm Vs/3$; (8 levels if you count ± 0). At best we can generate a 6 step waveform as opposed to a sine wave.



Space Vector PWM

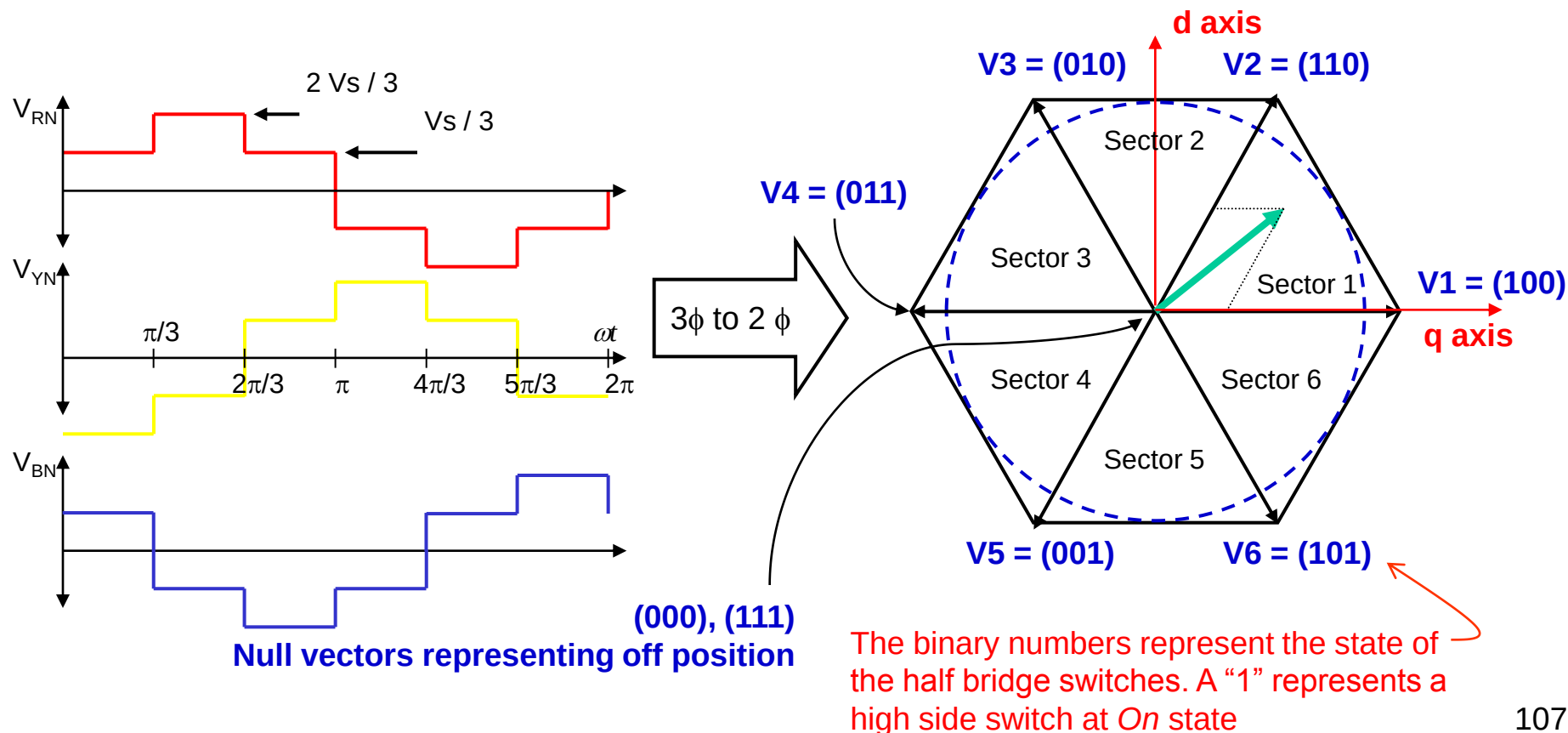
In an ideal world we could generate a perfect sinusoid.

In reality at best we can generate a 6 step:



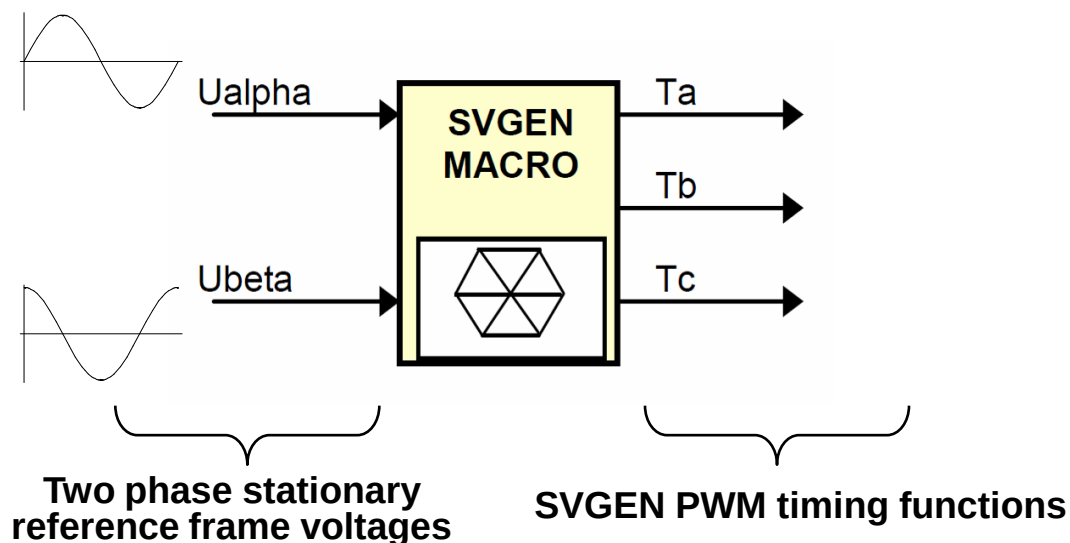
Space Vector PWM

- The resultant of these three voltages can be represented by one vector which can only exist in 6 predefined positions depending on the position of the switches
- To create our desired vector (i.e. green vector) we use high speed switching between our available voltage vectors including the Null vectors
- The algorithm for the switching patterns is non-trivial



Space Vector PWM Library Function

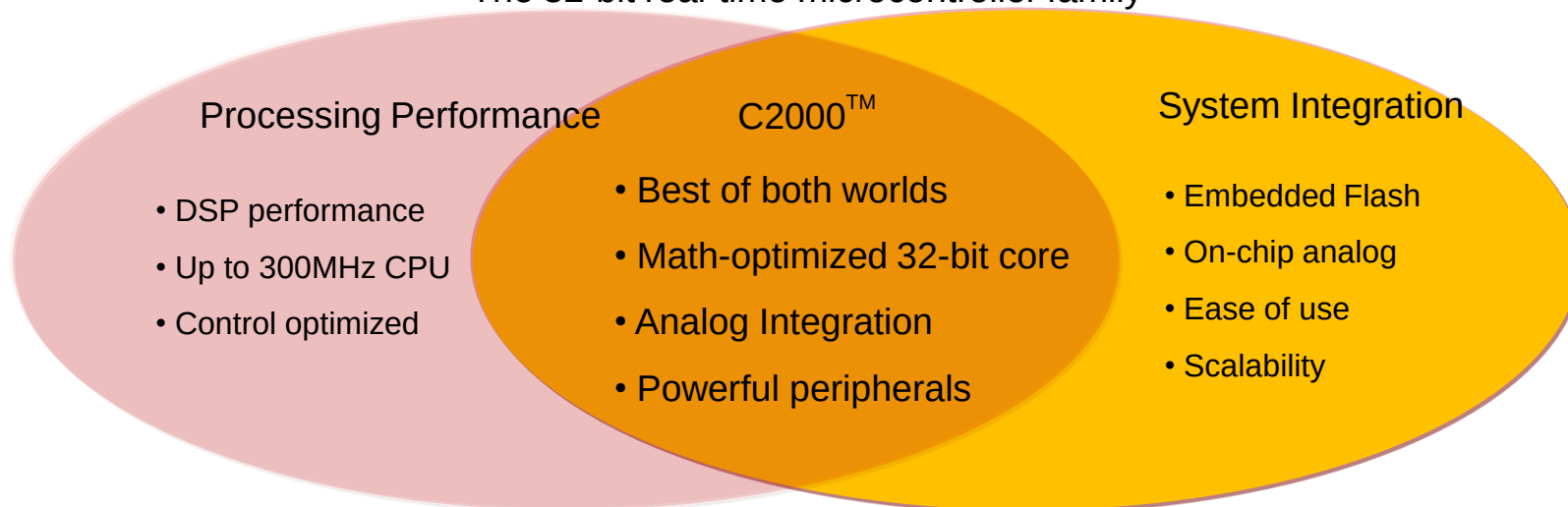
- TI provides this library function in an easy to use format within the DMC library
- This module calculates the appropriate duty ratios needed to generate a given stator reference voltage using space vector PWM technique



C2000 REAL-TIME MCUS FOR MOTOR CONTROL

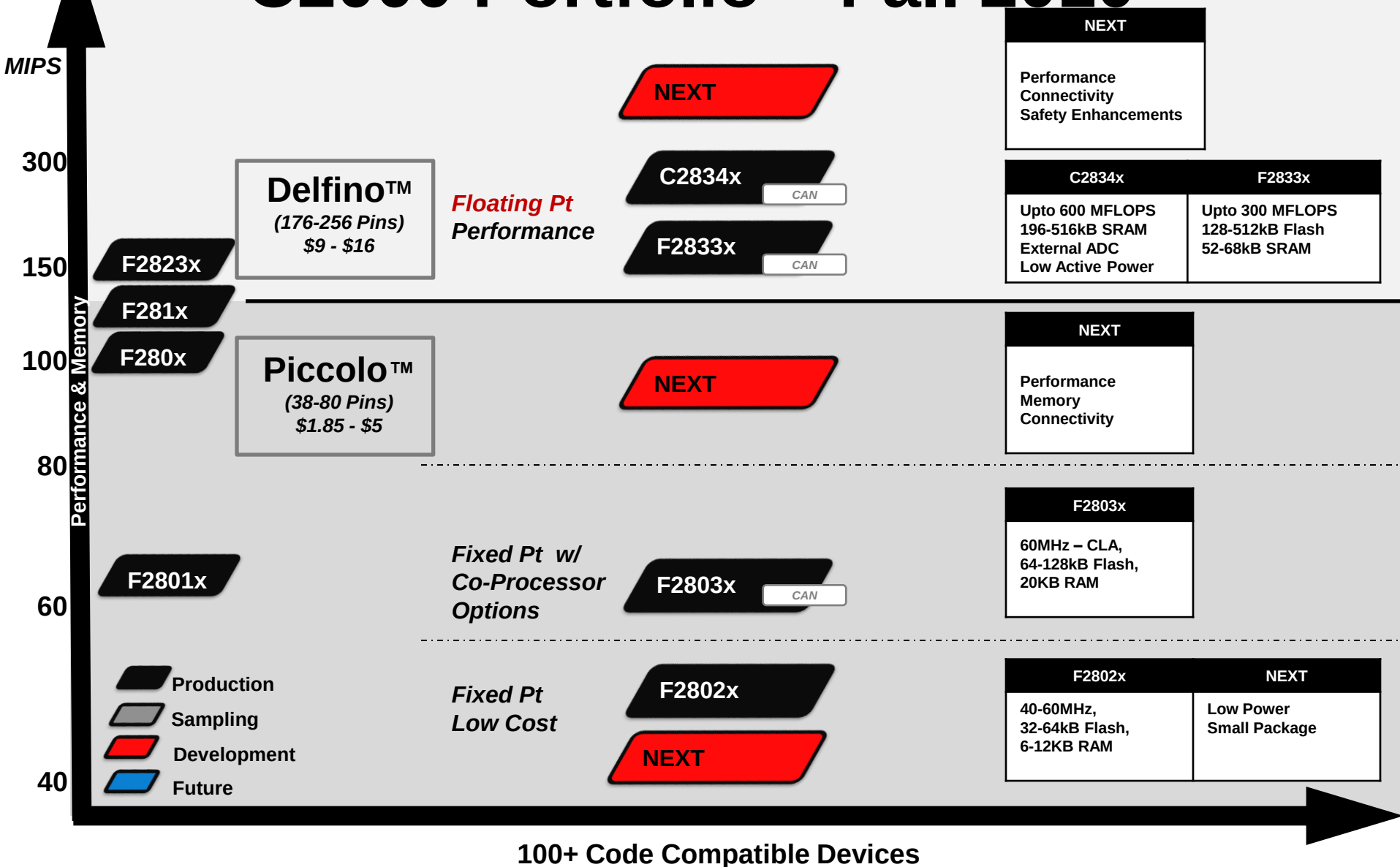
What is C2000?

The 32-bit real-time microcontroller family



- DSP performance within a Microcontroller architecture
 - 40-300MHz C28x CPU
 - Built-in DSP functions
 - Single Cycle 32x32-bit MAC
 - Control Law Accelerator
 - Fixed & Floating Point
 - Embedded Flash
- Fine-tuned for real-time control
 - Optimized core
 - Fast interrupts
 - Flexible interrupt system
 - Real-time debugging
- Comprehensive Peripheral Set
 - Best in class ADC performance
 - Flexible high resolution PWMs
 - Advanced Capture, Quadrature Encoder Interfaces
 - CAN, LIN, SPI, I2C, SCI/UART, McBSP
- Broad portfolio of configurations
 - 40-300 MHz
 - Fixed and Floating-point devices
 - 32-512KB of Flash
 - From sub \$2 to \$20
 - Software compatibility across C2000 family
 - *controlSUITE*

C2000 Portfolio – Fall 2010



C2000 Motor Control Applications

Transportation

E-bike



HEV Traction



Electric Power Steering



Off-Highway

CAN Gateways



*Medical
Pumps / Blowers*



Elevators



General Purpose

*Process Control
(Pumps, Fans, Valves)*



Power Tools



Drives & Automation

AC Drives



*Programmable
Logic + Servo*



*Soft Start &
Protection*



Robotics & CNC



Refrigeration



Sewing



Appliance

Washers

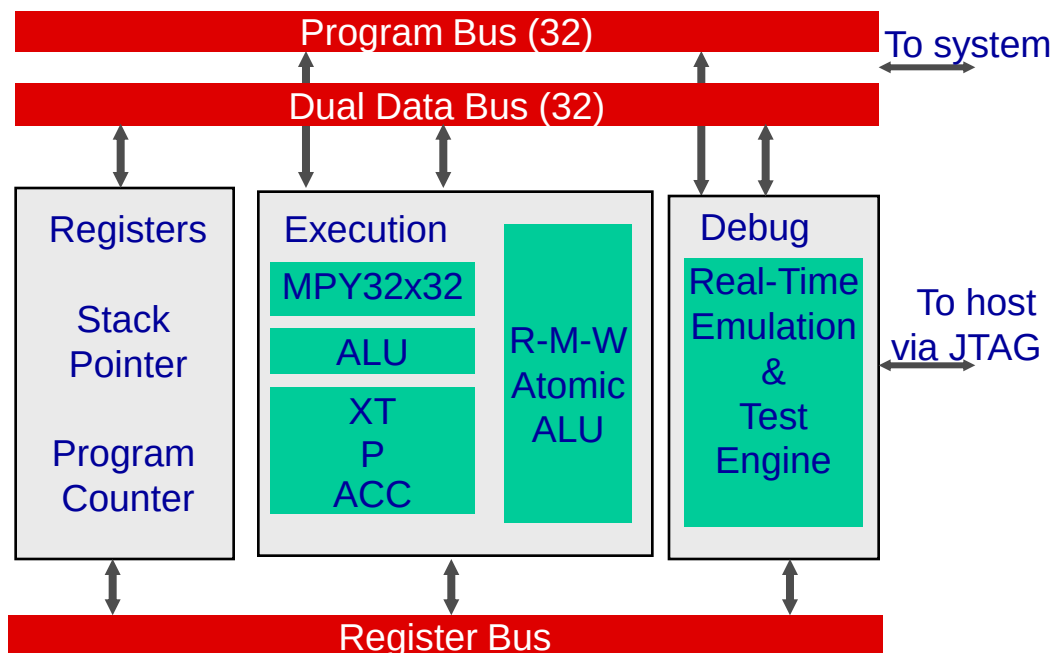


HVAC



**Variable Speed
Precision Position
High Efficiency
High Performance
Multi-Axis
System Integration
PFC**

The 32-bit C28x core is at the heart of every C2000 28x microcontroller. Based on a DSP architecture, the core is optimized to quickly execute math-based operations, but can also handily process general-purpose code.



C28x CPU

- 32-bit fixed-point DSP
- RISC instruction set
- 8-stage protected pipeline
- 32x32 bit fixed-point MAC for single-cycle 32-bit multiply
- Dual 16x16 bit fixed-point MACs
- Single-cycle instruction execution

Modified Harvard Bus Architecture

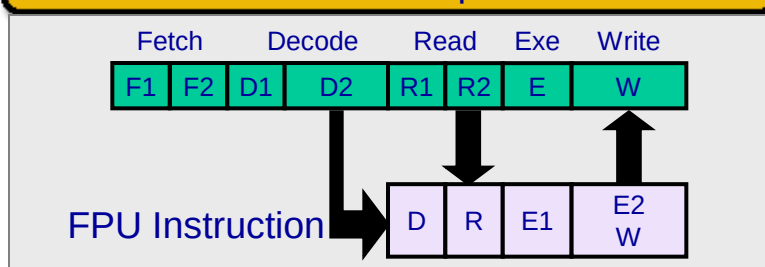
- Separate data and instruction buss
- Two data buses – one for read, one for write
- Enables fetch, read, and write in a single cycle

Emulation Logic

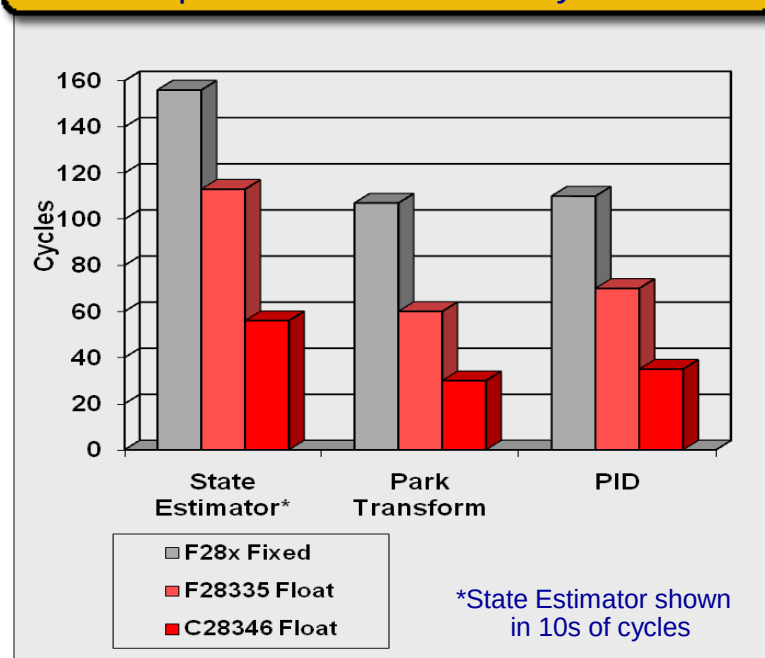
- Real-time emulation allows interrupt servicing even when main program is halted
- Debug host has direct access to registers and memory
- Enables data logging to the debug host
- Multiple hardware debug events and breakpoints

The FPU on Delfino devices is a logic unit that extends the C28x core to include floating-point instructions. Supports full IEEE single-precision 754 (most widely used format).

C28x + FPU Pipeline



Up to 64% reduction in cycles



C28x + FPU

- Full fixed-point support with floating-point support
- FPU instructions share same first half of pipeline as fixed-point instructions, but has its own second half
- Floating-point
 - Is Inherently more robust
 - Removes scaling & saturation burden
 - Reduces time-to-market
- Using floating point can reduce the cycle count for math functions by 52%. In addition, C2834x devices have reduced memory access time, resulting in 64% cycle reduction over other 28x devices.

FPU compilation and execution

- Fixed-point C2000 processors use costly support libraries to simulate floating-point math, store variables in stack
- Floating-point C2000 processors natively support single-precision floating point instructions, store variables in floating-point registers
- Use simple compiler switch to assemble code using FP instructions
- Separate floating-point version of IQMath, Flash API, etc.

Software

- FPU, FastRTS, and Motor Control Libraries

Independent 32-bit floating-point math accelerator

Independent Operation

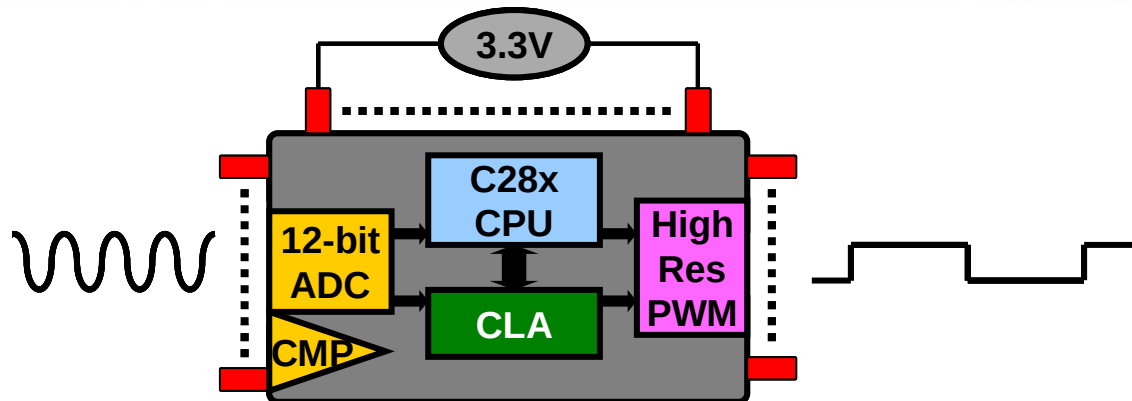
- Independent register set, memory bus structure & processing unit
- Low interrupt response time

Direct access to peripherals

- Execution of algorithms in parallel with the C28x CPU

Floating Point (32-bit)

- Removes scaling and saturation burden
- Assembly and C-callable Libraries only → meant for small fast loops



Reduced Sample-To-Output Delay

Faster System Response & Higher MHz Control Loops

Improved Support For Multi-Channel (Phase/Freq) Loops

Digital Power Applications

Dedicated engine for PFC

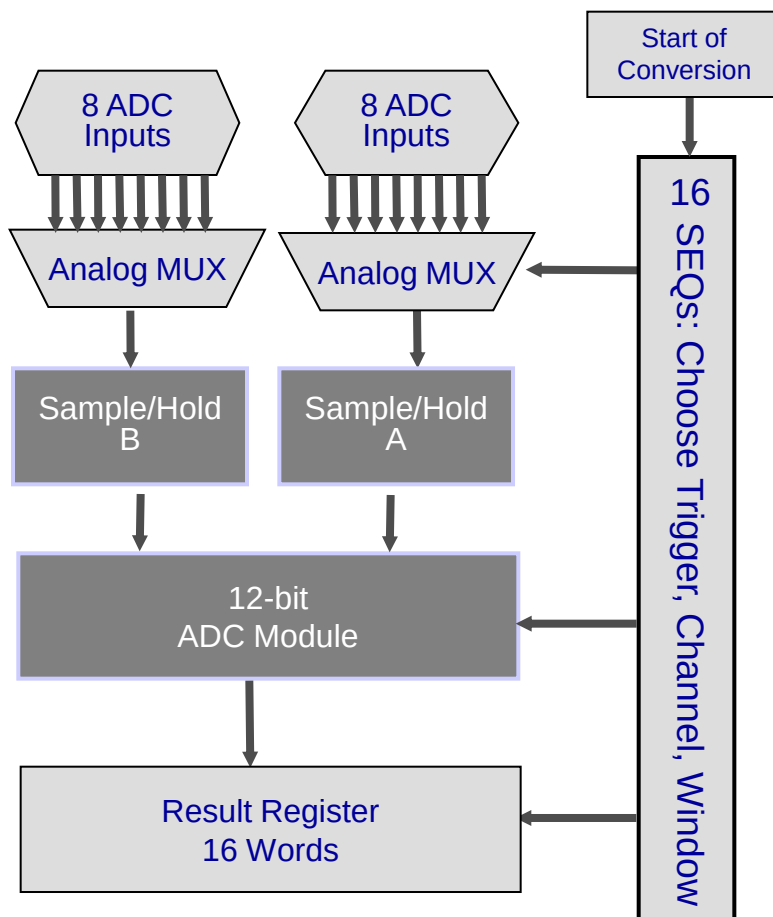
Motor Control

Free-Up C28x CPU For Other Tasks (communication, diagnostics)

General Purpose MCU Applications

controlSUITE contains CLA Libraries for: Math, Trig, Digital Power, and Motor Control (late 2010)

Delfino's Pipeline ADC allows for highest speed (12.5 MSPS)
Piccolo's hybrid ADC allows even more flexible creation of conversion sequences.



Start of Conversion (SOC) Configuration and Logic Block

- 16 (Piccolo) or 2 (Delfino) programmable Sequencers
- Triggers from Software, CPU timers, any ePWM "Events", and GPIOs
- Multiple conversions can be processed in Round Robin or Priority Modes
- 9 flexible interrupts

Up to 16 Analog Inputs

- 16 channel, multiplexed analog inputs.
- Supports both fixed range and ratio-metric (Piccolo) input range

Analog Mux

- Uses SOC input to select which channels will be processed

Dual Sample and Hold

- Dual sample/hold enable simultaneous sampling or sequencing sampling modes
- Adjustable acquisition window ensures proper sampling

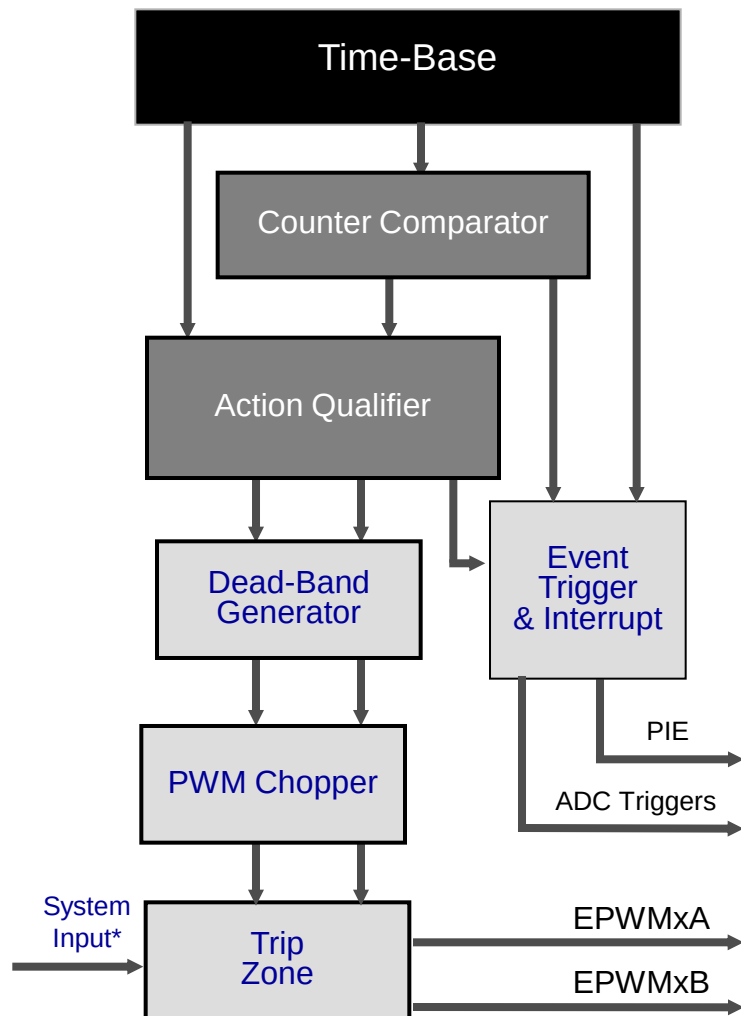
12-bit Analog-Digital Converter

- Fast conversion rate: Up to 5MSPS (Piccolo) or 12.5 MSPS (Delfino)
- Just-in-time interrupts (early interrupts) eliminates context switch latency by calling interrupts before conversion finishes (Piccolo)

Result Registers

- Sixteen result registers (individually addressable) to store conversion values
- Delfino offers DMA access of ADC registers

Each ePWM module has two outputs, EPWMxA and EPWMxB (same frequency, independent duty)
Each module is independent frequency, but can be synched or phase delayed



Dedicated 16-bit Time Base

- Uses pre-scaled CPU system clock; Registers are shadowed
- Up, Up-Down, Down-Up; Events: Zero, Period

Counter Comparator (CC)

- Registers and comparators eliminate the need to interrupt the CPU in PWM generation
- Events: Zero, Period, CMPA (rising & falling), CMPB (rising & falling)

Action Qualifier

- At any Event: Set PWMxA/B (High, Low, Toggle, Do Nothing); Trigger a programmable Event or Interrupt

Programmable Dead-Band Generator

- Programmable rising-edge and falling-edge delay

PWM Chopper

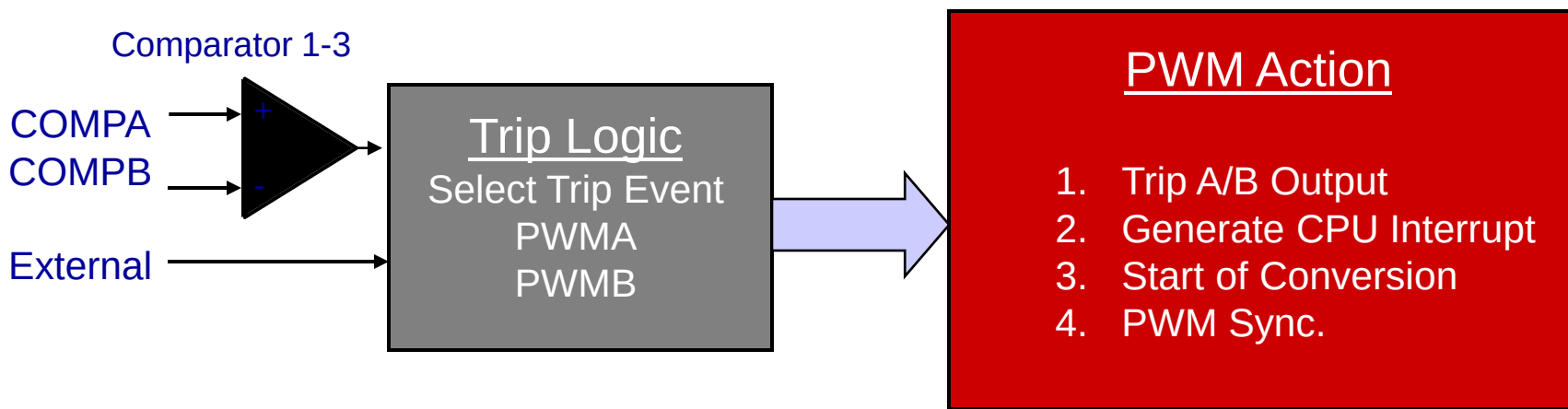
- Allows a high-frequency carrier signal to modulate PWM waveforms
- Programmable chopping frequency, duty cycle, and first pulse width

Programmable Trip Zone Generator

- Quickly overrides PWM signals to Hi, Low, or Hi-Z
- One-shot or cycle-by-cycle (current limiting) operation
- Can generate events, filtered events, or trip conditions

High Resolution on A Channels

- 55ns duty (Delfino) or 150ps duty & frequency (Piccolo) resolution



- Trip Zones operate even if the clocks go missing!
- Each trip pin/event can map to any combination of PWM outputs, Interrupt, SOC, or PWM Synchronization
- Trip event can be synchronous or asynchronous
- User can block trip action for a given offset and window after start of PWM period
- Automatically records counter value in case of trip
- One Shot (Fault) or Continuous (Cycle By Cycle Control)

Lower System Cost / Increased System Reliability

- On-Chip Voltage Regulation
 - On-chip regulator eliminates requirement for external 1.8V rail
 - BOR/POR protection eliminates requirement for external supervisor
 - Eliminates any start-up glitches on PWM outputs
- Dual On-Chip Oscillators
 - No external clock circuitry required
 - Independent time bases for main CPU and Watchdog support standards such as IEC-60730
 - Auto PWM Trip on Fail
- Analog Comparators
 - Trip PWM Outputs, Generate Interrupts, Sync PWM Outputs, Generate ADC SOC, Route to GPIO Pins
- Analog-to-Digital Converter
 - Continuous sampling up to 5 MSPS
 - Ratio metric across full 3.3V input range
 - No support pins
- High Resolution PWM
 - High Resolution Duty Cycle Modulation with 150ps Steps
 - High Resolution Frequency Modulation with 150ps Steps
- GPIO Input Digital Filters
 - Removes Noise
 - No external filters saves cost

Based On The Required Dynamic Range Or Resolution

31

C

S I I I I I I I I I I I I I I . Q Q Q Q Q Q Q Q Q Q Q Q Q Q Q (Q15)

GLOBAL_Q	Max Val	Min Val	Resolution
28	7.999 999 996	-8.000 000 000	0.000 000 004
24	127.999 999 94	-128.000 000 00	0.000 000 06
20	2047.999 999	-2048.000 000	0.000 001

The user selects a “Global Q” value for the entire application:

```
#define GLOBAL_Q 24 // set in "IQmathLib.h" file
_iq Y, M, X, B;

Y = IQmpy(M,X) + B; // all values are in I8Q24
```

The user can also explicitly specify the IQ value to use:

```
_iq20  Y, M, X, B;
Y = __IQ20mpy(M,X) + B;  // all values are in I12Q20
```

DMCLib uses Q24 as a standard, can be tuned to your needs

- C SOURCE, Fixed & Float Versions
- Modular macros with variable inputs and variable outputs
- At initialization all variables are defined and outputs of one block are set as inputs to the next
- At run-time the macro functions are called
- Complete documentation – including equations and theory – is provided for every module

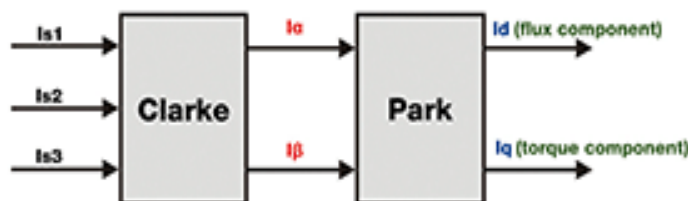
Peripheral Blocks

- BLDC PWM
- PWM Full Compare
- HALL CAP
- HALL GPIO
- QEP
- PWM DAC
- Data Log

Control Blocks

- PID
- Speed Estimators
- Speed Freq/Period
- Clarke / iClarke
- Park / iPark
- SVGen
- Commutation Trig
- Impulse
- Mod6 Counter
- Phase Voltage Calc
- Ramp Controllers
- Sliding Mode Observer
- ACI Flux/Speed Estimators
- CLA Versions Q410

Ex: Using “Park” from DMC Library



//initialization code, define macro per library

```

#define PARK_MACRO(v)\
    v.Ds = _IQmpy(v.Alpha,v.Cosine) +\
    _IQmpy(v.Beta,v.Sine);\
    v.Qs = _IQmpy(v.Beta,v.Cosine) -\
    _IQmpy(v.Alpha,v.Sine);
  
```

//incremental build code, connect outputs and inputs

```

    park1.Alpha = clarke1.Alpha;
    park1.Beta = clarke1.Beta;
  
```

//run-time code, call the function

```

    PARK_MACRO(park1)
  
```

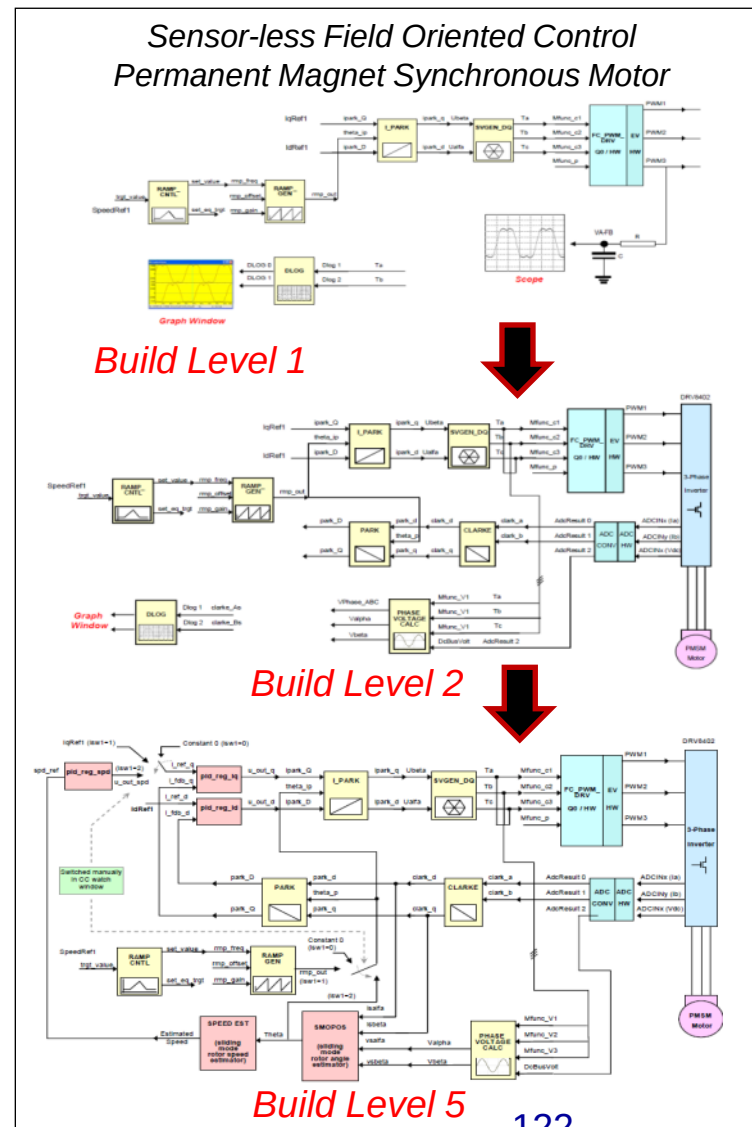
Library Blocks are “Wired” into **Systems**
Inside a full System **Framework (OS)**

1. Initialization of Device
2. **Instance & Initialize** Blocks
3. Run Framework / OS
4. Incremental Build Levels
5. **Connect & Call** Blocks

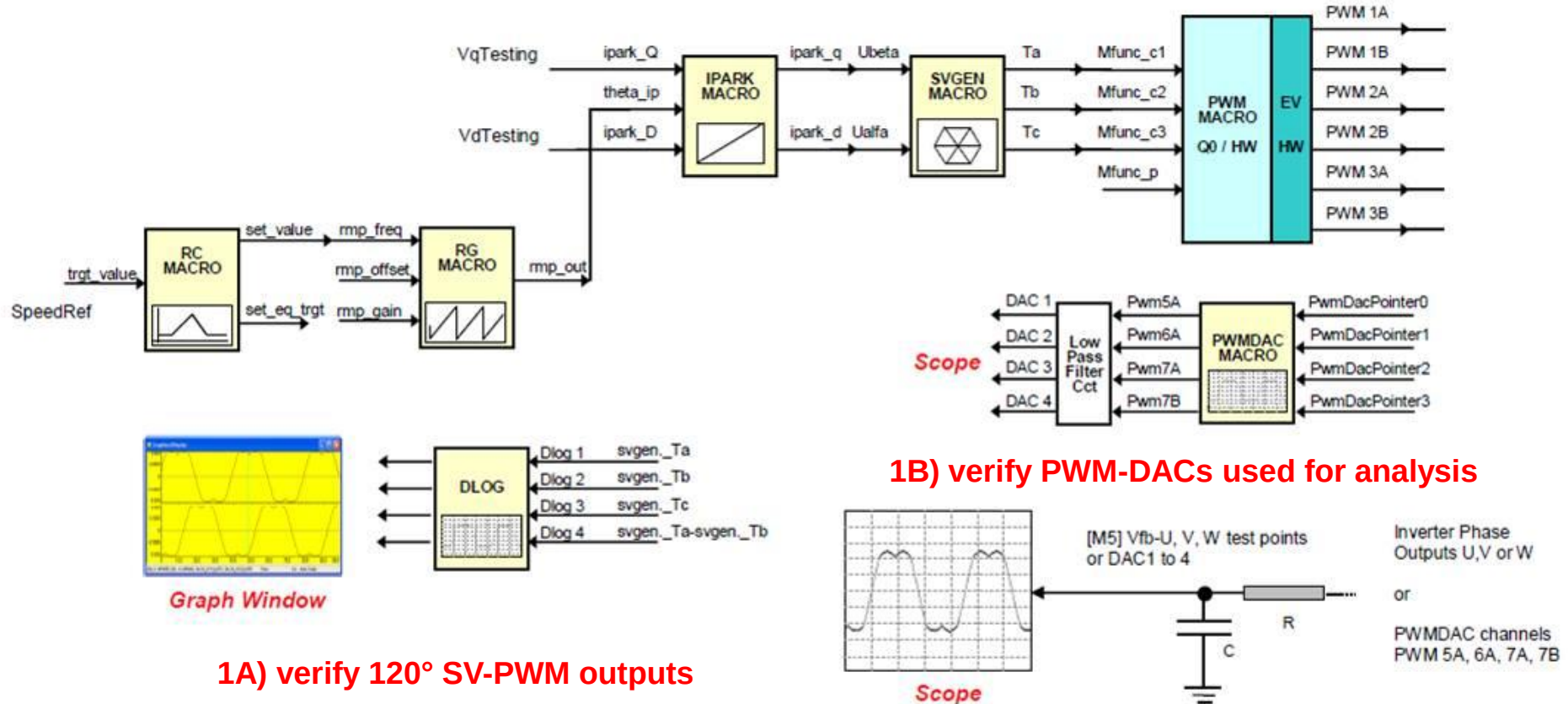


Example:
Sensor-less FOC Motor Control

- Build Level 1:** Verify Space Vector and PWMs
Build Level 2: Verify ADC conversion & Phase Voltage
Build Level 3: Tune PID for current control
Build Level 4: Verify Sensorless estimator
Build Level 5: Tune PID speed control
Build Level 6: Close all loops

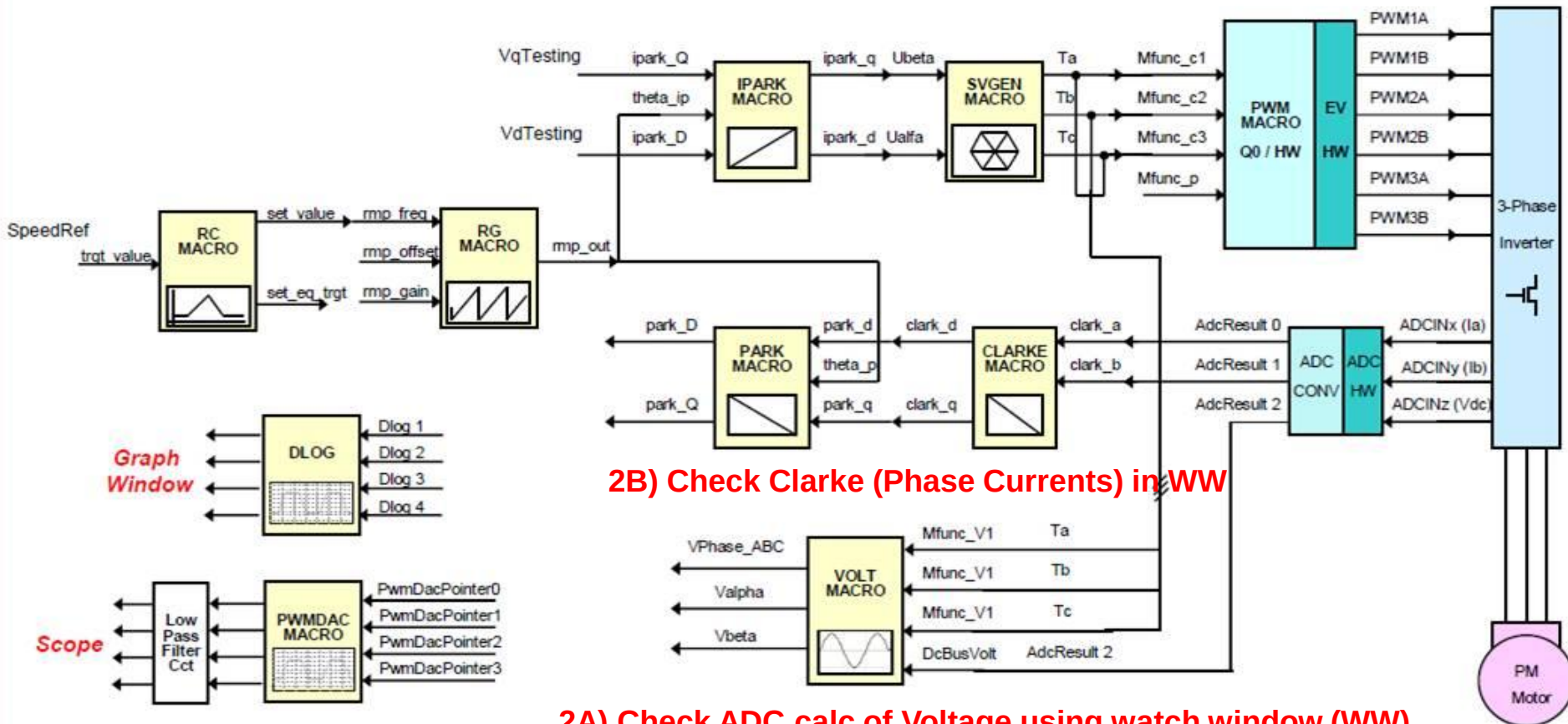


Level 1 - Incremental System Build Block Diagram



Level 1 verifies the target independent modules, duty cycles and PWM update. The motor is disconnected at this level.

Level 2 - Incremental System Build Block Diagram



2B) Check Clarke (Phase Currents) in WW

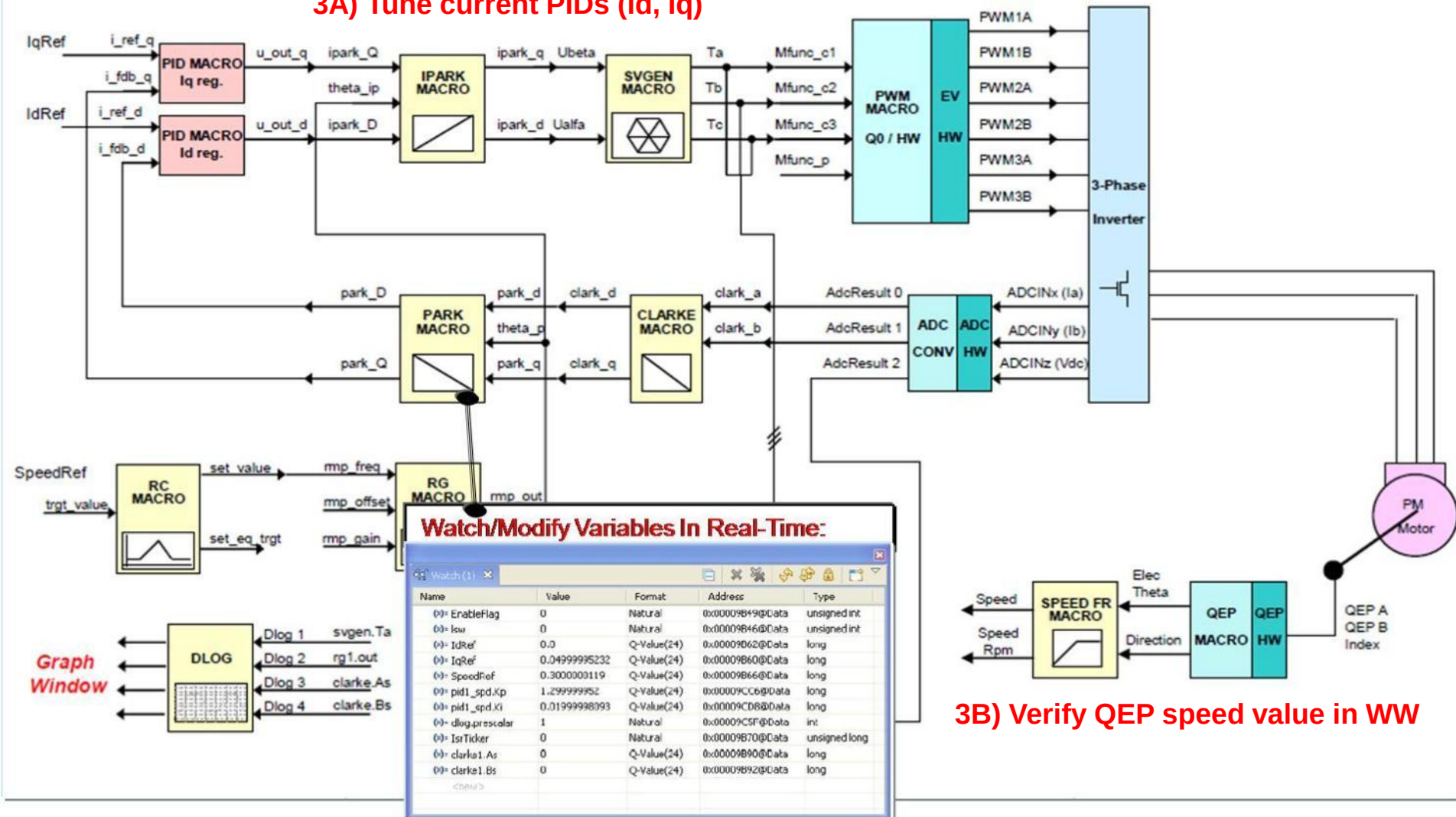
2A) Check ADC calc of Voltage using watch window (WW)

2C) Calibrate phase current off-set to enable low load sensorless

Level 2 verifies the analog-to-digital conversion, offset compensation, clarke / park transformations, phase voltage calculations

Level 3 - Incremental System Build Block Diagram

3A) Tune current PIDs (Id, Iq)



3B) Verify QEP speed value in WW



Traditional debugging (Stop Mode)

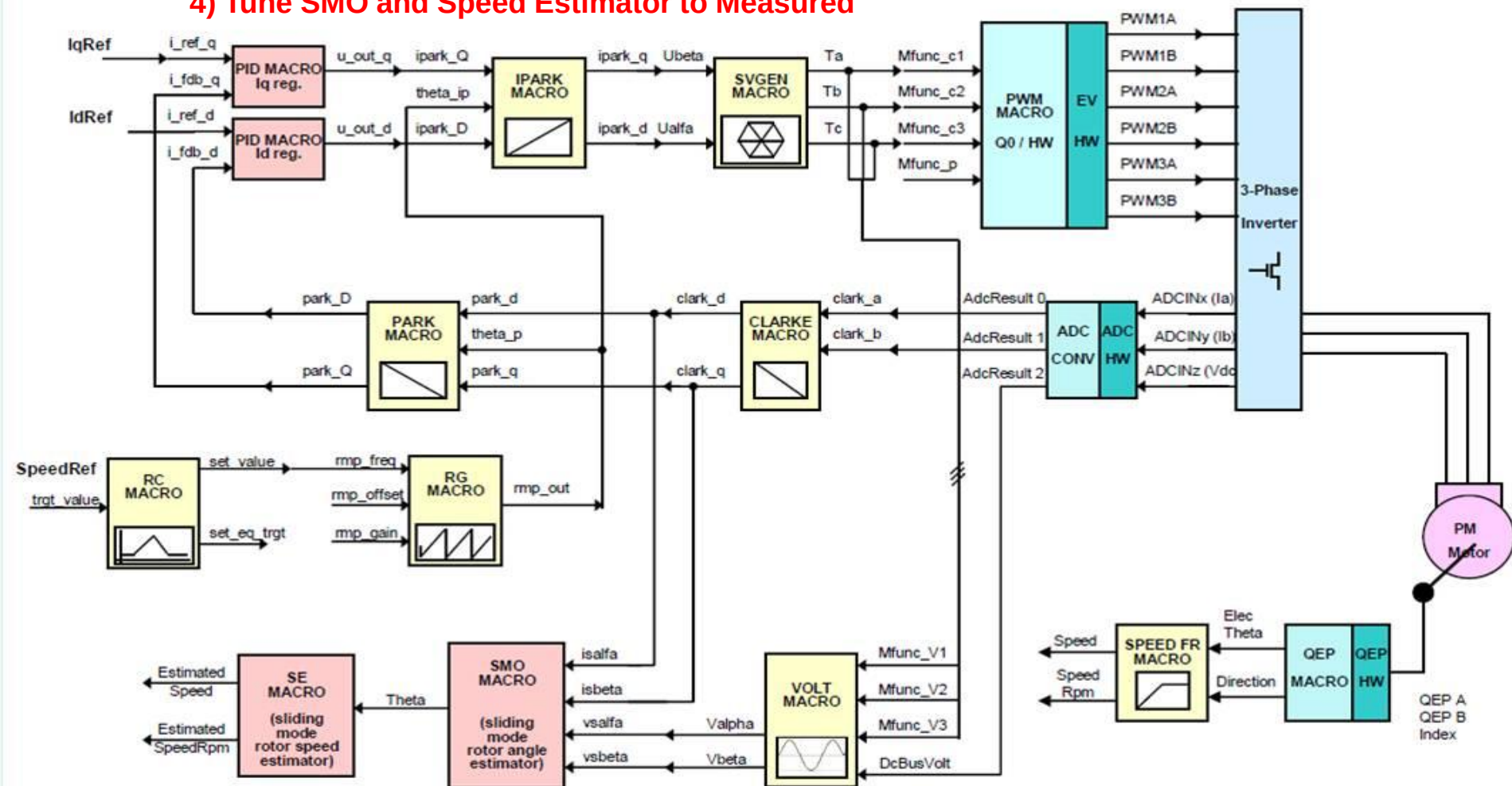
- stops all threads and prevents interrupts from being handled
- makes debugging real-time systems extremely difficult

C2000 Real-time Mode:

- real-time, non-intrusive, continuous
- Does not require use of target memory, special interrupts, or SW intrusiveness
- Allows time critical interrupts to be marked for special treatment (high priority)
- Allows time-critical interrupts to be serviced while background program execution is suspended
- Included on all C2000 devices and integrated with Code Composer Studio

Level 4 - Incremental System Build Block Diagram

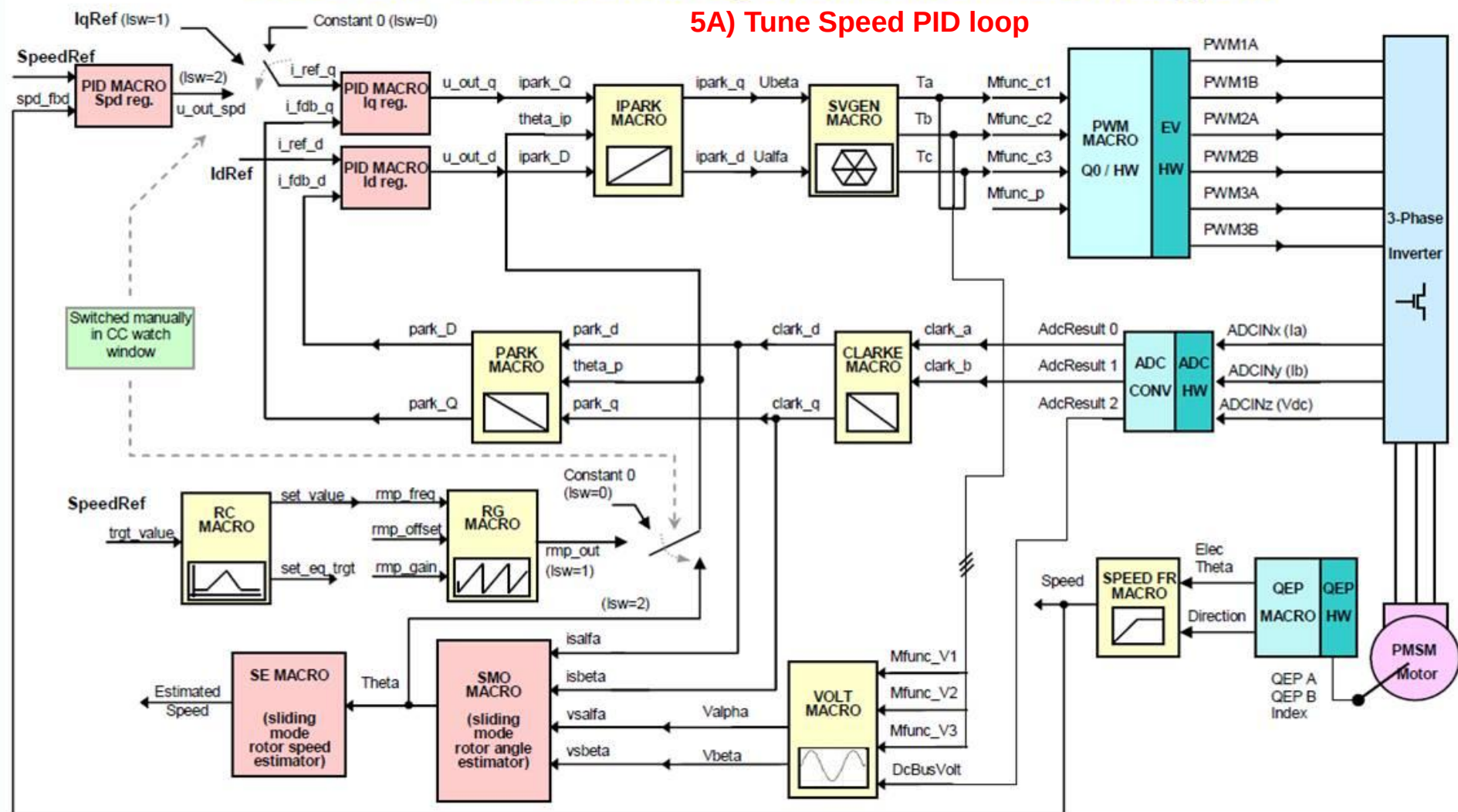
4) Tune SMO and Speed Estimator to Measured



Level 4 verifies the rotor position and speed estimation performed by SMO and speed estimation modules.

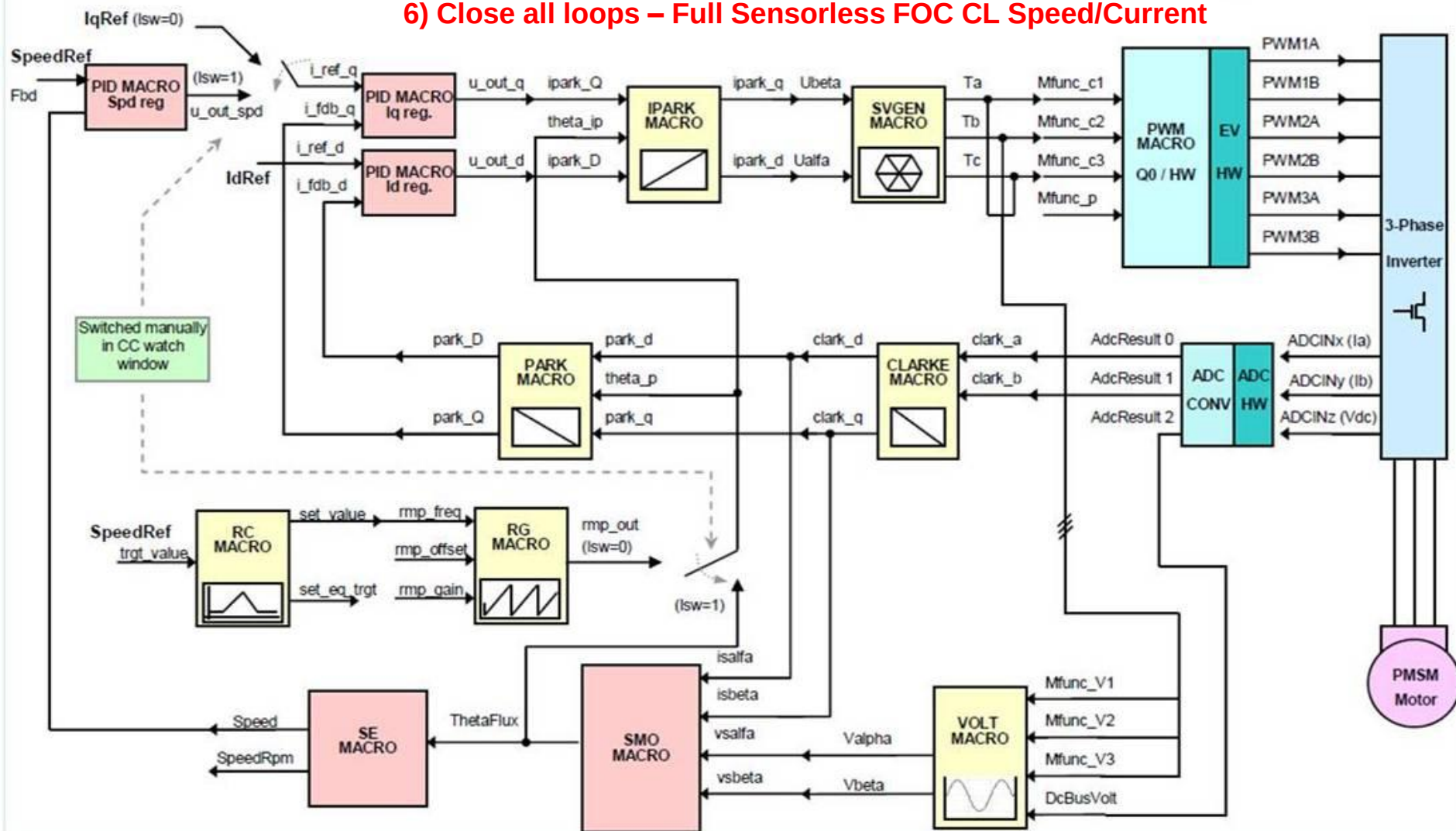
Level 5A - Incremental System Build Block Diagram

5A) Tune Speed PID loop

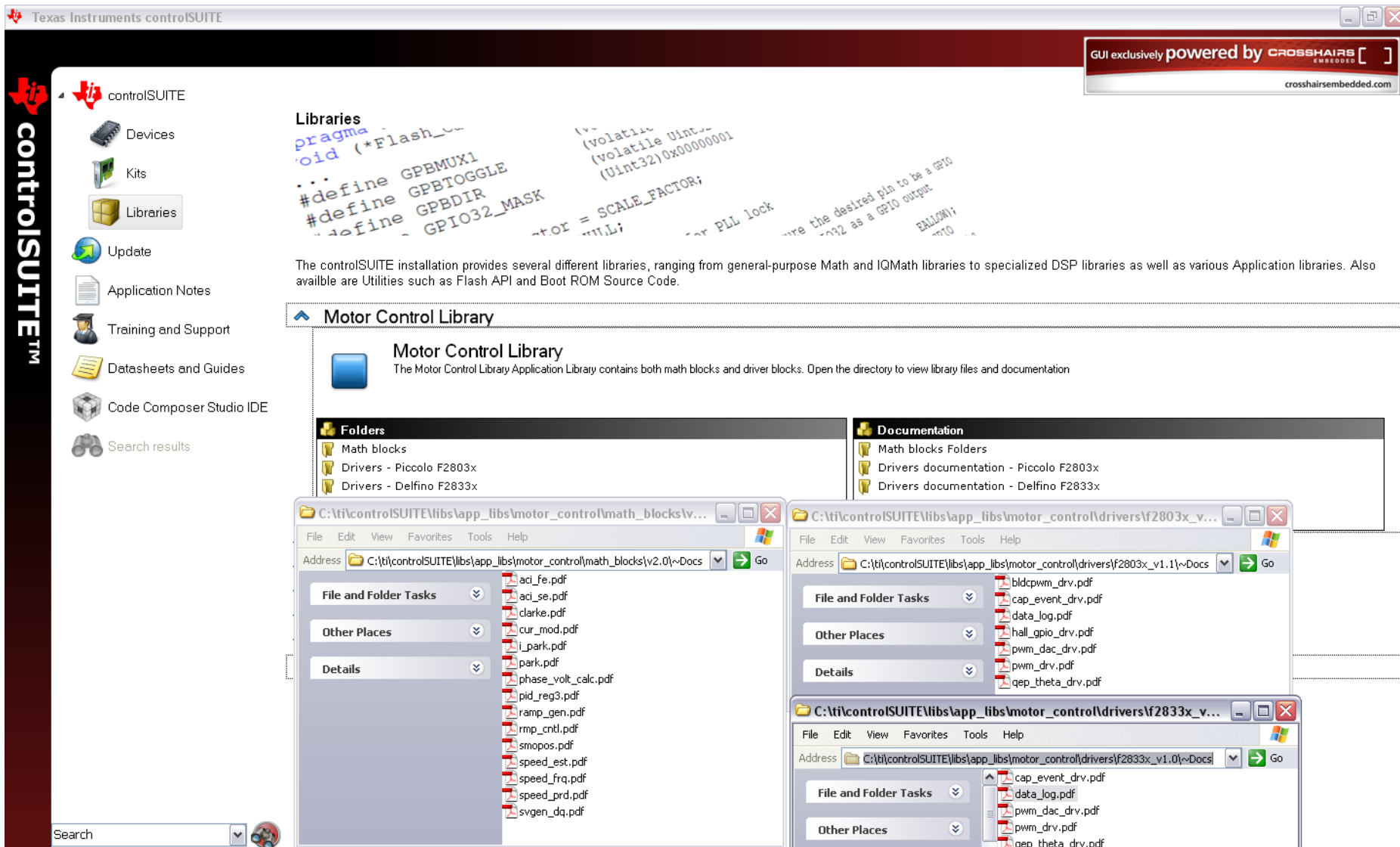


Level 6 Incremental System Build Block Diagram

6) Close all loops – Full Sensorless FOC CL Speed/Current



controlSUITE: One Stop Shopping



controlSUITE™

GUI exclusively powered by CROSSHAIRS EMBEDDED []
crosshairsembedded.com

Libraries

```

#pragma
void (*Flash_...
...
#define GPBMUX1
#define GPBTGGLE
#define GPBDIR
#define GPIO32_MASK
...
... = SCALE_FACTOR;
...
... for PLL lock
... the desired pin to be a GPIO
... as a GPIO output
...
... EALLOW;
...

```

The controlSUITE installation provides several different libraries, ranging from general-purpose Math and IQMath libraries to specialized DSP libraries as well as various Application libraries. Also available are Utilities such as Flash API and Boot ROM Source Code.

Motor Control Library

The Motor Control Library Application Library contains both math blocks and driver blocks. Open the directory to view library files and documentation

Folders

- Math blocks
- Drivers - Piccolo F2803x
- Drivers - Delfino F2833x

Documentation

- Math blocks Folders
- Drivers documentation - Piccolo F2803x
- Drivers documentation - Delfino F2833x

File Explorer Windows:

- C:\ti\controlSUITE\libs\app_libs\motor_control\math_blocks\...**
 - File and Folder Tasks
 - Other Places
 - Details
 - Files: aci_fe.pdf, aci_se.pdf, clarke.pdf, cur_mod.pdf, i_park.pdf, park.pdf, phase_volt_calc.pdf, pid_reg3.pdf, ramp_gen.pdf, rmp_cntl.pdf, smopos.pdf, speed_est.pdf, speed_frq.pdf, speed_prd.pdf, svgen_dq.pdf
- C:\ti\controlSUITE\libs\app_libs\motor_control\drivers\2803x_v1.1\~Docs**
 - File and Folder Tasks
 - Other Places
 - Details
 - Files: bldcpwm_drv.pdf, cap_event_drv.pdf, data_log.pdf, hall_gpio_drv.pdf, pwm_dac_drv.pdf, pwm_drv.pdf, qep_theta_drv.pdf
- C:\ti\controlSUITE\libs\app_libs\motor_control\drivers\2833x_v1.0\~Docs**
 - File and Folder Tasks
 - Other Places
 - Details
 - Files: cap_event_drv.pdf, data_log.pdf, pwm_dac_drv.pdf, pwm_drv.pdf, qep_theta_drv.pdf

www.ti.com/controlsuite

controlSUITE: One Stop Shopping



controlSUITE

Devices

Kits

Libraries

Update

Application Notes

Training and Support

Datasheets and Guides

Code Composer Studio IDE

Search results

GUI exclusively powered by CROSSHAIRS EMBEDDED

crosshairseembedded.com

Hardware Development Tools

C2000 offers a variety of hardware development tools designed to accelerate and simplify the design process. Each tool comes with completely open source hardware schematics and software framework. Combined with controlSUITE software, C2000 tools provide an effective way to evaluate devices and seamlessly transition into.







ControlSTICKS

ControlCARDS

ControlCARD Evaluation Kits

ControlCARD Application Kits

Emulators and Other Tools

Piccolo controlSTICK

High-voltage Motor Control + PFC Kit (v1.3)

Low-voltage Multi Axis + PFC Kit (v1.1)

High Voltage Motor Control and PFC Developers Kit

Main features

- Piccolo F28035 controlCARD
- PFC and dual motor control baseboard with onboard isolated USB JTAG emulation
- 24V DC desktop power supply
- 55W permanent magnet motor
- Code Composer V3.3 with 32KB code size limit
- Detailed example software and documentation
- Complete hardware schematics, gerber files, etc

Example projects (To run, import in CCSv4)

2xPM Sensorless

Documentation

Sensorless FOC of 2xPM

How To Run Guide

Hardware Guide

Online contents

Low Voltage Motor Control Kit Info Page

All C2000 kits and tools

Interactive MCU Selection Tool

Folders

Hardware Development Package

Digital Motor Control Library

Kit Folder

LED Lighting DC/DC Kit (v2.2)

High Voltage Power Factor Correction Kit (v1.0)

C2000 controlCARDS

131

Graphical Development & Code Gen for C2000

Metatools

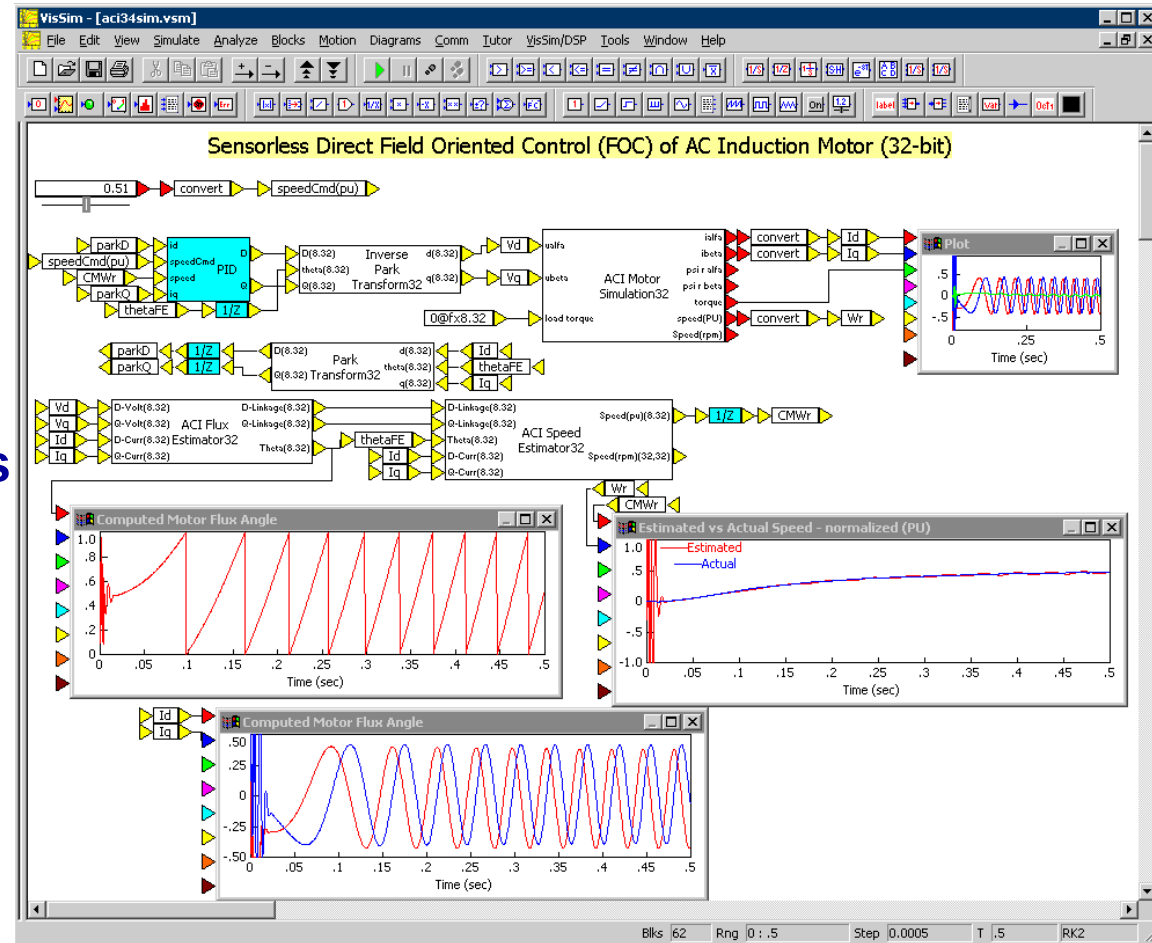
Visual Solutions

VisSim

The Mathworks

Embedded Target

- DMC and Peripheral Blocks
- Simulation and Modeling
- Auto-Tuning
- Code Generation
- Interface to CCStudio IDE
- Works seamlessly with TI Dual-Axis Kit



C2000 Motor Control Summary

Methodologies

High energy efficiency via Advanced Control

- Variable speed Real-time control
- Better dynamic and transient control

Broadest MCU Architecture

- 40-300MHz Fixed & Floating Point
- Parallel FP CLA for fastest loops
- Single Cycle 32x32-bit MAC
- Fast interrupts
- Flexible & Fast interrupt system
- Real-time debugging
- Best in class ADC performance

Piccolo Family for Lowest System Cost

- High Level of Integration
- Integrated Dual OSC, VREG, Watchdogs
- Limited life support
- No external GPIO filters needed

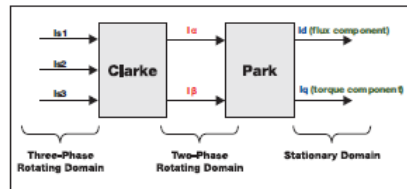
C2000 Digital Motor Control Gives You MORE

C2000 based motor control systems enable energy and cost savings throughout products

- | | |
|--------------------------|-------------------------------|
| - Variable speed control | → MORE efficient motors |
| - Field-oriented control | → MORE efficient control |
| - Space vector PWM | → MORE efficient power stage |
| - Sensor-less control | → MORE cost effective |
| - Multi-axis control | → MORE motors per controller |
| - Integrated digital PFC | → MORE system functions |
| - Meeting IEC standards | → MORE reliable and robust |
| - Broadest MCU portfolio | → MORE products, one platform |

Incremental Build Based Projects

- Incremental section of code built each level
- Verify each portion of their system
 - PWMs, feedback, calculations
 - Control laws, inner/outer loop, supervision
- Critical in motor control with so many different system variables



Application Libraries

Specialized, application specific software functions

- Modular macros with variable inputs and outputs
- At initialization all variables are defined and outputs of one block are set as inputs to the next
- Complete documentation – including source code, use and technical theory – is provided for every module

Digital Motor Control Library

Transforms and Estimators

- Clarke, Park, SMObsolver, Phase Voltage, Resolver, Flux, Speed Calculators and Estimators

Control

- Signal Generation, PID, BEMF Commutation, Space Vector Generators

Peripheral Drivers

- Different modes and topology support
- ADC, PWM, Encoders, Sensor Captures

Ex: Using "Park" from DMC Library

//Initialization code, define macro per library

```
#define PARK_MACRO(v)
v.Ds = _IQmpy(v.Alpha,v.Cosine) + _IQmpy(v.Beta,v.Sine);
v.Qs = _IQmpy(v.Beta,v.Cosine) - _IQmpy(v.Alpha,v.Sine);
```

//Incremental build code, connect outputs and inputs

```
park1.Alpha = clarke1.Alpha;
park1.Beta = clarke1.Beta;
```

//run-time code, call the function

```
PARK_MACRO(park1)
```

Solutions

Low Voltage

Dual Axis

DMC + PFC Kit

TMDS1MTRPFCKIT

TMDS2MTRPFCKIT

\$369/\$399

High Voltage

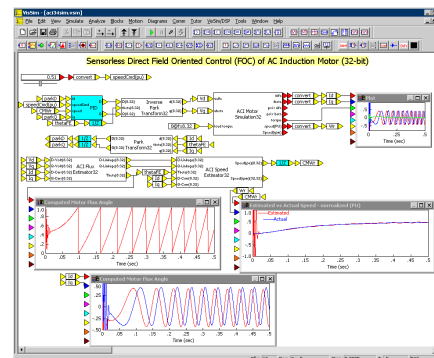
DMC + PFC Kit

TMDSHVMTRPFCKIT

\$599



**Quick Start
GUI with all
projects
Flashed
in MCU**



Matlab & VisSim Integration

Simulation, Modeling, Loop Optimization, Graphical Development, Peripheral Abstraction, Auto Code Gen, Works with TI Hardware

PN: tmds2mtrpfckit



LABS

The lab manual for this course is available at the location you are watching this video, or through controlSUITE. It uses the Piccolo Low Voltage Dual Axis Kit, CCSv4.x, and projects from controlSUITE.

The labs are self-paced and walk you through the incremental build process for Sensorless Field Oriented Control of a Synchronous Motor